



廈門大學

《计算机组成原理》 课程实验报告

姓名： 马鑫

学院： 信息学院

系： 软件工程系

学号： 37220232203780

2026 年 4 月

第 3 次实验 存储系统

1. 实验环境

Windows 系统下运行 Logisim 软件（需安装 JDK）。

2. 实验目的

- (1) 验证 Logisim 中的存储器组件电路、MIPS RAM 存储器电路、MIPS 寄存器文件电路、MIPS 寄存器文件电路（R0=0）、cache 电路（直接映射、全相联映射、2 路组相联映射、4 路组相联映射）。
- (2) 设计存储器扩展电路、寄存器文件电路（4 个寄存器；4 个寄存器、R0=0）、cache 电路（4 个 cache 块、16 个 cache 块），并进行验收。

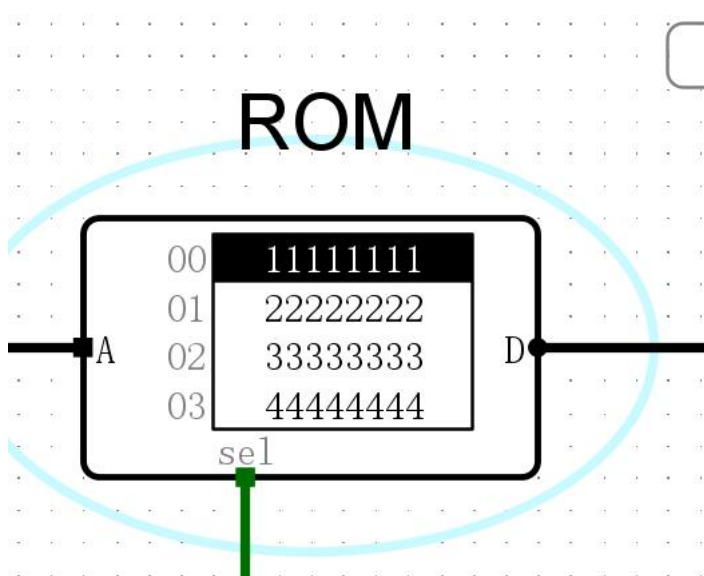
3. 实验内容

3.1 验证实验

- (1) ROM 存储器组件电路、RAM 存储器组件电路（同步模式）、RAM 存储器组件电路（异步模式）、RAM 存储器组件电路（分离模式）。

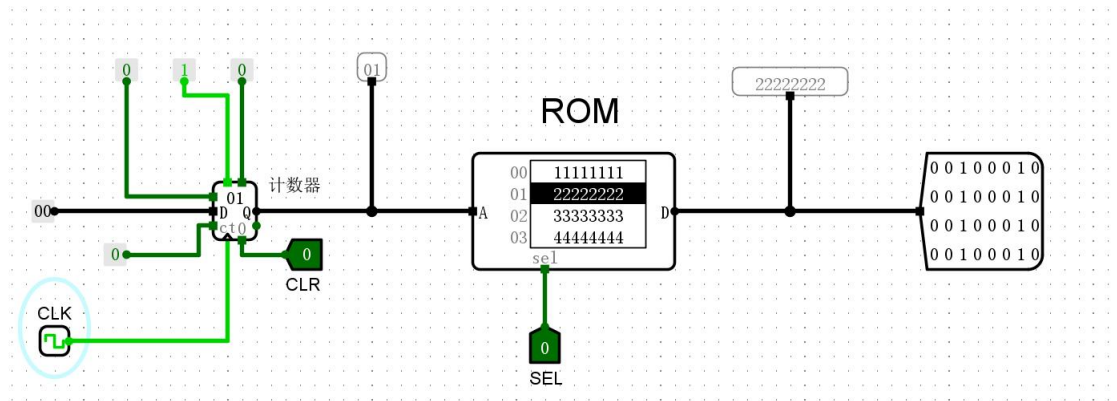
ROM 存储器组件电路：

- ① 向 ROM 中装入内容，具体内容在“ROM 内容.txt”文件中；

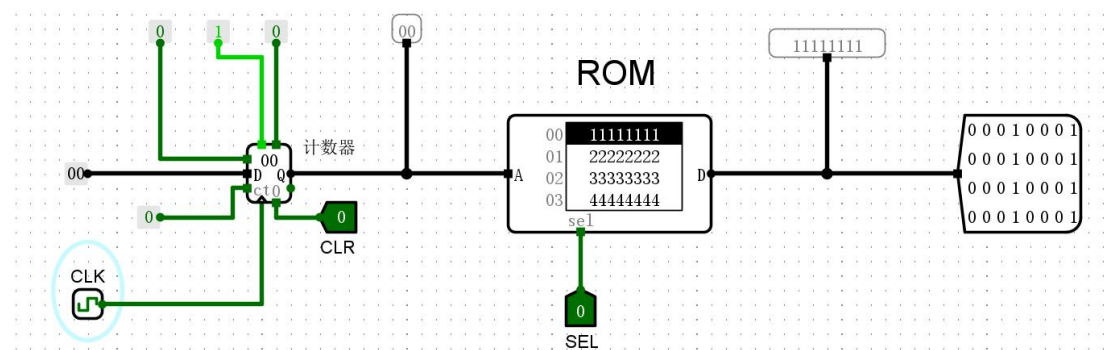


- ② 置 CLR=0、SEL=0；然后点击 CLK，则改变地址 A 的值，观察对应的输出数

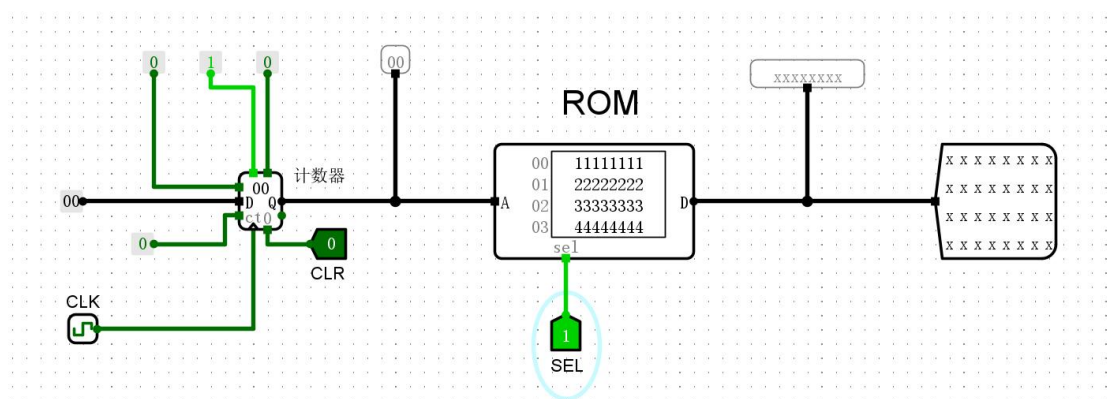
据 D;



③ 置 CLR=1，让地址 A 变为 0；再置 CLR=0，然后点击 CLK，则重新开始；

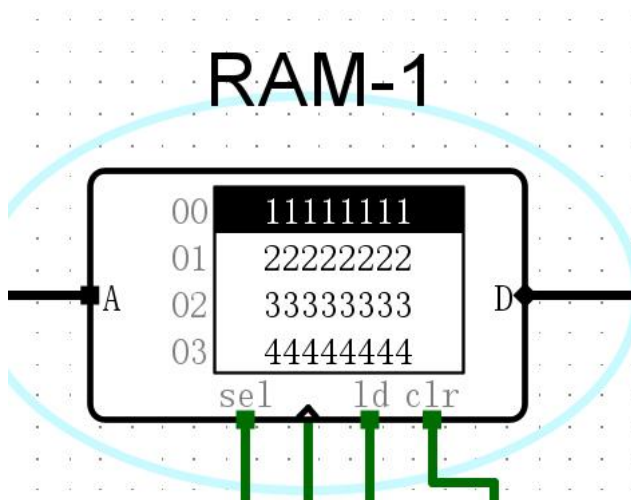


④ 置 SEL=1，则 ROM 存储器不工作。

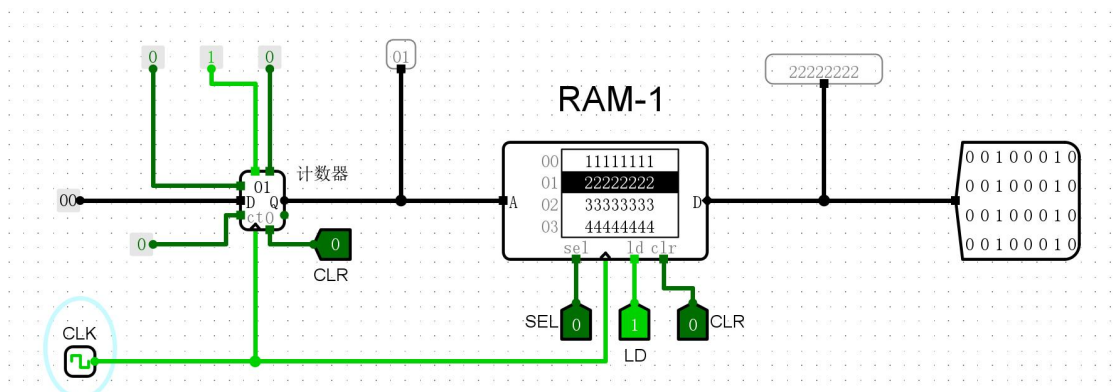


RAM 存储器组件电路（同步模式）：

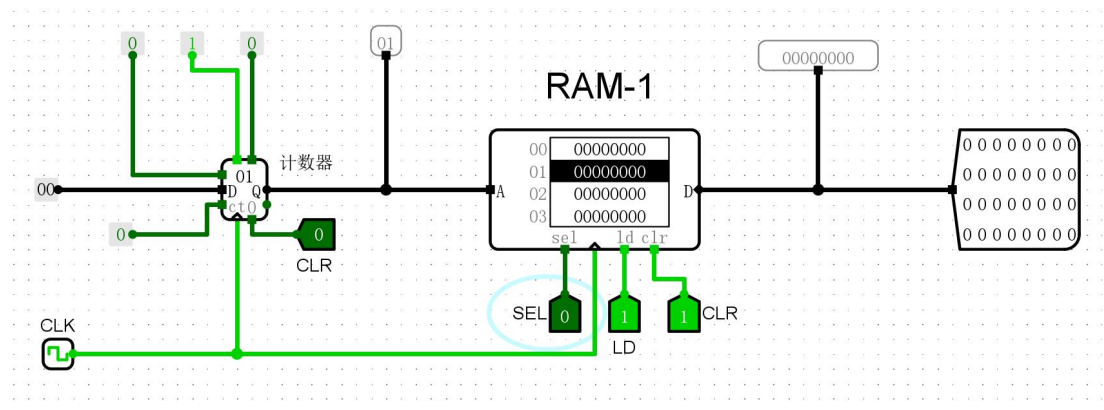
① 向 RAM 中装入内容，具体内容在“ROM 内容.txt”文件中；



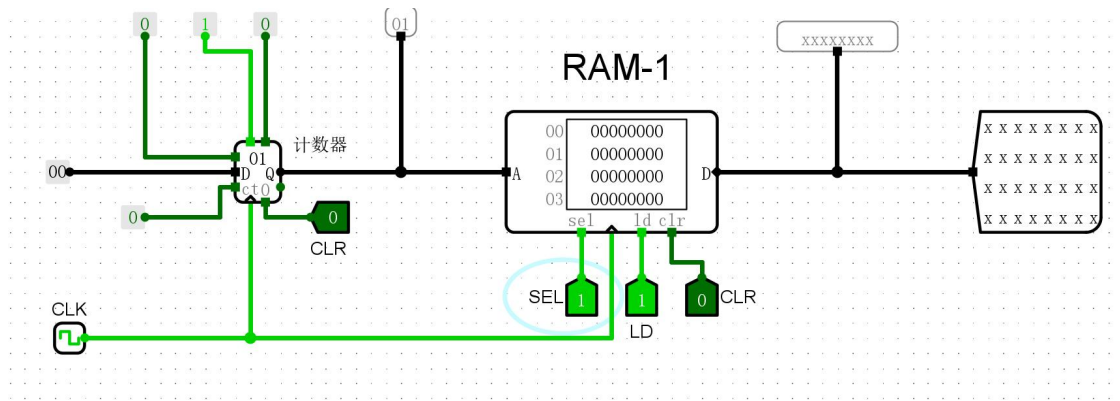
② 置 CLR=0、SEL=0、LD=1；然后点击 CLK，则改变地址 A 的值，观察对应的输出数据 D；



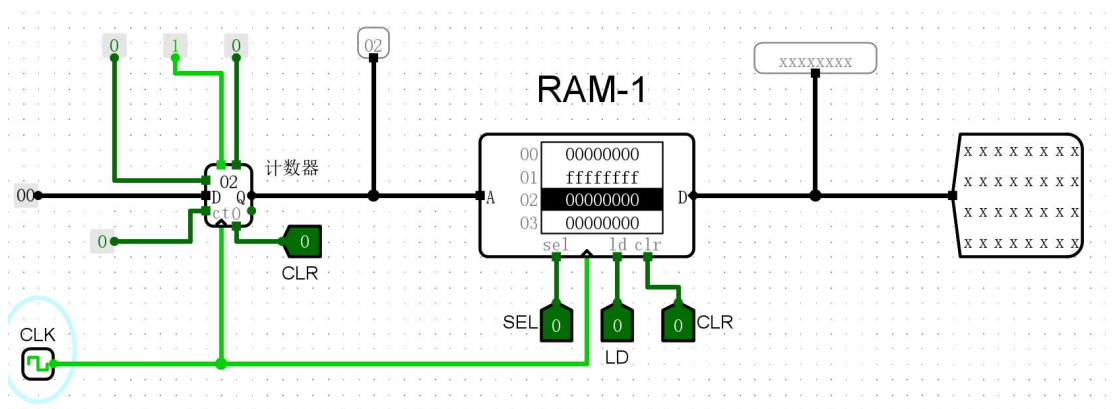
③ 置 CLR=1，RAM 的内容清 0；置 CLR=0，则 RAM 内容不清 0；



④ 置 SEL=1，则 RAM 存储器不工作。

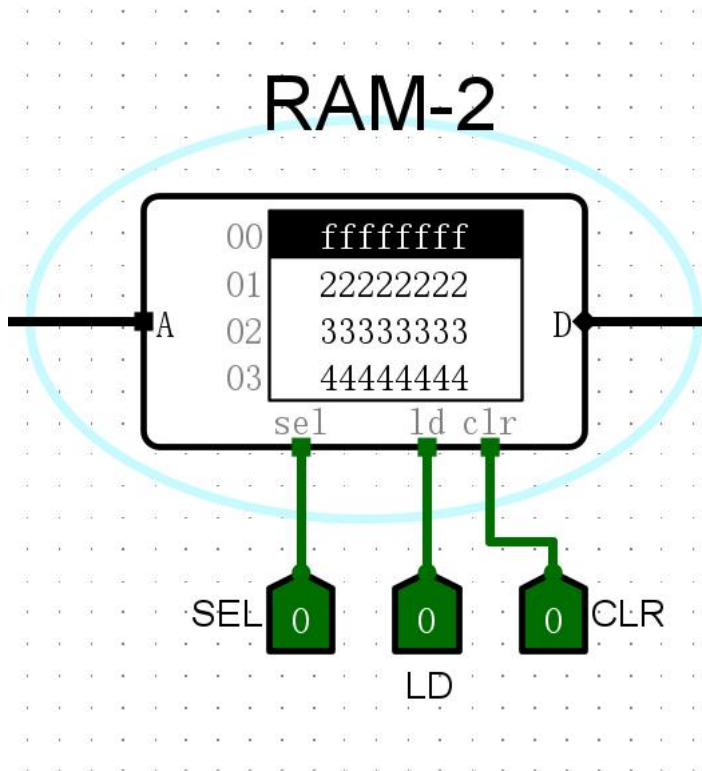


⑤ 置 LD=0，则 RAM 存储器不工作。

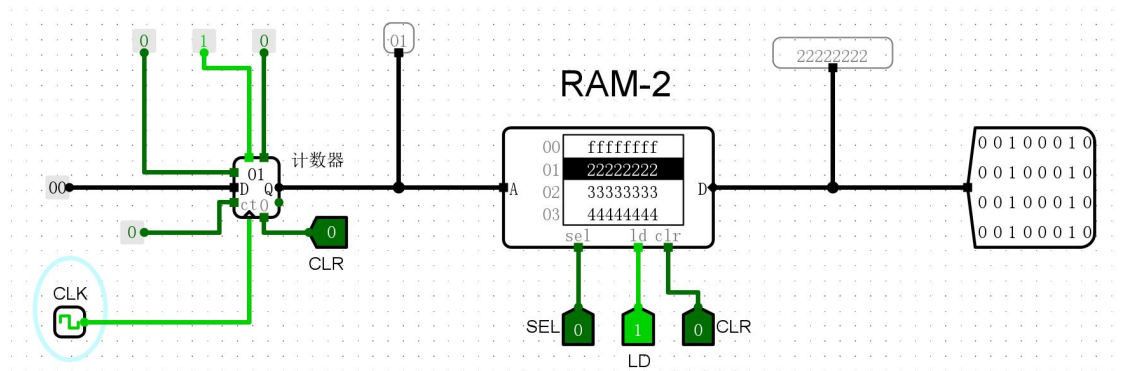


RAM 存储器组件电路（异步模式）：

① 向 RAM 中装入内容，具体内容在“ROM 内容.txt”文件中；

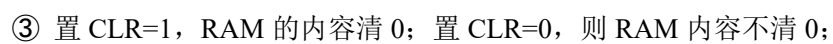
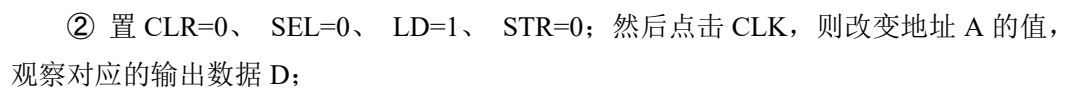


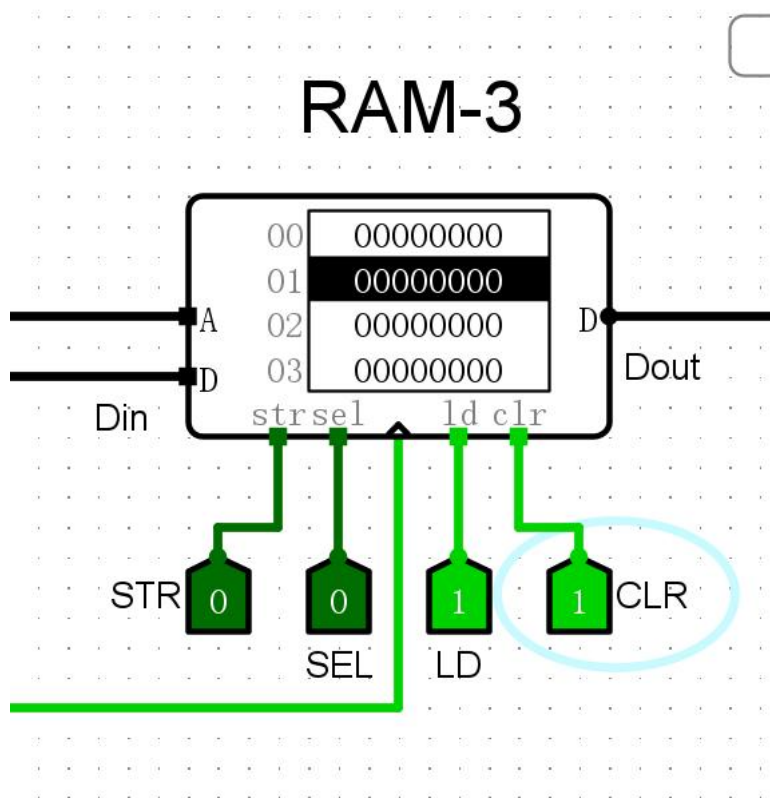
② 置 CLR=0、SEL=0、LD=1；然后点击 CLK，则改变地址 A 的值，观察对应的输出数据 D；



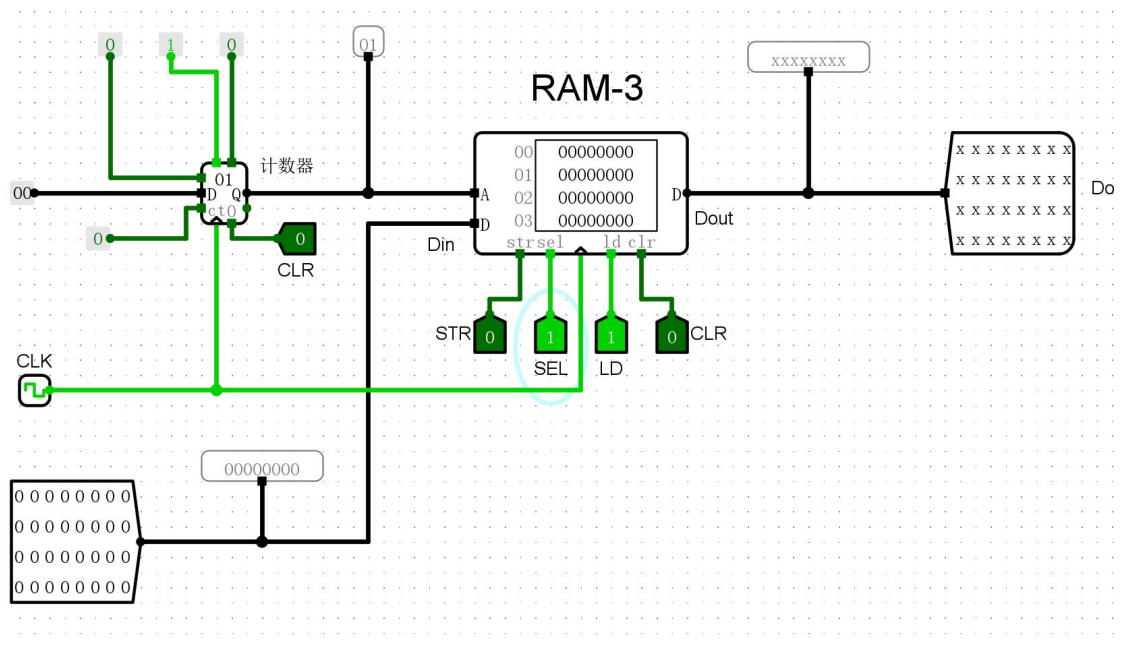
③ 置 CLR=1，RAM 的内容清 0；置 CLR=0，则 RAM 内容不清 0；

① 向 RAM 中装入内容，具体内容在“ROM 内容.txt”文件中；

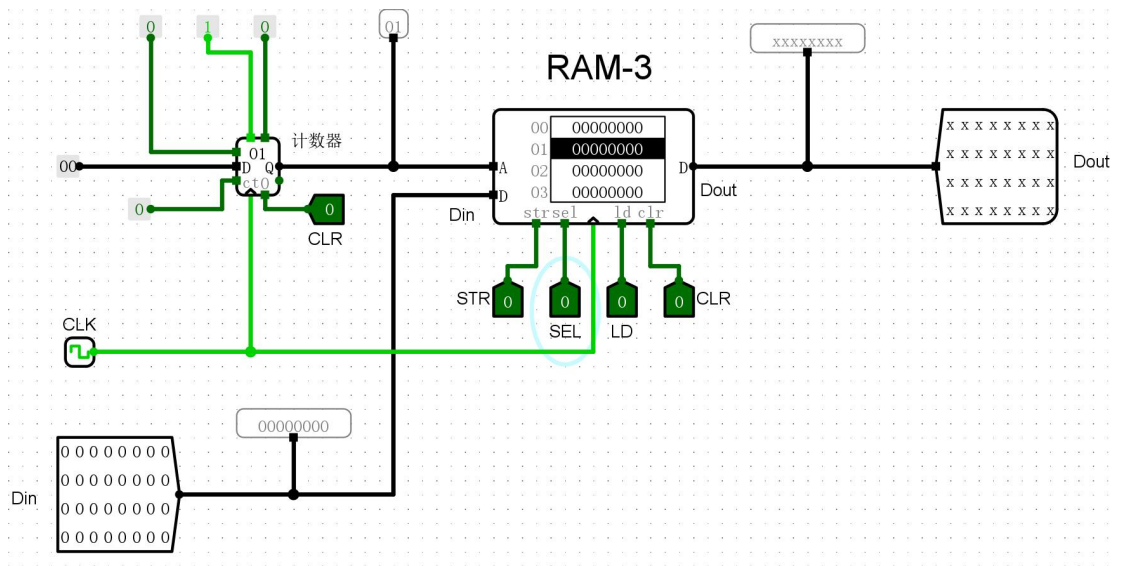




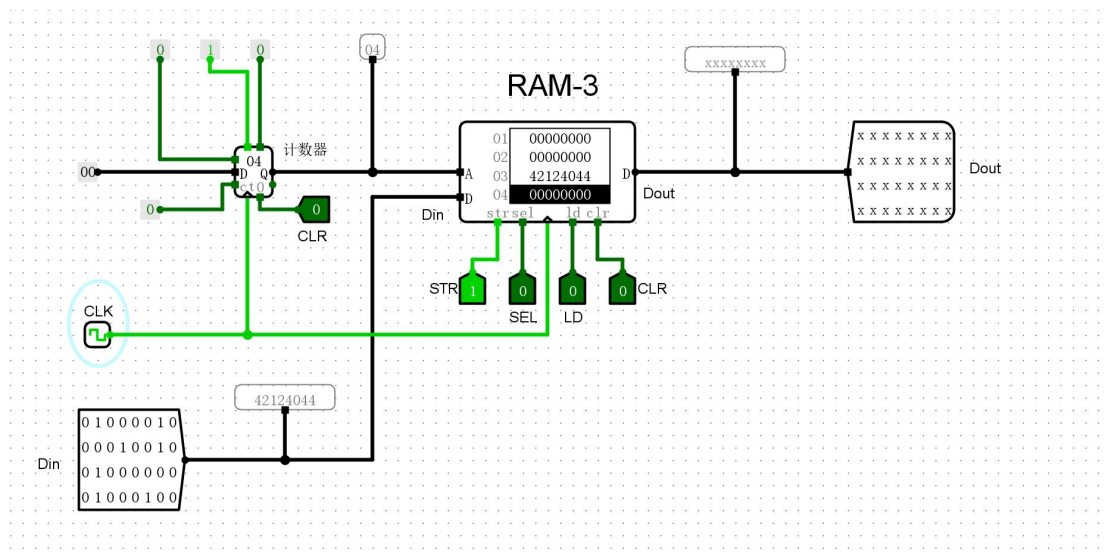
④ 置 SEL=1，则 RAM 存储器不工作。



⑤ 置 LD=0，则 RAM 存储器不工作。



⑥ 置 STR=1，则将 Din 的内容装入地址 A 的 RAM 单元。



回答问题：

1) 基于已有电路，分析在同步与异步模式下，RAM 存储器工作原理有何不同（可结合时钟 CLK 以及控制信号等阐述）？

答：同步模式下其控制端输入较异步模式多了 CLK 时钟控制信号，其依赖该信号来控制数据的读写周期，当外部设备向同步 RAM 发送读写请求时，这些请求会等待下一个时钟周期的边沿才能被执行。异步模式下没有固定的时钟周期要求，只要满足读写操作的时序要求即可。

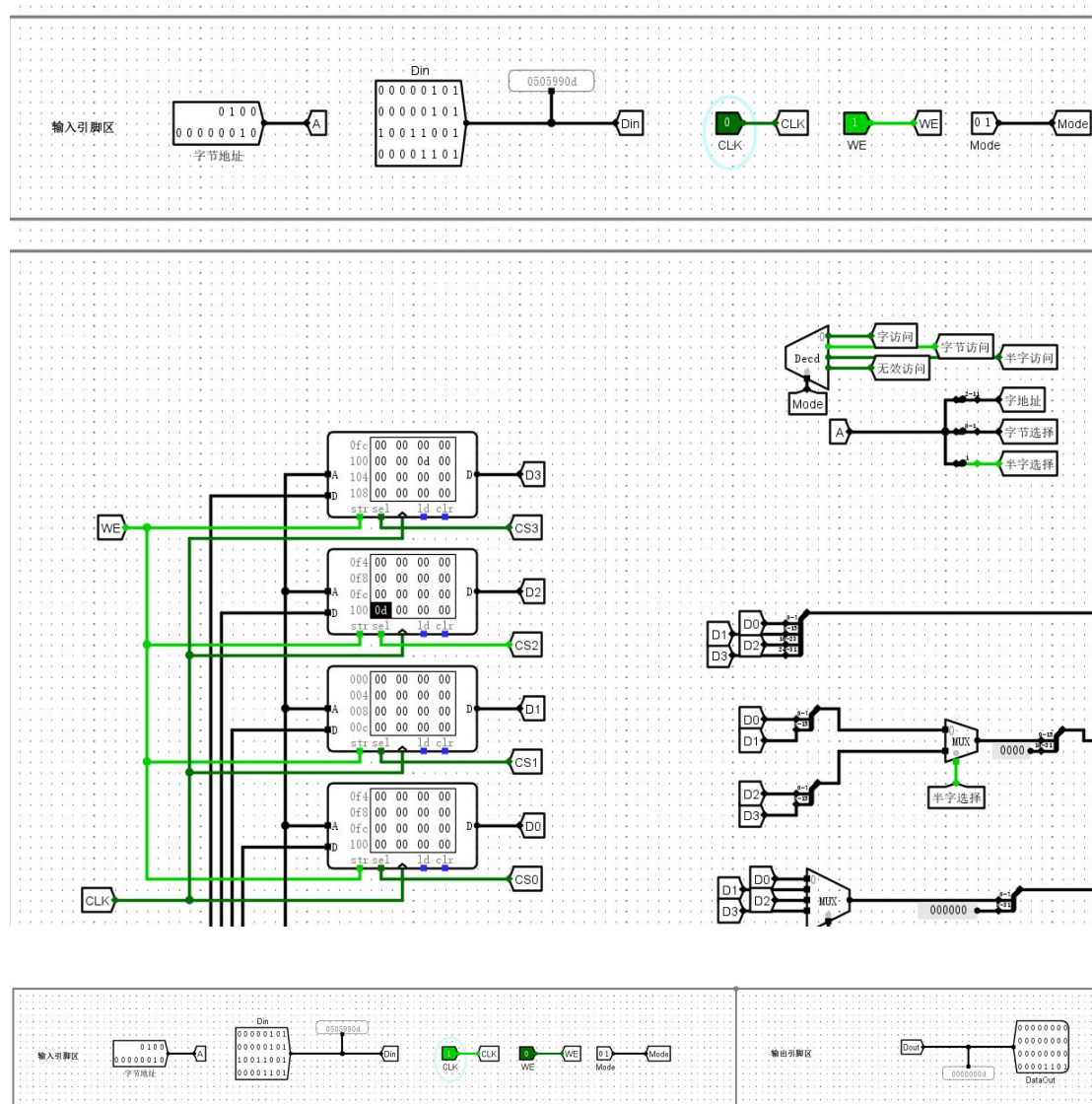
2) 根据已有电路，结合控制信号和 CLK，分析分离模式如何实现只读不写，只写不读以及读写同步的？

答：均需保持 SEL=0，调整 LD 与 STR 开关，点击 CLK 通过时钟控制读写周期。
只读不写：LD=0，关闭输出，STR=1，允许输入，将 Din 的内容装入地址 A 的 RAM 单元；只写不读：LD=1，允许输出，STR=0，不装入；读写同步：LD=1，允许输出，STR=1，允许输入。

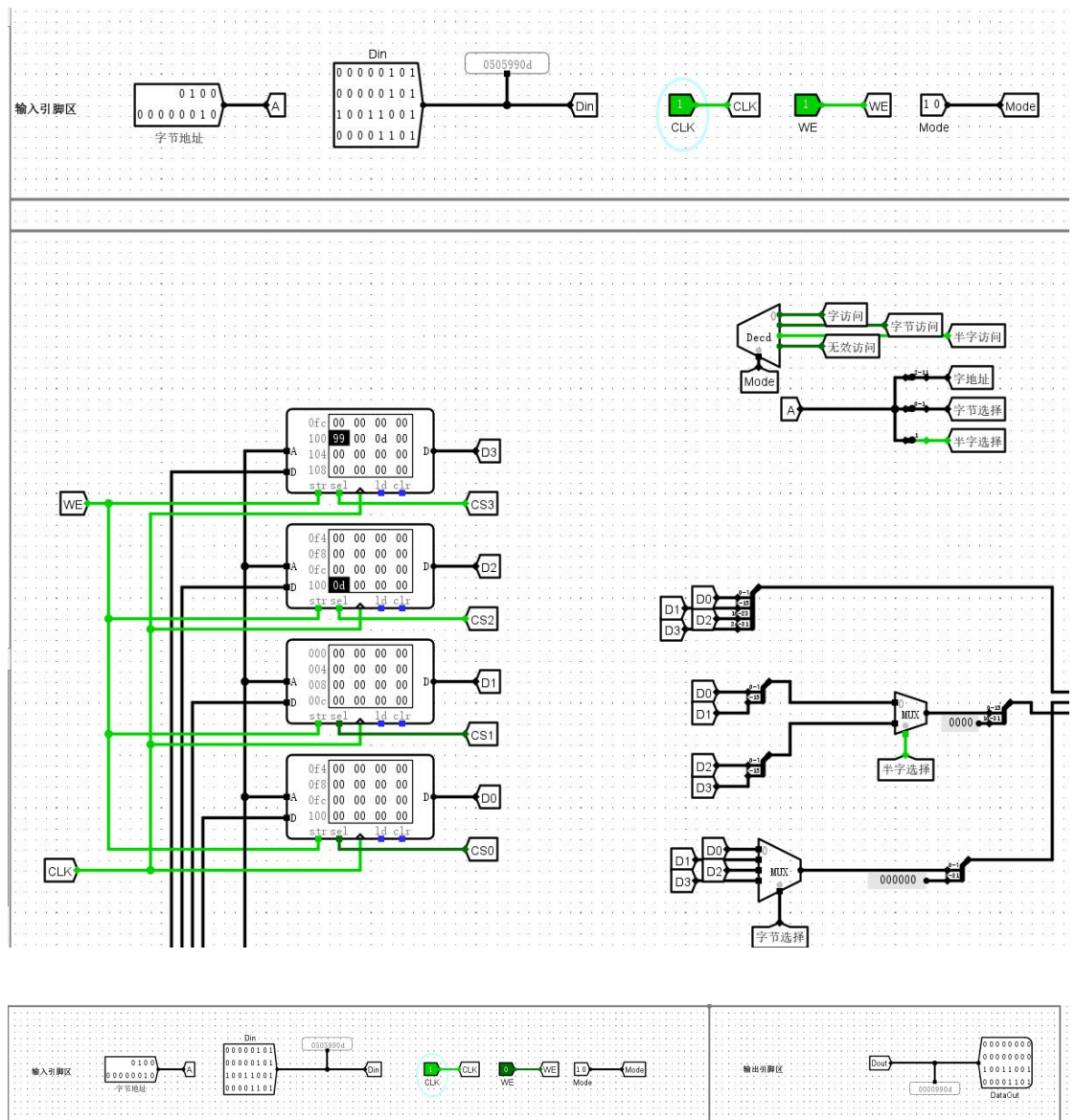
(2) MIPS RAM 存储器电路、简单的 MIPS RAM 存储器测试电路、复杂的 MIPS RAM 存储器测试电路。

MIPS RAM 存储器电路：

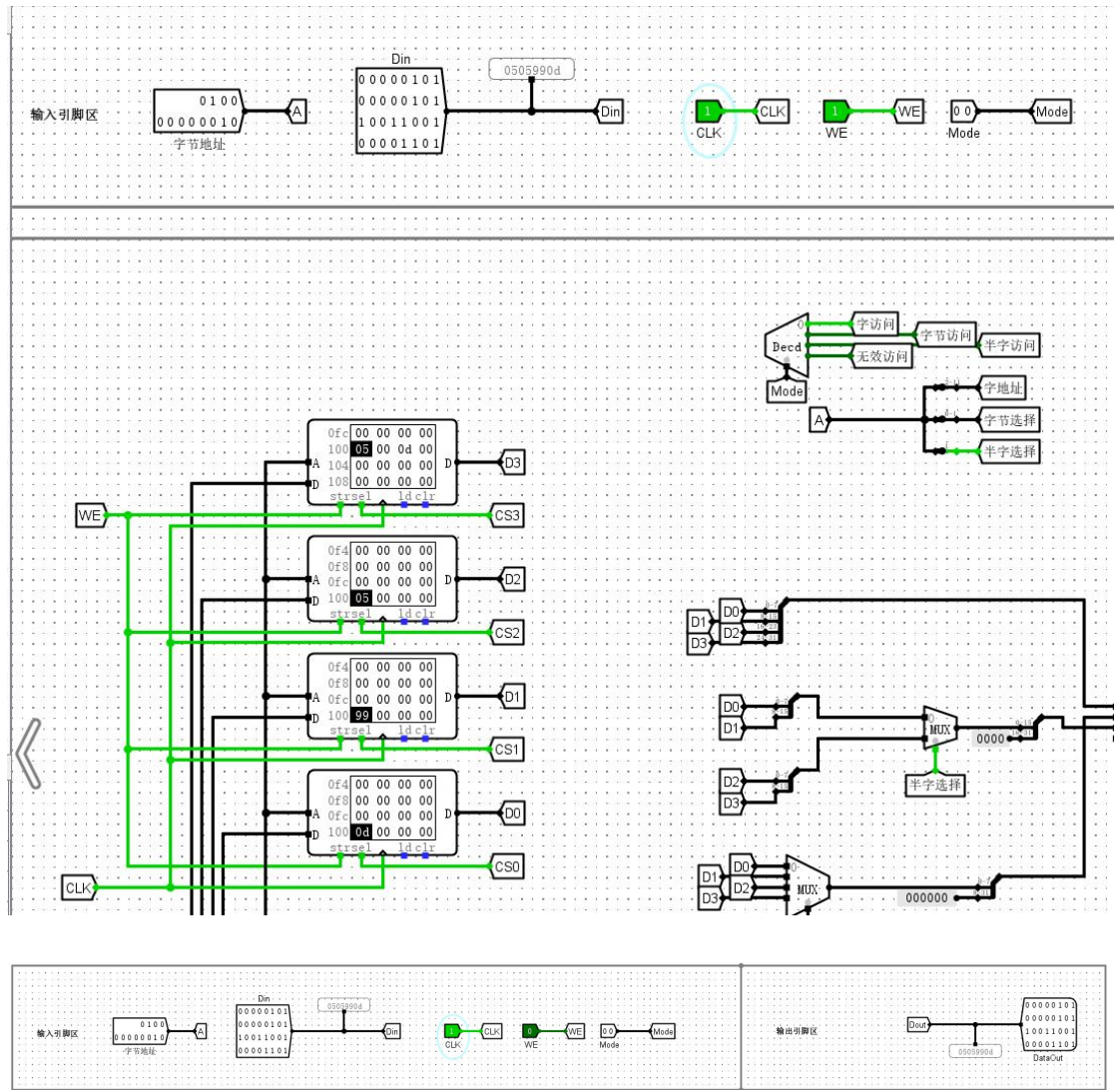
按字节访问：



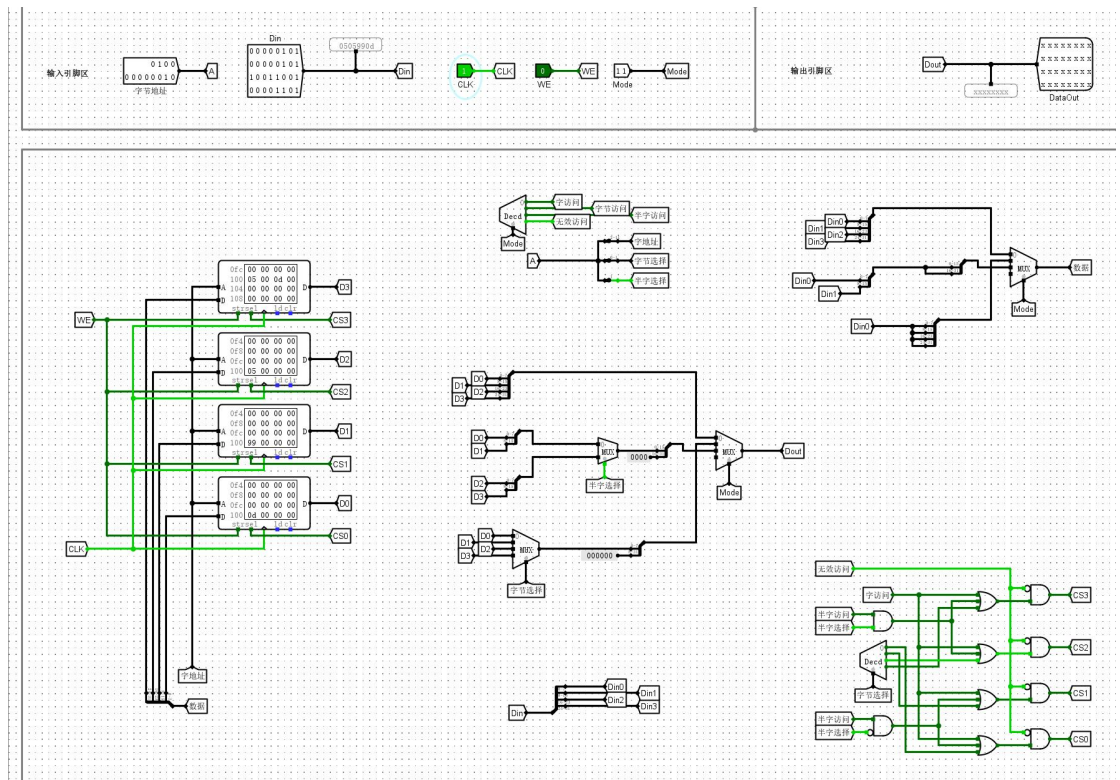
按半字访问：



按字访问:



无效访问：



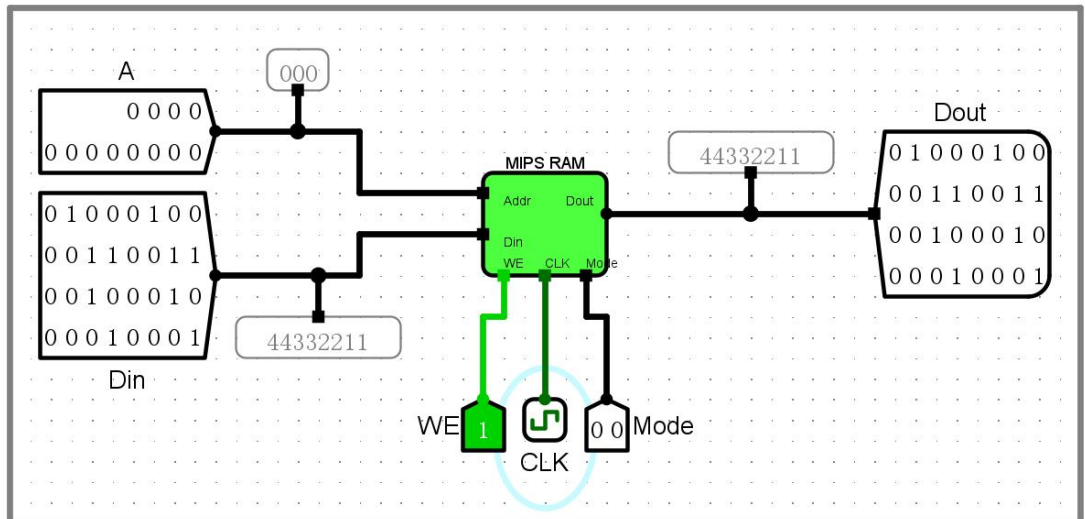
简单的 MIPS RAM 存储器测试电路：

第 1 步：将存储单元 0~ 7，分别写入 11~ 88（均为十六进制，下同）。

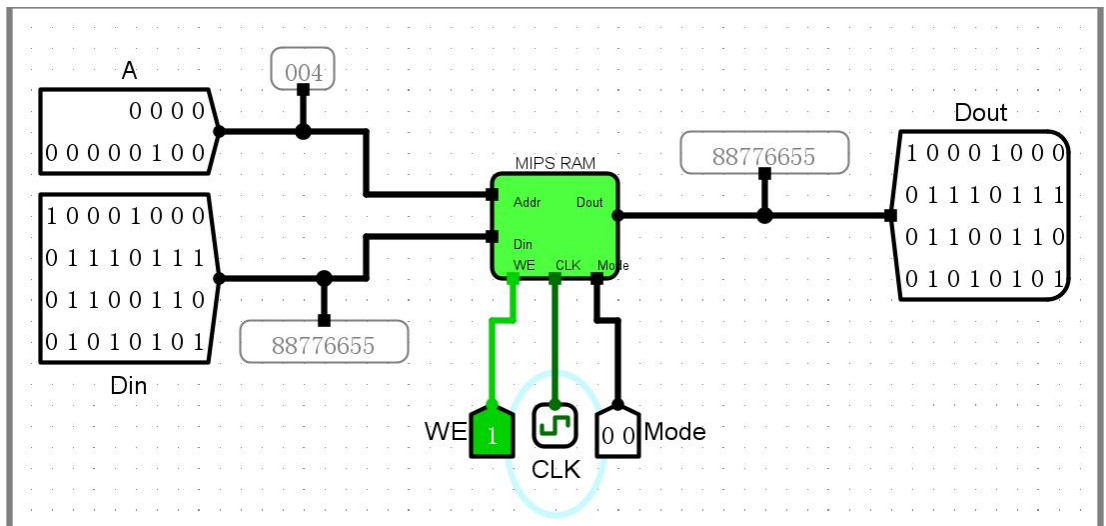
- 置 Mode=00，按字访问；置 WE=1，存储器写



- 置 A=0、 Din=44332211，按 CLK 按钮 2 次，则写入到 0~ 3 单元

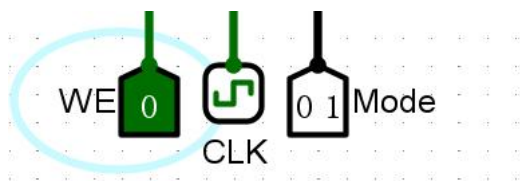


- 置 A=4、Din=88776655，按 CLK 按钮 2 次，则写入到 4~ 7 单元

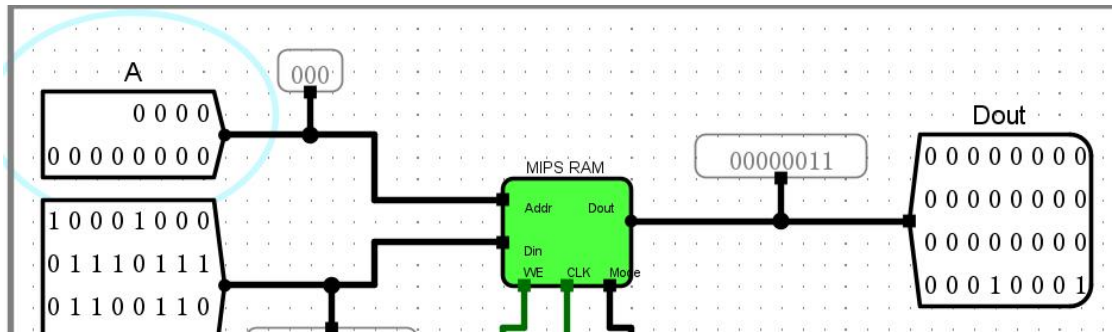


第 2 步：按字节访问方式读。

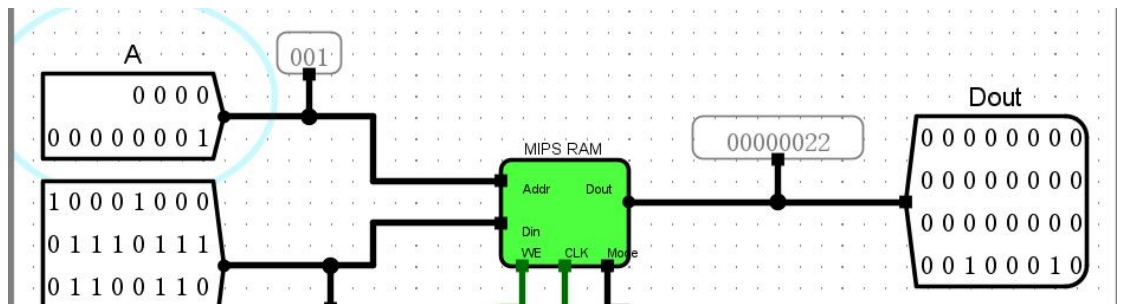
- 置 Mode=01，按字节访问；置 WE=0，存储器读



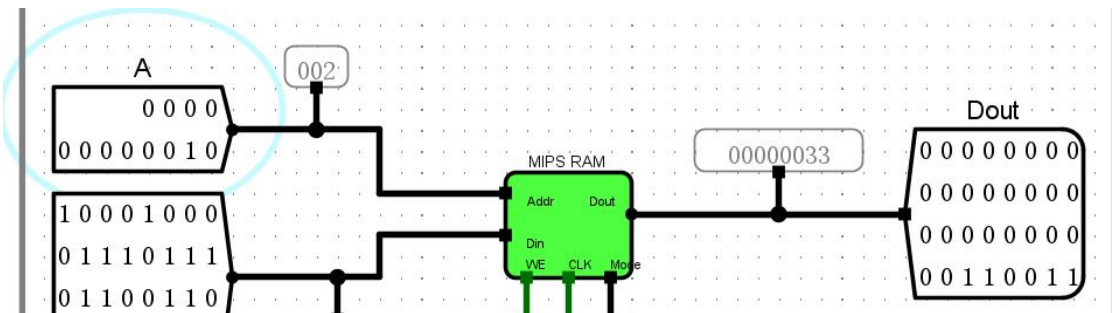
- 置 A=0，则 Dout=00000011=11



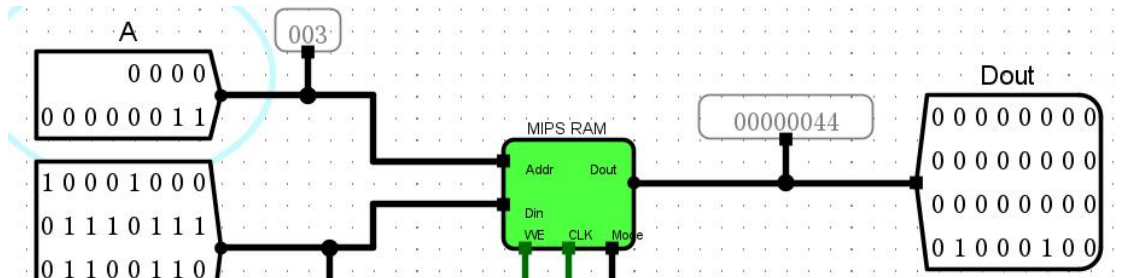
- 置 A=1, 则 Dout=00000022=22



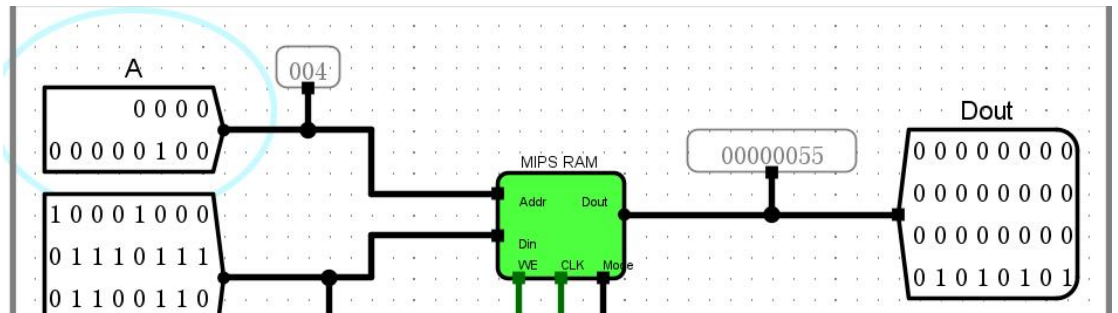
- 置 A=2, 则 Dout=00000033=33



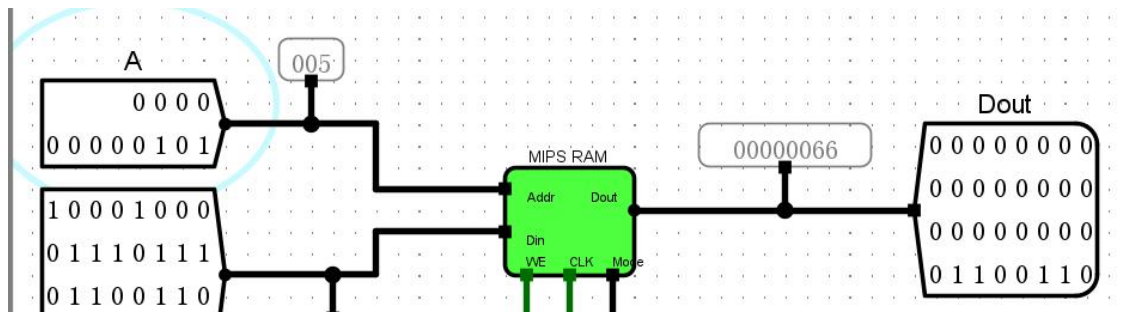
- 置 A=3, 则 Dout=00000044=44



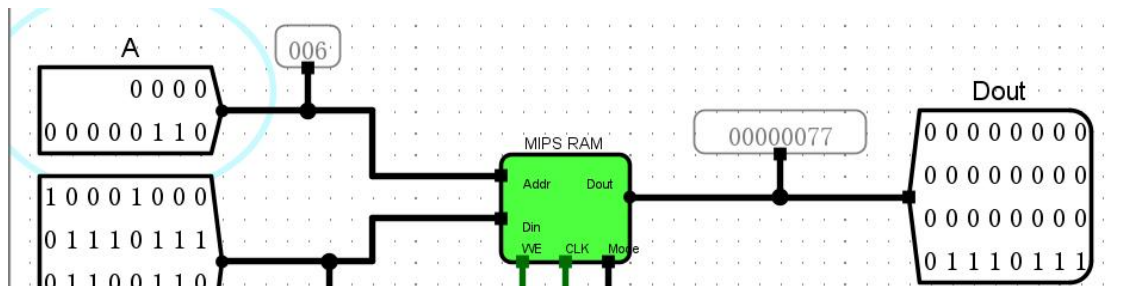
- 置 A=4, 则 Dout=00000055=55



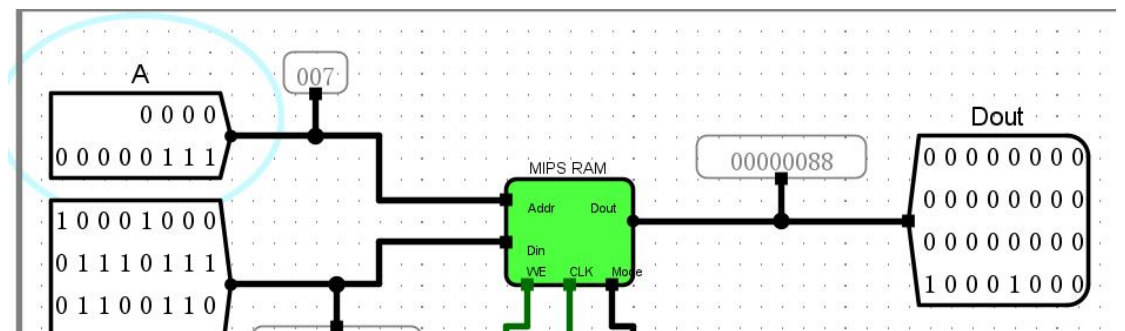
- 置 A=5，则 Dout=00000066=66



- 置 A=6，则 Dout=00000077=77



- 置 A=7，则 Dout=00000088=88

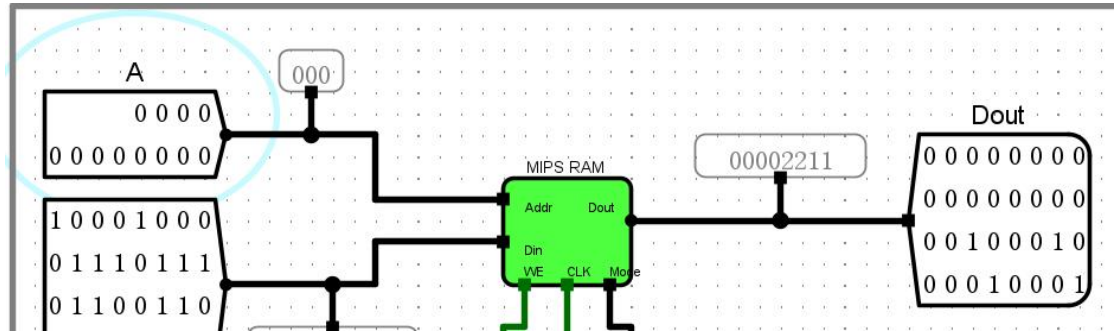


第 3 步：按半字访问方式读。

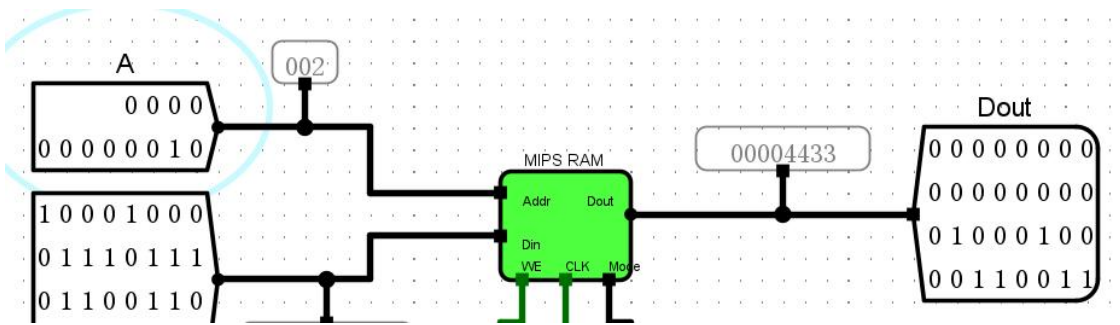
- 置 Mode=10，按半字访问；置 WE=0，存储器读



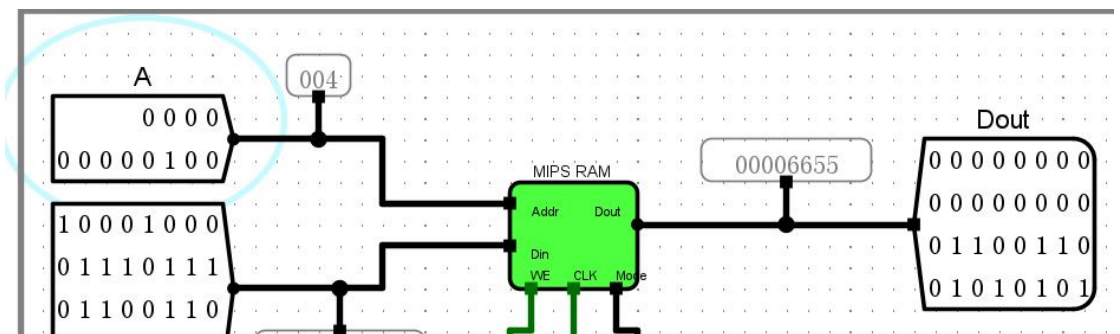
- 置 A=0, 则 Dout=00002211=2211



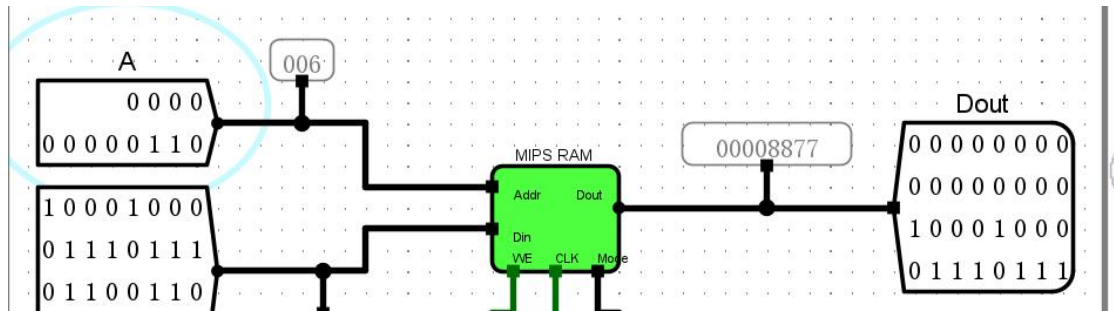
- 置 A=2, 则 Dout=00004433=4433



- 置 A=4, 则 Dout=00006655=6655



- 置 A=6, 则 Dout=00008877=8877

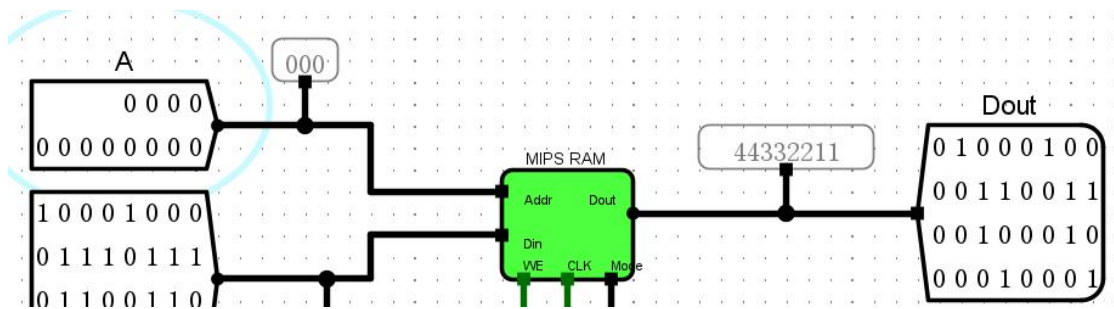


第4步：按字访问方式读。

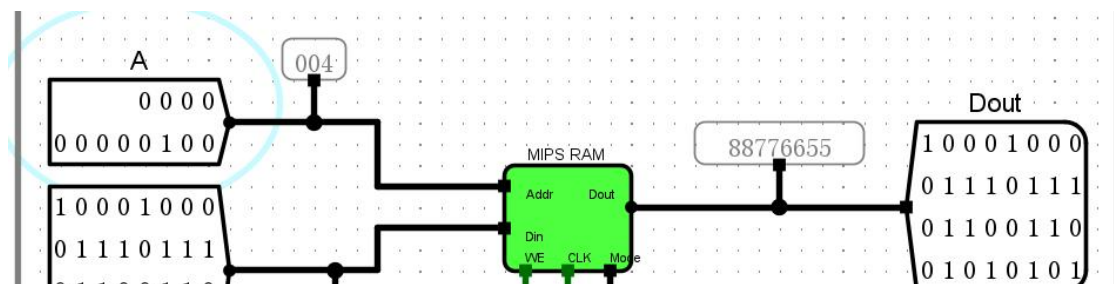
- 置 Mode=00，按字访问；置 WE=0，存储器读



- 置 A=0，则 Dout=44332211

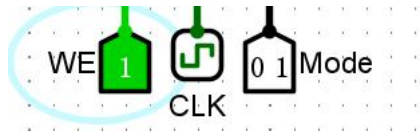


- 置 A=4，则 Dout=88776655

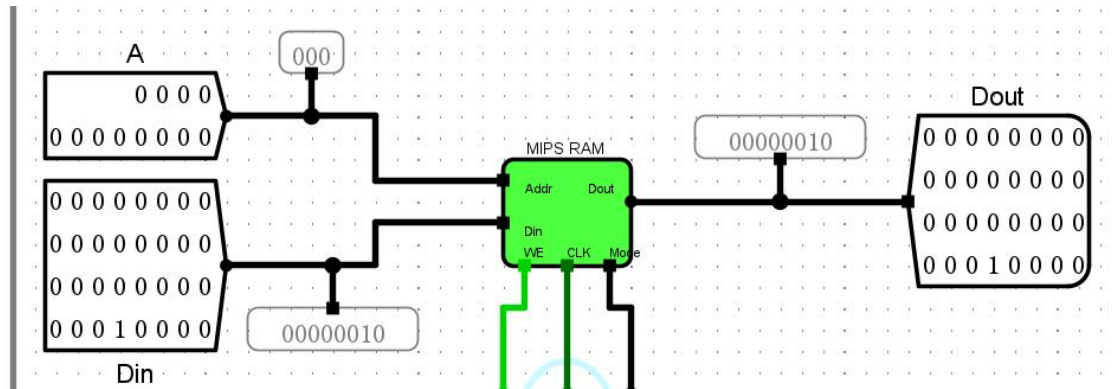


第5步：按字节访问方式写。

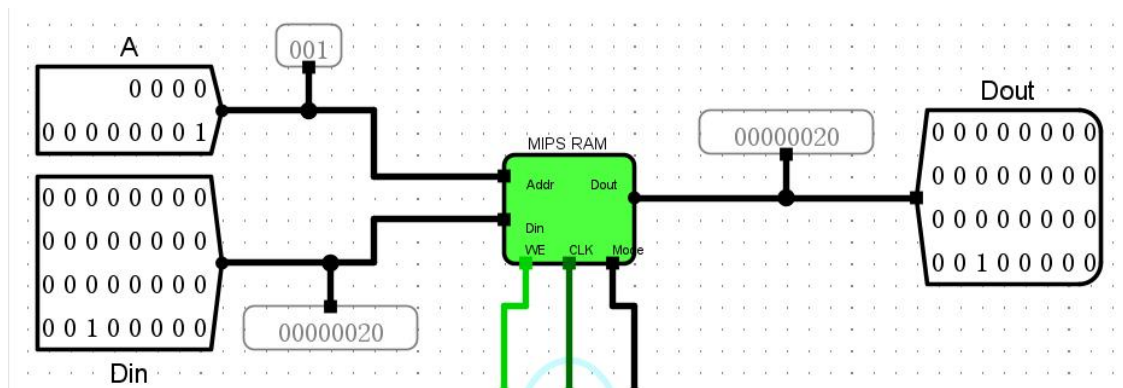
- 置 Mode=01，按字节访问；置 WE=1，存储器写



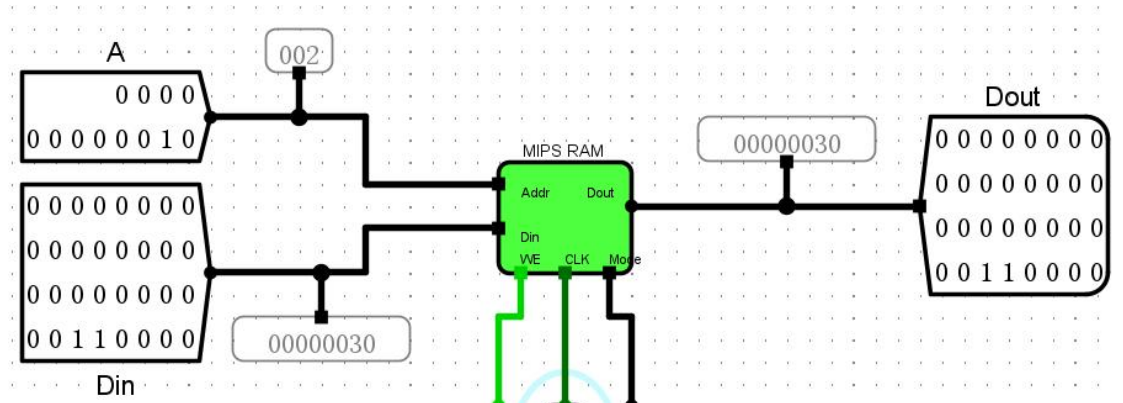
- 置 A=0、Din=10；按 CLK 键 2 次，则将 10 写入 0 号单元



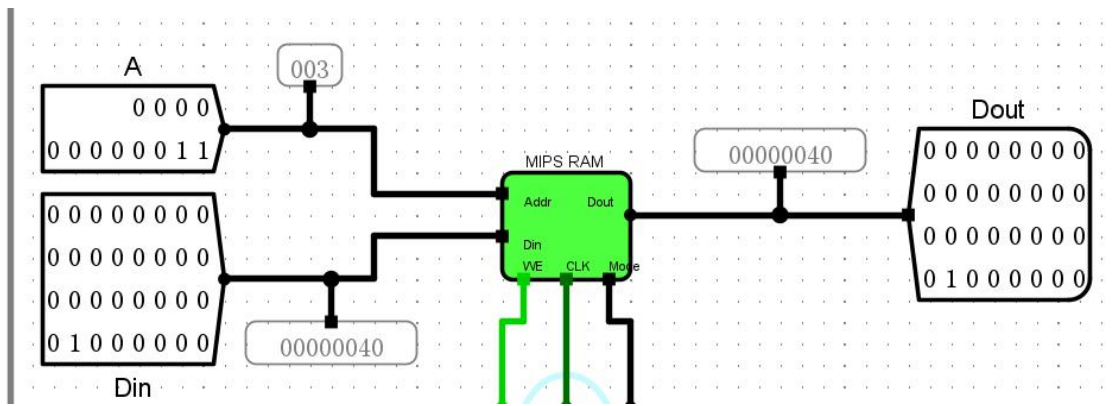
- 置 A=1、Din=20；按 CLK 键 2 次，则将 20 写入 1 号单元



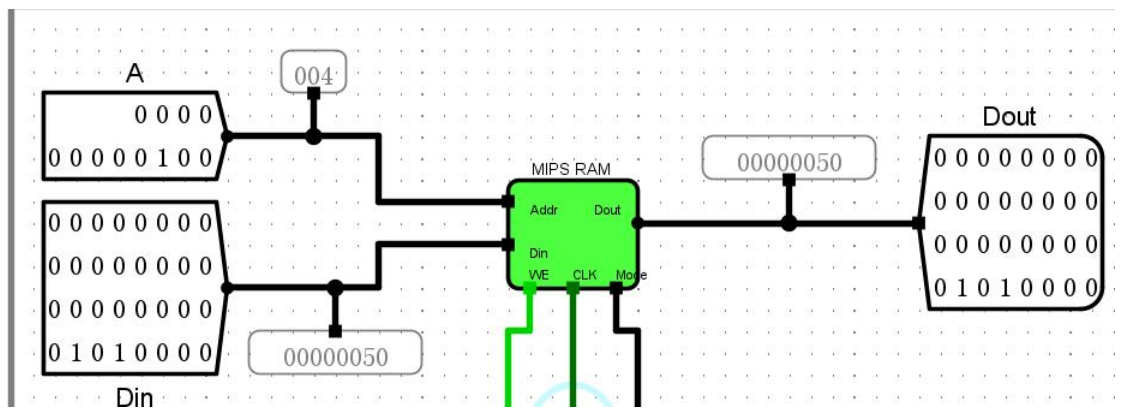
- 置 A=2、Din=30；按 CLK 键 2 次，则将 30 写入 2 号单元



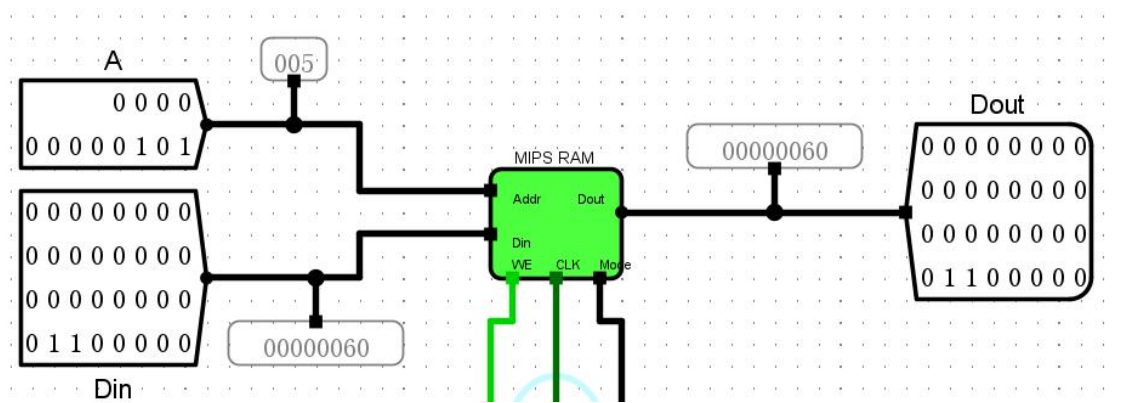
- 置 A=3、Din=40；按 CLK 键 2 次，则将 40 写入 3 号单元



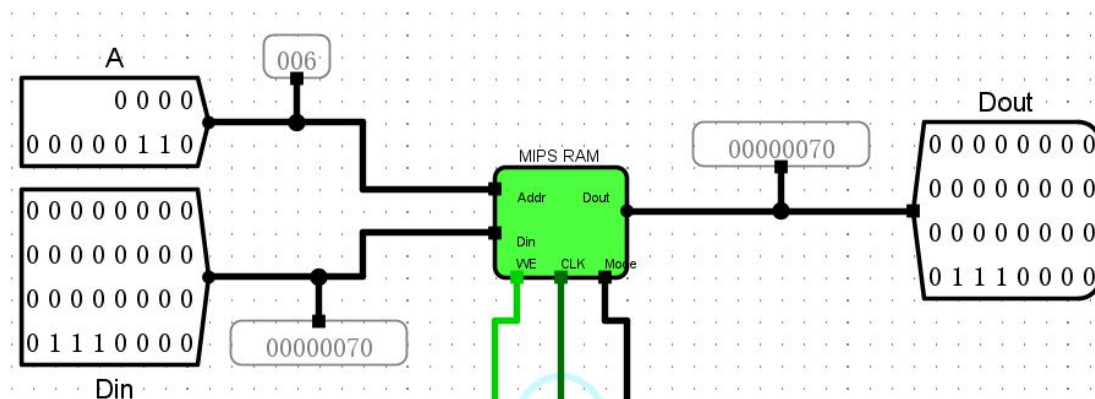
- 置 A=4、Din=50；按 CLK 键 2 次，则将 50 写入 4 号单元



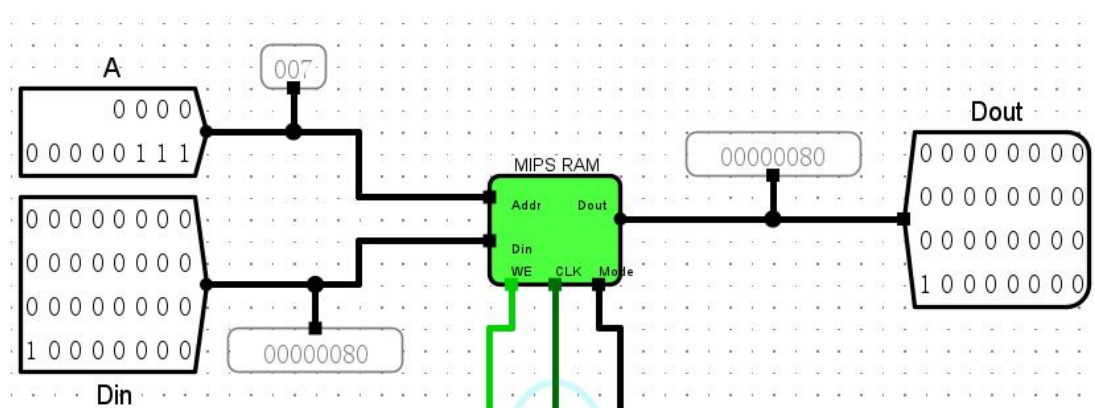
- 置 A=5、Din=60；按 CLK 键 2 次，则将 60 写入 5 号单元



- 置 A=6、Din=70；按 CLK 键 2 次，则将 70 写入 6 号单元



- 置 A=7、Din=80；按 CLK 键 2 次，则将 80 写入 7 号单元

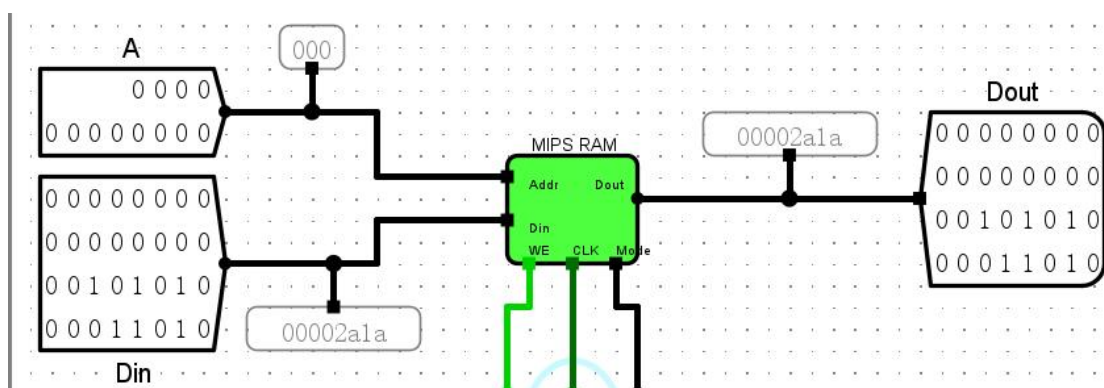


第 6 步：按半字访问方式写。

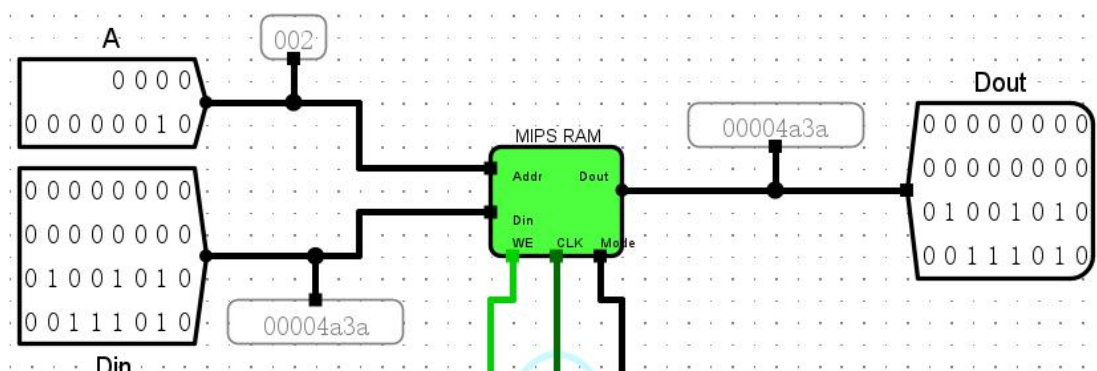
- 置 Mode=10，按半字访问；置 WE=1，存储器写



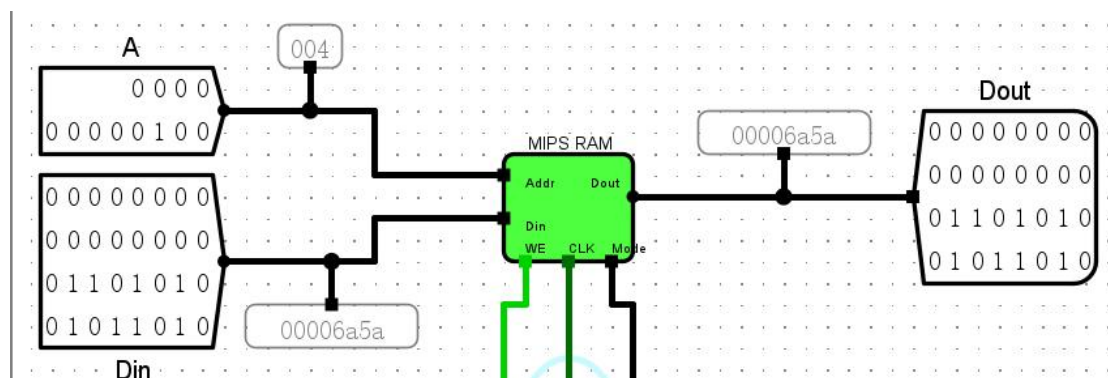
- 置 A=0、Din=2a1a；按 CLK 键 2 次，则将 2a1a 写入 0-1 号单元



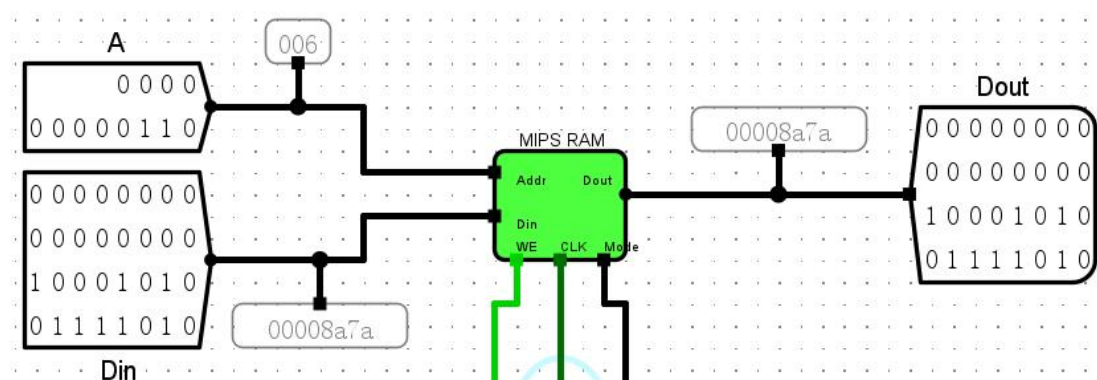
- 置 A=2、 Din=4a3a; 按 CLK 键 2 次, 则将 4a3a 写入 2-3 号单元



- 置 A=4、 Din=6a5a; 按 CLK 键 2 次, 则将 6a5a 写入 4-5 号单元



- 置 A=6、 Din=8a7a; 按 CLK 键 2 次, 则将 8a7a 写入 6-7 号单元

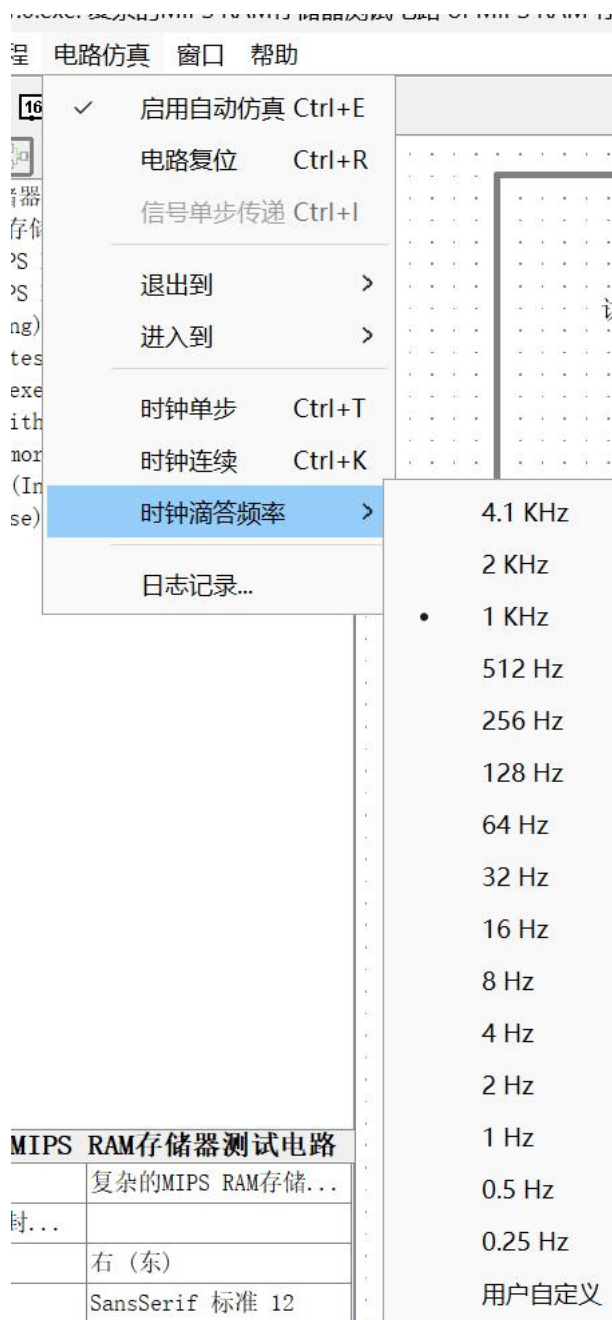


第 7 步: 按字访问方式写。

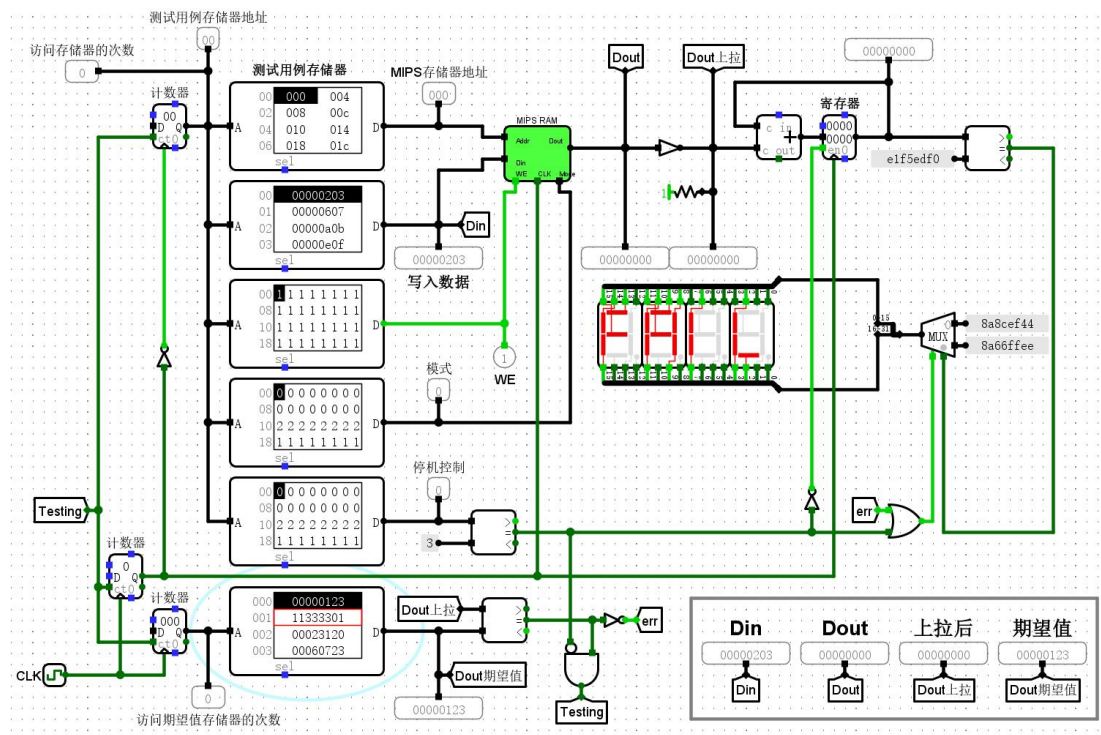
同第 1 步。

复杂的 MIPS RAM 存储器测试电路:

- ① 设置时钟频率=1KHz。



- ② 按 Ctrl+R，使电路复位。
- ③ 按 Ctrl+K，启动时钟。
- ④ 如果显示“PASS”，表示测试通过。



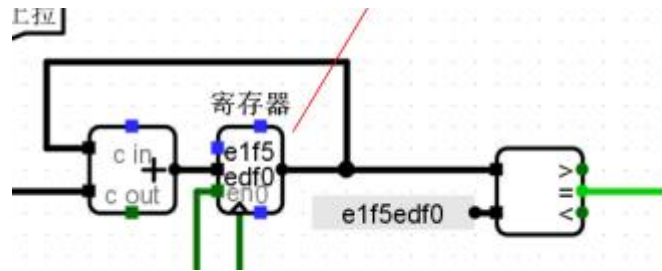
回答问题:

1) 基于 MIPS RAM 存储器电路, 分析其如何实现存储器按照不同的访问字长(字节、半字、字)进行读写操作的?

答: 使用 4 个独立的 8 位 RAM, 每个存储 1 字节, 共同组成 32 位完整的字。使用多路选择器来控制三种访问字长的读写实现: 若 mode 为 00, 一次性读取 4 个 RAM 的 D0,D1,D2,D3 构成 32 位完整的字, 向 4 个 RAM 写入 Din0,Din1,Din2,Din3 到 4 个 RAM 中; 若 mode 为 01, 只读取 D0,D1,D2,D3 其中之一, 并在高位补 0, 只写入 Din 的最低位 Din0; 若 mode 为 10, 则读取 D0,D1 或 D2,D3, 并在高位补 0, 只写入最低的两位 Din0,Din1; 若 mode 为 11, 则既无写入, 也无读出。

2) 复杂的 MIPS RAM 存储器测试电路中, 六个存储器的内容都是什么? Din, Dout, Dout 期望值三者各代表什么, 彼此有什么关系? 下图中的 e1f5edf0 什么意思? 如果 FAIL, 可能是什么原因?

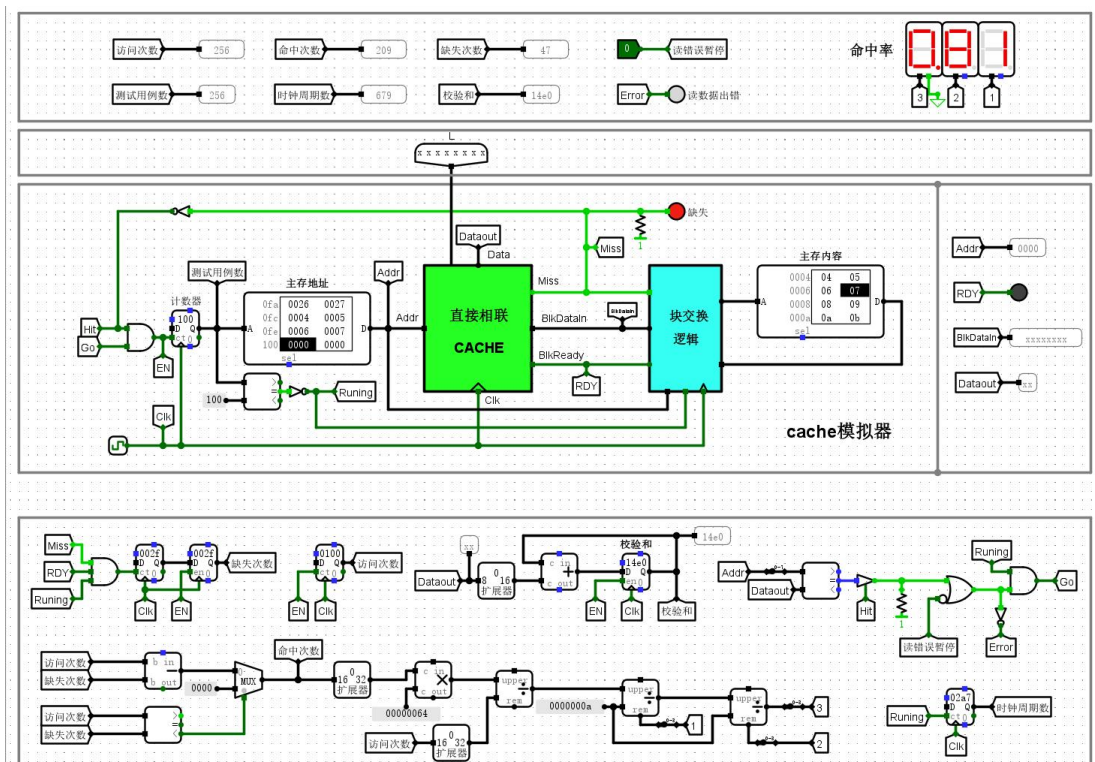
答: 从上至下, 分别为: 地址、写入数据、WE、Mode、停机信号、期望值。Din 表示测试电路主动发给 MIPS RAM 本次要写入指定地址的原始数据, Dout 表示 MIPS RAM 经过本次读写后, 真实对外输出的数据, Dout 期望值表示本应该得到的理想的正确读出值。Din 输入给 MIPS RAM, 输出 Dout, 根据预期计算得到 Dout 期望值, 若 Dout=Dout 期望值, 则输出 PASS, 否则输出 Fail。32 为寄存器是把多轮 Dout 的输出、上拉扩展后的值进行拼接得到的最终校验特征值, 常量是预设的正确固定特征值。测试值本身计算出错, 期望值错误; 上次测试结束后, 没有 Ctrl+R 复位电路。



- (3) 直接相联映射方式 cache 电路、直接相联映射方式 cache 测试电路、全相联映射方式 cache 电路、全相联映射方式 cache 测试电路、2 路组相联映射方式 cache 电路、2 路组相联映射方式 cache 测试电路、4 路组相联映射方式 cache 电路、4 路组相联映射方式 cache 测试电路。

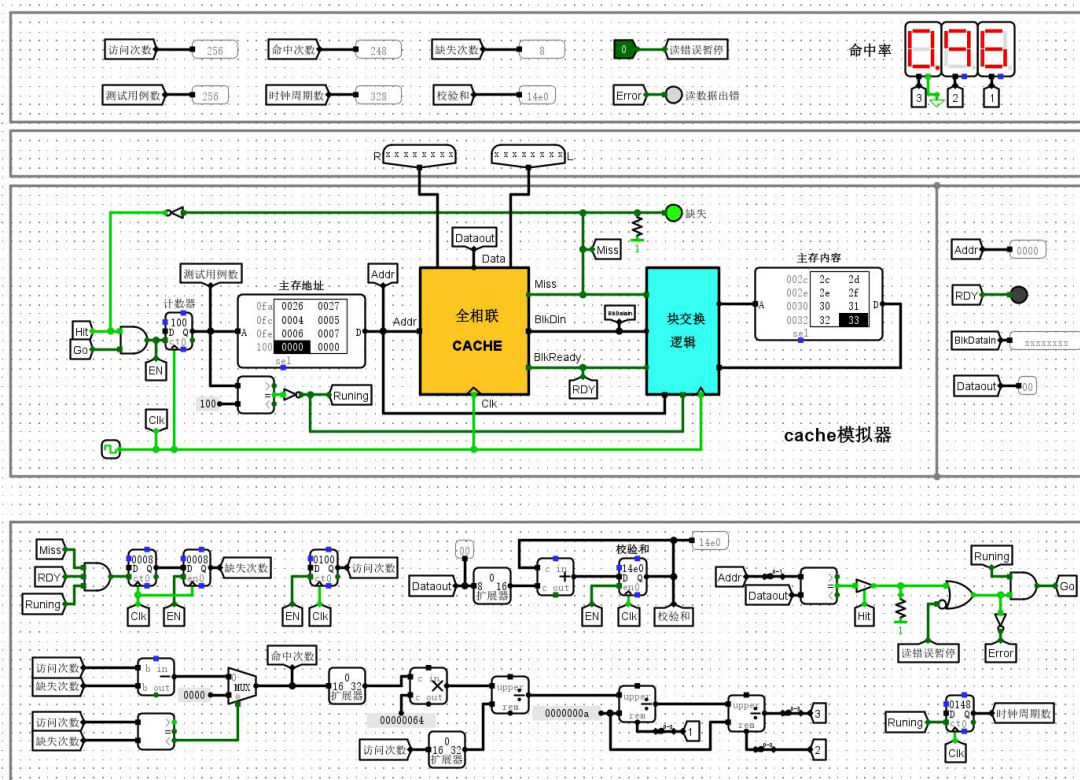
直接相联映射方式 cache 测试电路：

先按 Ctrl+R，使电路复位；然后按 Ctrl+K，启动时钟；等到命中率数字不变（计数器=100），表示表示测试完成。



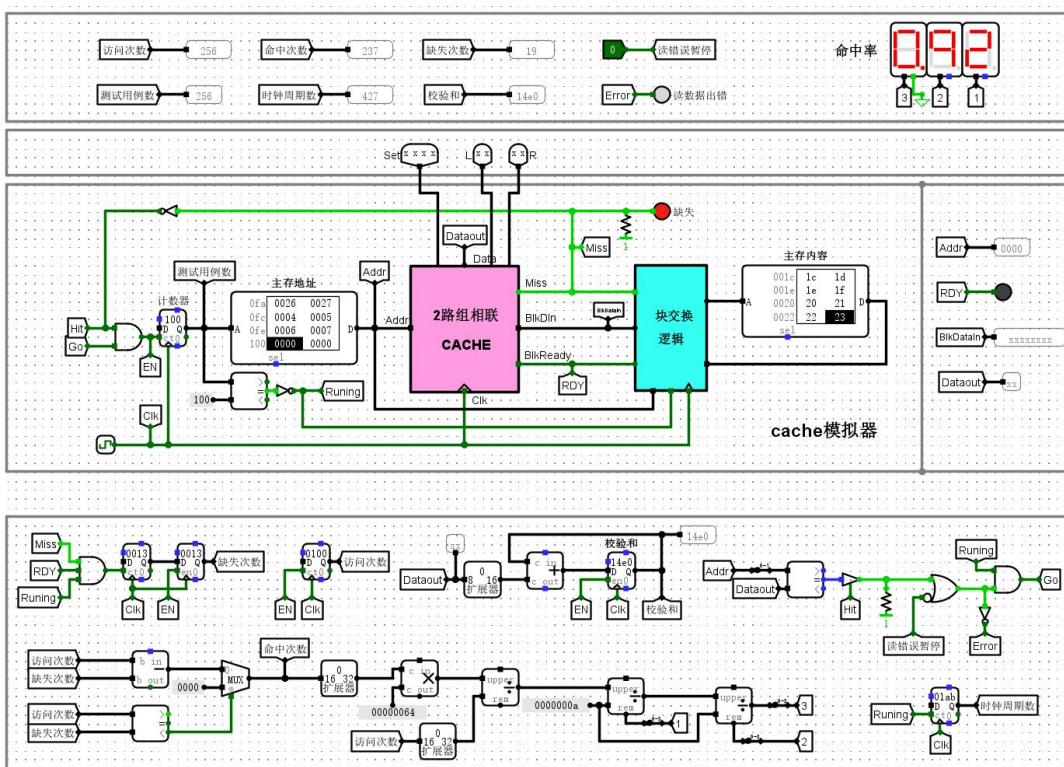
全相联映射方式 cache 测试电路：

先按 Ctrl+R，使电路复位；然后按 Ctrl+K，启动时钟；等到命中率数字不变（计数器=100），表示表示测试完成。

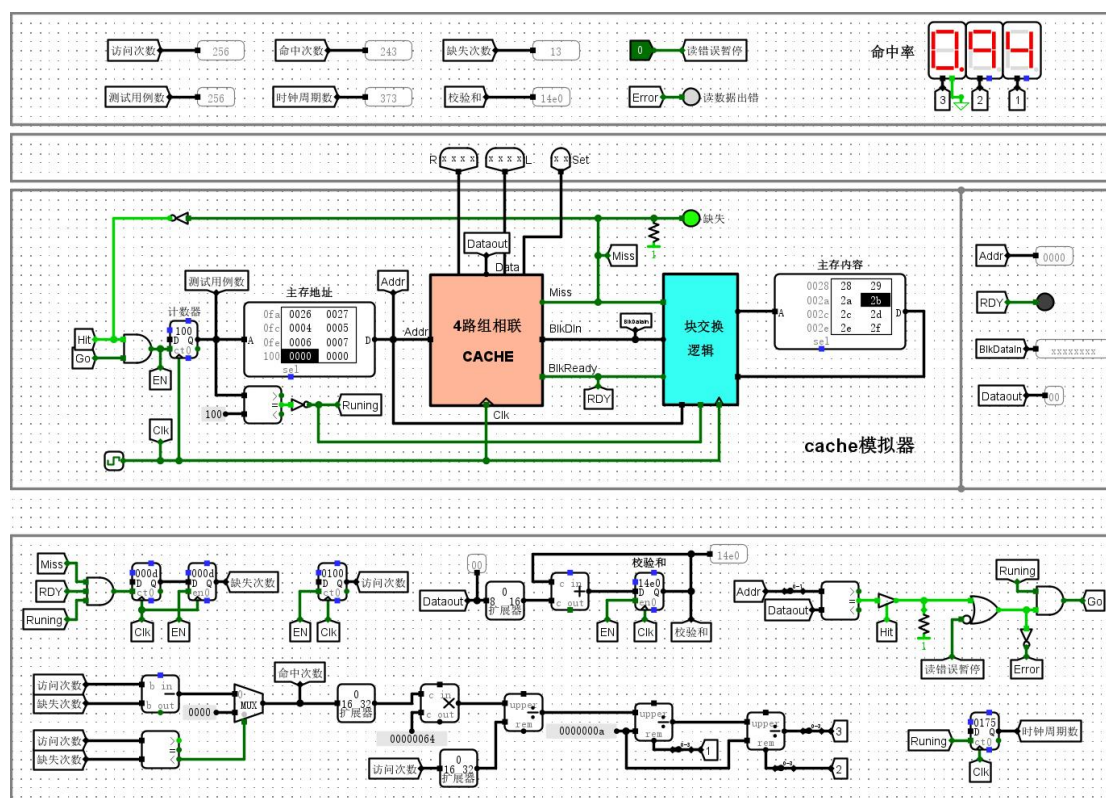


2 路相联映射方式 cache 测试电路：

先按 Ctrl+R, 使电路复位；然后按 Ctrl+K, 启动时钟；等到命中率数字不变（计数器=100），表示表示测试完成。



4 路相联映射方式 cache 测试电路：



回答问题：

- 1) 根据直接相联映射方式 cache 电路，分析 cache 块交换逻辑电路是如何实现块替换的？可结合实例阐述

答：当 Tag 与 TagData 不匹配，或者 Valid 为 0，判定为未命中，Hit=0，Miss=1，向主存请求数据，cache 块交换状态机电路分 4 个周期依次将 4 个字节从主存读入缓存寄存器，cache 块交换逻辑电路收集这 4 个字节，拼接成一个 32 位的完整块信号，BlockDataIn 覆盖掉 SlotData 完成替换。

- 2) 直接相联映射方式下，分析第 33~第 64 次 cache 访问的情况，统计命中率和缺失率

```
cacahe (直接相联映射) 测试用例——主存地址.txt
文件(F) 编辑(E) 格式(O) 查看(V) 帮助(H)
v2.0 raw
0 1 2 3 8 9 a b
10 11 12 13 20 21 22 23
8 9 a b 4 5 6 7
24 25 26 27 4 5 6 7
0 1 2 3 8 9 a b
8 9 a b 20 21 22 23
28 29 2a 2b 24 25 26 27
4 5 6 7 24 25 26 27
0 1 2 3 8 9 a b
20 21 22 23 20 21 22 23
30 31 32 33 4 5 6 7
24 25 26 27 4 5 6 7
0 1 2 3 8 9 a b
20 21 22 23 20 21 22 23
28 29 2a 2b 4 5 6 7
24 25 26 27 24 25 26 27
0 1 2 3 8 9 a b
8 9 a b 20 21 22 23
30 31 32 33 4 5 6 7
24 25 26 27 24 25 26 27
0 1 2 3 0 1 2 3
8 9 a b 20 21 22 23
28 29 2a 2b 4 5 6 7
24 25 26 27 4 5 6 7
0 1 2 3 8 9 a b
0 1 2 3 20 21 22 23
30 31 32 33 24 25 26 27
4 5 6 7 24 25 26 27
0 1 2 3 8 9 a b
20 21 22 23 20 21 22 23
28 29 2a 2b 4 5 6 7
24 25 26 27 4 5 6 7
```

答:

- (1) 访问 0 1 2 3
 - 访问地址为 0 存储单元，不命中，从主存中装入 0 1 2 3 四个单元，之后访问 1 2 3 存储单元，全部命中； 命中 3 次、不命中 1 次
- (2) 访问 8 9 a b
 - 访问地址为 8 存储单元，不命中，从主存中装入 8 9 a b 四个单元，之后访问 9 a b 存储单元，全部命中； 命中 3 次、不命中 1 次
- (3) 访问 8 9 a b
 - 访问地址为 8 存储单元，命中（第（2）步已经将 8 9 a b 装入 cache 中了），之后访问 9 a b 存储单元，全部命中； 命中 4 次、不命中 0 次
- (4) 访问 20 21 22 23
 - 访问地址为 20 存储单元，不命中，从主存中装入 20 21 22 23 四个单元，之后访问 21 22 23 存

储单元，全部命中； 命中 3 次、不命中 1 次

- (5) 访问 28 29 2a 2b

- 访问地址为 28 存储单元，不命中，从主存中装入 20 21 22 23 四个单元，之后访问 29 2a 2b 存储单元，全部命中； 命中 3 次、不命中 1 次

- (6) 访问 24 25 26 27

- 访问地址为 24 存储单元，不命中，从主存中装入 24 25 26 27 四个单元，之后访问 25 26 27 存储单元，全部命中； 命中 3 次、不命中 1 次

- (7) 访问 4 5 6 7

- 访问地址为 4 存储单元，不命中，从主存中装入 4 5 6 7 四个单元，之后访问 5 6 7 存储单元，全部命中； 命中 3 次、不命中 1 次

- (8) 访问 24 25 26 27

- 访问地址为 24 存储单元，命中（第（6）步已经将 24 25 26 27 装入 cache 中了），访问 25 26 27 存储单元，全部命中； 命中 4 次、不命中 0 次

因此总访问 32 次，命中 26 次，缺失 6 次，命中率=81.25%，缺失率=18.75%。

3) 全相联映射方式，采用了 LRU（近期最少使用）替换算法，结合电路分析其工作原理，可通过实例阐述

答：cache 电路图中，使用了 16 位计数器进行淘汰计数，每当该 cache 命中时，其计数器会被清零，未被命中的行会进行自增，计数器的值越大，说明上次被访问的时间越久。在未命中，即 Tag 与 Tag0-Tag7 都不相等，Miss=1 时，通过 Pri 扫描 V0-V7，找到编号最小的空闲 cache，若 cache 已满，通过 Max3 电路进行比较找到找到计数器值最大的 cache，配合 BlkReady 完成替换并计数器清零。

4) 分析 4 路组相联映射方式中第 33~第 64 次 cache 访问的情况，统计命中率和缺失率


```

cacahe (4路组相联映射) 测试用例——主存地址.txt
文件(F) 编辑(E) 格式(O) 查看(V) 帮助(H)
v2.0 raw
0 1 2 3 8 9 a b
10 11 12 13 20 21 22 23
8 9 a b 4 5 6 7
24 25 26 27 4 5 6 7
0 1 2 3 8 9 a b
8 9 a b 20 21 22 23
28 29 2a 2b 24 25 26 27
4 5 6 7 24 25 26 27
0 1 2 3 8 9 a b
20 21 22 23 20 21 22 23
30 31 32 33 4 5 6 7
24 25 26 27 4 5 6 7
0 1 2 3 8 9 a b
20 21 22 23 20 21 22 23
28 29 2a 2b 4 5 6 7
24 25 26 27 24 25 26 27
0 1 2 3 8 9 a b
8 9 a b 20 21 22 23
30 31 32 33 4 5 6 7
24 25 26 27 24 25 26 27
0 1 2 3 0 1 2 3
8 9 a b 20 21 22 23
28 29 2a 2b 4 5 6 7
24 25 26 27 4 5 6 7
0 1 2 3 8 9 a b
0 1 2 3 20 21 22 23
30 31 32 33 24 25 26 27
4 5 6 7 24 25 26 27
0 1 2 3 8 9 a b
20 21 22 23 20 21 22 23
28 29 2a 2b 4 5 6 7
24 25 26 27 4 5 6 7

```

答:

- (1) 访问 0 1 2 3
 - 访问地址为 0 存储单元，不命中，从主存中装入 0 1 2 3 四个单元，之后访问 1 2 3 存储单元，全部命中； 命中 3 次、不命中 1 次
- (2) 访问 8 9 a b
 - 访问地址为 8 存储单元，不命中，从主存中装入 8 9 a b 四个单元，之后访问 9 a b 存储单元，全部命中； 命中 3 次、不命中 1 次
- (3) 访问 8 9 a b
 - 访问地址为 8 存储单元，命中（第（2）步已经将 8 9 a b 装入 cache 中了），之后访问 9 a b 存储单元，全部命中； 命中 4 次、不命中 0 次
- (4) 访问 20 21 22 23
 - 访问地址为 20 存储单元，不命中，从主存中装入 20 21 22 23 四个单元，之后访问 21 22 23 存储单元，全部命中； 命中 3 次、不命中 1 次

- (5) 访问 28 29 2a 2b

• 访问地址为 28 存储单元，不命中，从主存中装入 28 29 2a 2b 四个单元，之后访问 29 2a 2b 存储单元，全部命中； 命中 3 次、不命中 1 次

- (6) 访问 24 25 26 27

• 访问地址为 24 存储单元，不命中，从主存中装入 24 25 26 27 四个单元，之后访问 25 26 27 存储单元，全部命中； 命中 3 次、不命中 1 次

- (7) 访问 4 5 6 7

• 访问地址为 4 存储单元，不命中，从主存中装入 4 5 6 7 四个单元，之后访问 5 6 7 存储单元，全部命中； 命中 3 次、不命中 1 次

- (8) 访问 24 25 26 27

• 访问地址为 24 存储单元，命中（第（6）步已经将 8 9 a b 装入 cache 中了），之后访问 25 26 27 存储单元，全部命中； 命中 4 次、不命中 0 次

因此总访问 32 次，命中 26 次，缺失 6 次，命中率=81.25%，缺失率=18.75%。

5) 观察 cache 验证试验部分中的不同映射方式（直接，全相联，2 路组相联，4 路组相联）的实验结果，基于命中率或者缺失率指标，对实验结果进行解释分析

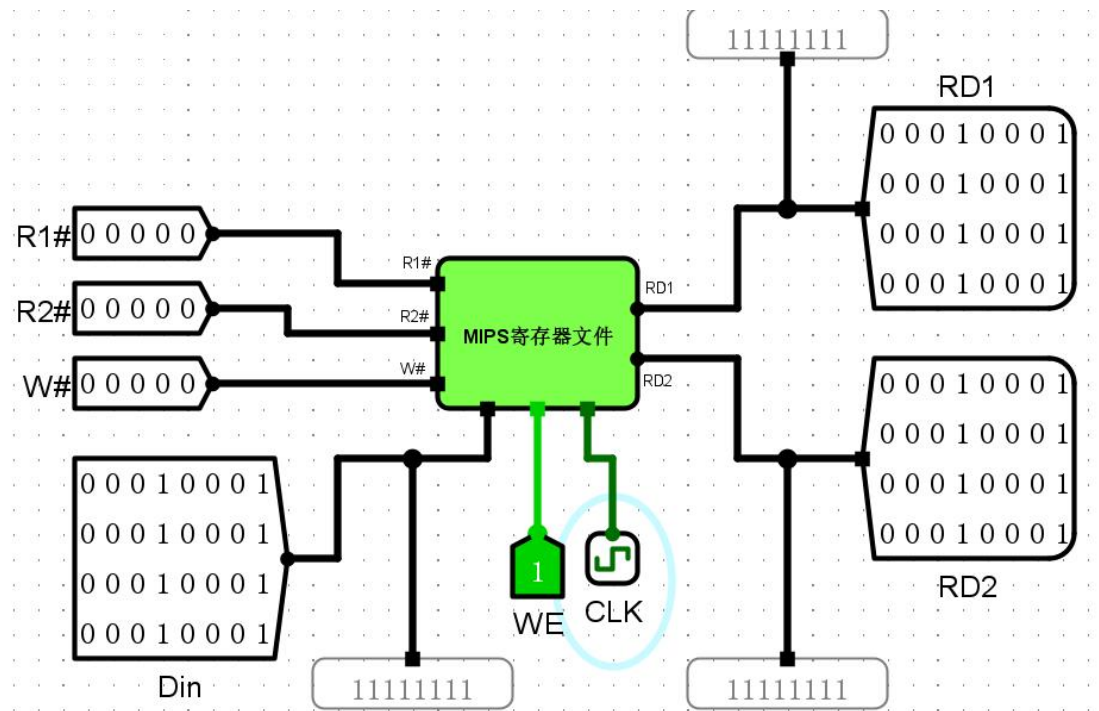
答：根据命中率排序：全相联>4 路组相联>2 路组相联>直接相联。相联度越低，经常访问的主存块越容易映射到同一个 cache 行，产生相互覆盖，导致命中率降低，但电路实现更简单，延迟更低，反之，相联度越高，需要查找比较的行数越多，延迟越高，命中率越高。

(4) MIPS 寄存器文件电路、简单的 MIPS 寄存器文件测试电路、复杂的 MIPS 寄存器文件测试电路。

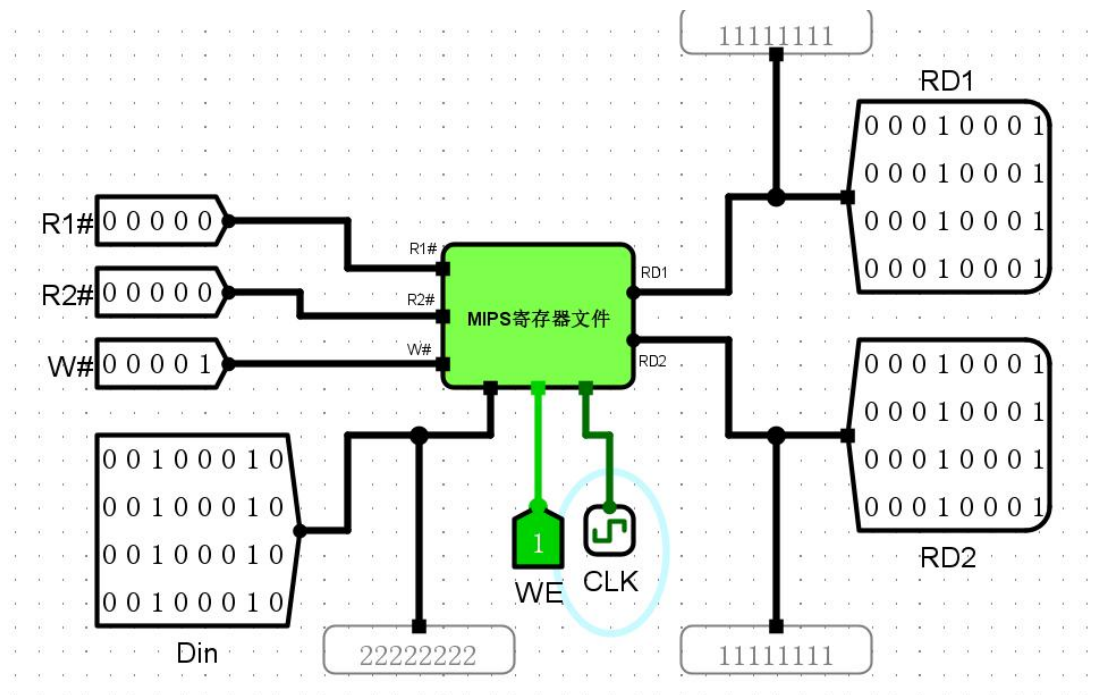
简单的 MIPS 寄存器文件测试电路：

第 1 步：向寄存器 0~寄存器 3，分别写入 11111111、 22222222、 33333333、 44444444（均为十六进制，下同）。

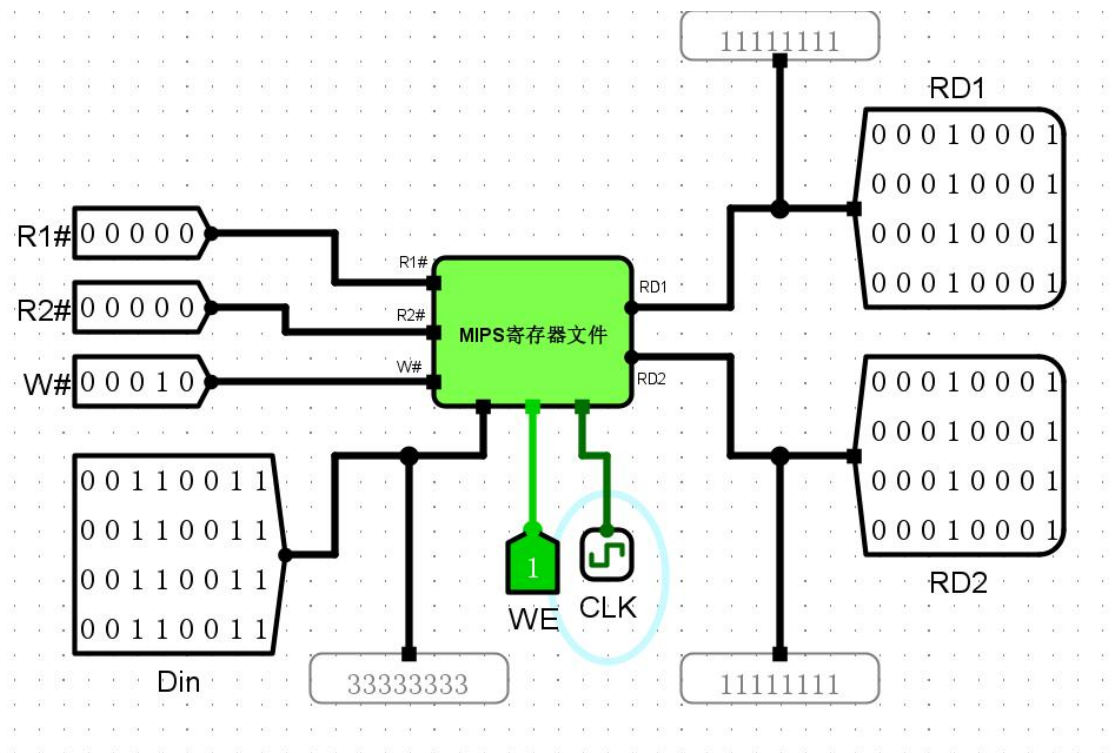
- 置 W#=00000， Din=11111111H， WE=1，按时钟 CLK 按钮 2 次



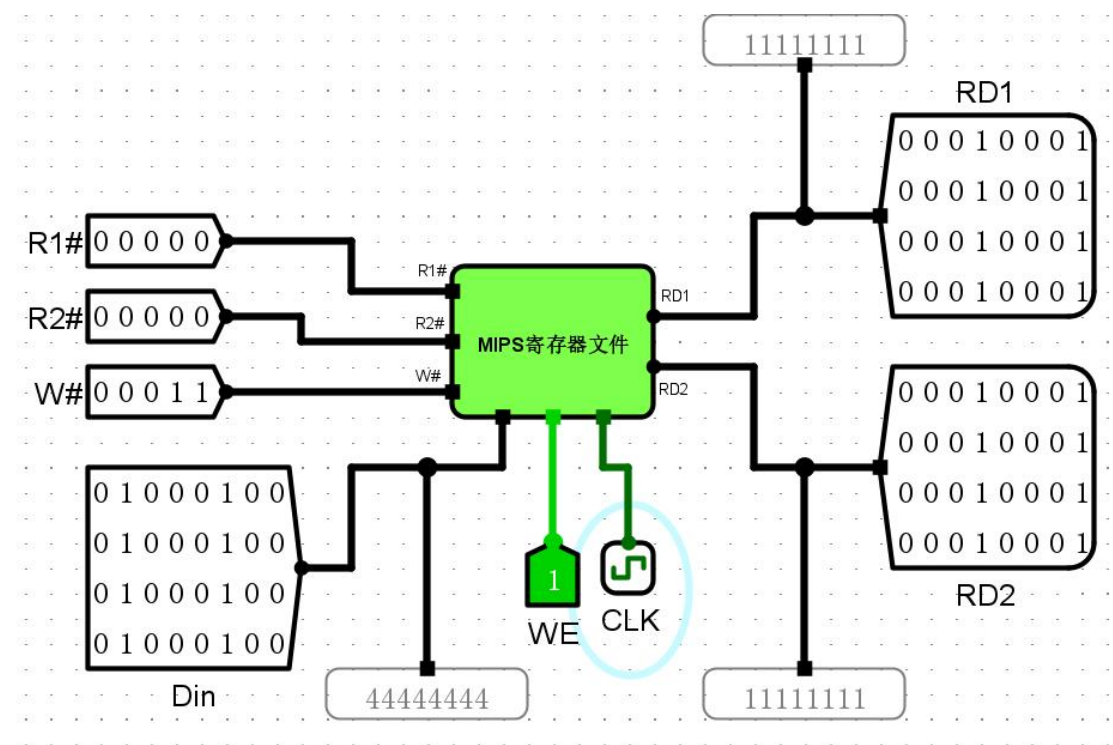
- 置 W#=00001, Din=2222222H, WE=1, 按时钟 CLK 按钮 2 次



- 置 W#=00010, Din=3333333H, WE=1, 按时钟 CLK 按钮 2 次

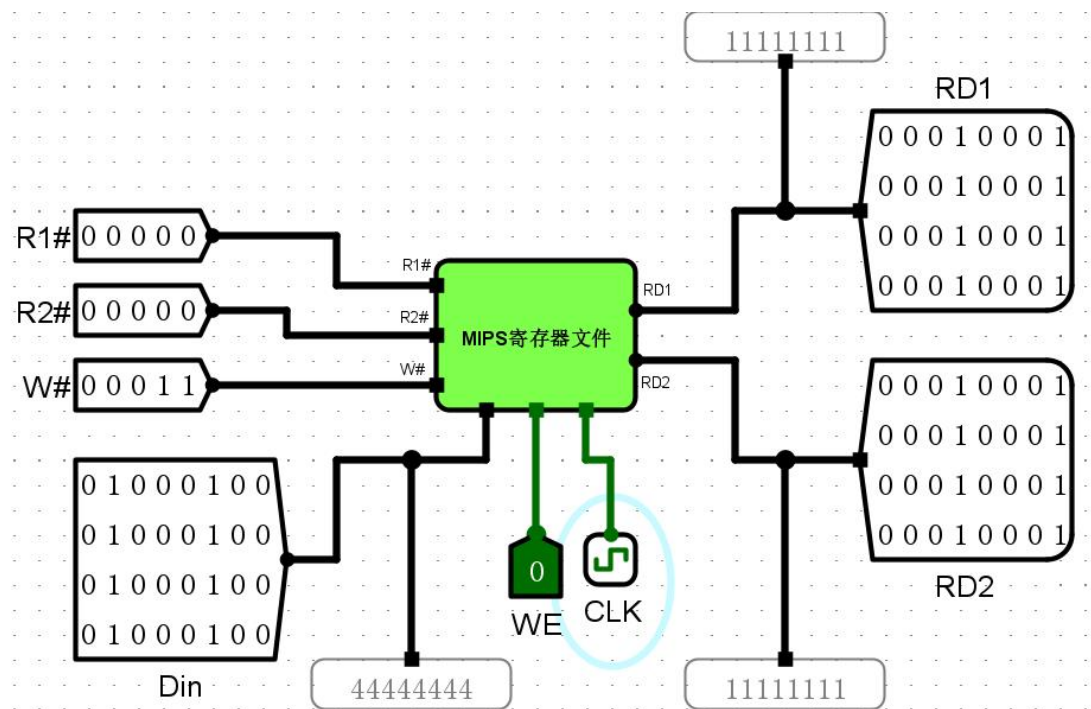


- 置 W#=00011， Din=44444444H， WE=1， 按时钟 CLK 按钮 2 次

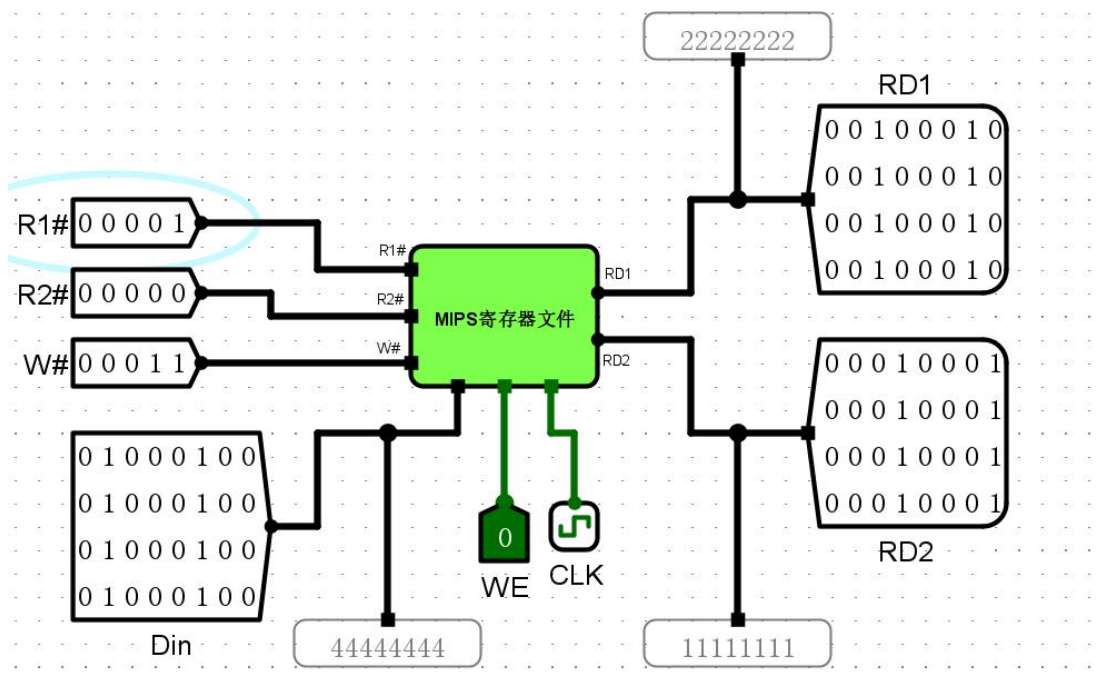


第 2 步：从 RD1 口读寄存器 0~寄存器 3。

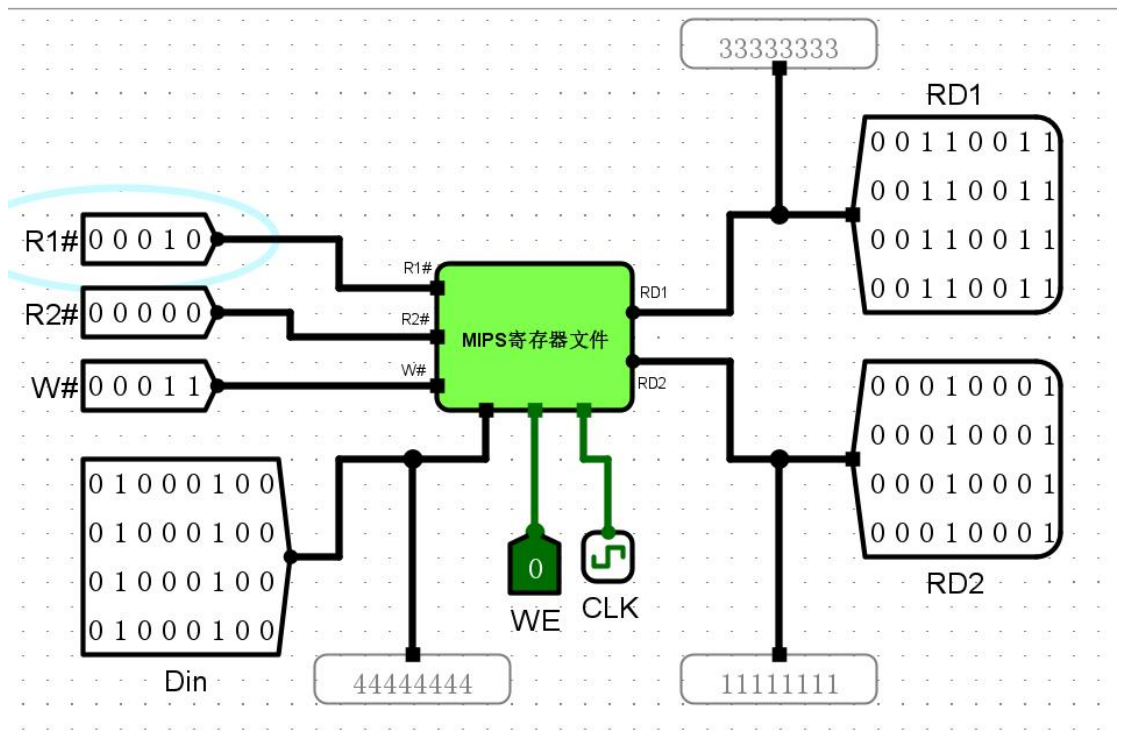
- 置 R1#=00000， WE=0， 观察 RD1 是否=11111111H



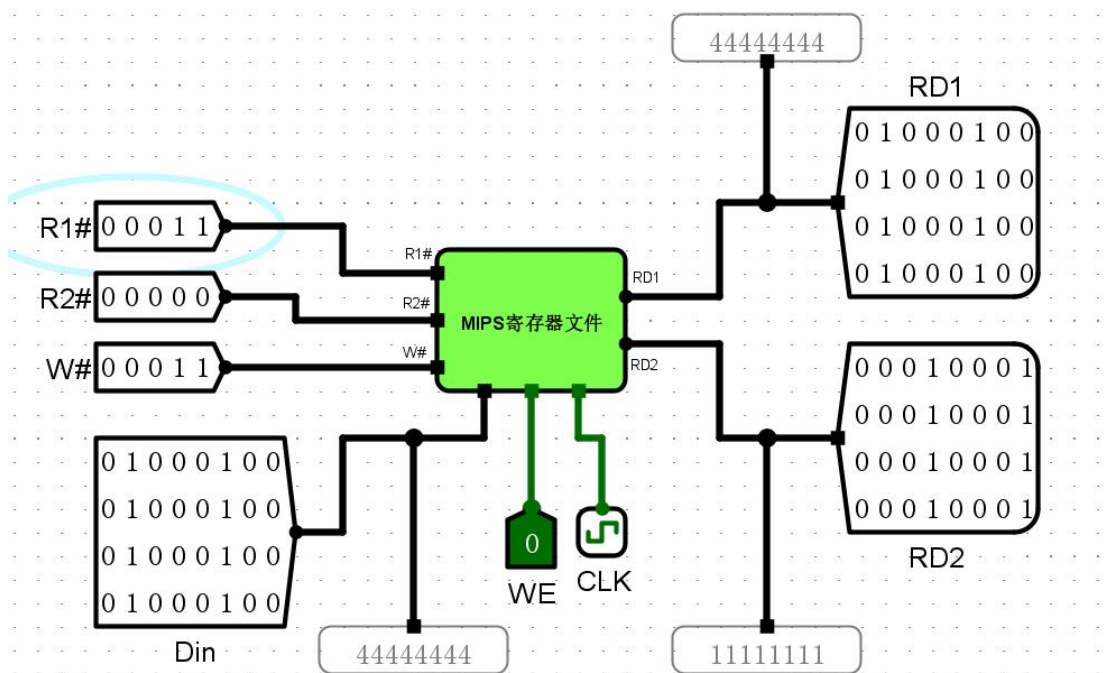
- 置 R1#=00001, WE=0, 观察 RD1 是否=22222222H



- 置 R1#=00010, WE=0, 观察 RD1 是否=33333333H

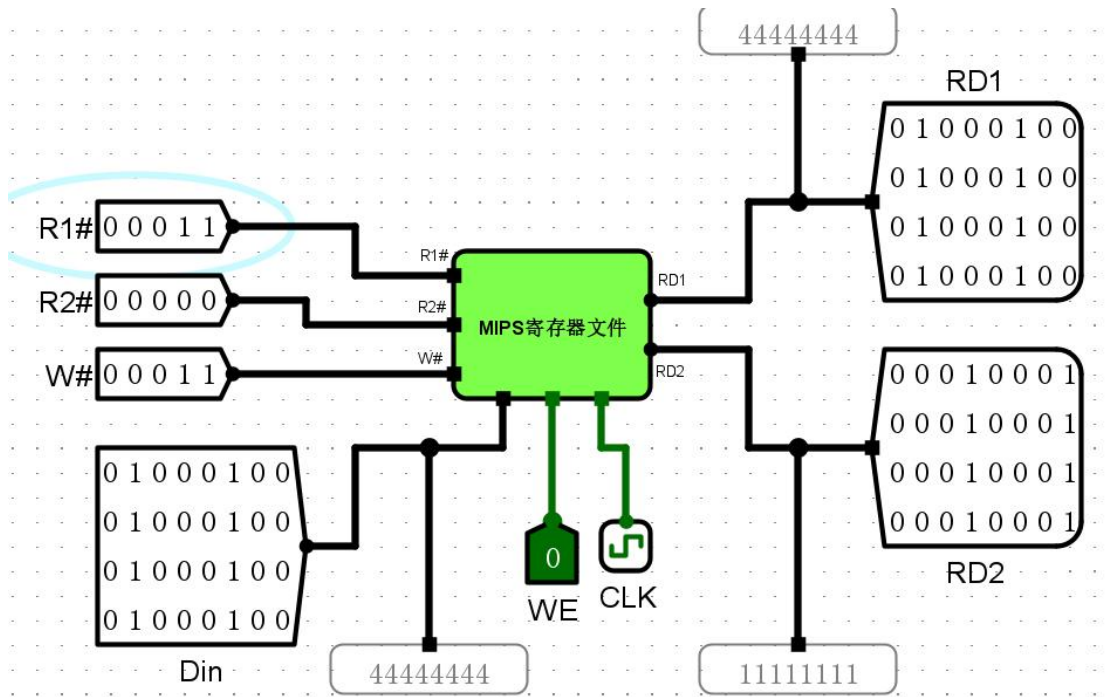


- 置 R1#=00011, WE=0, 观察 RD1 是否=44444444H

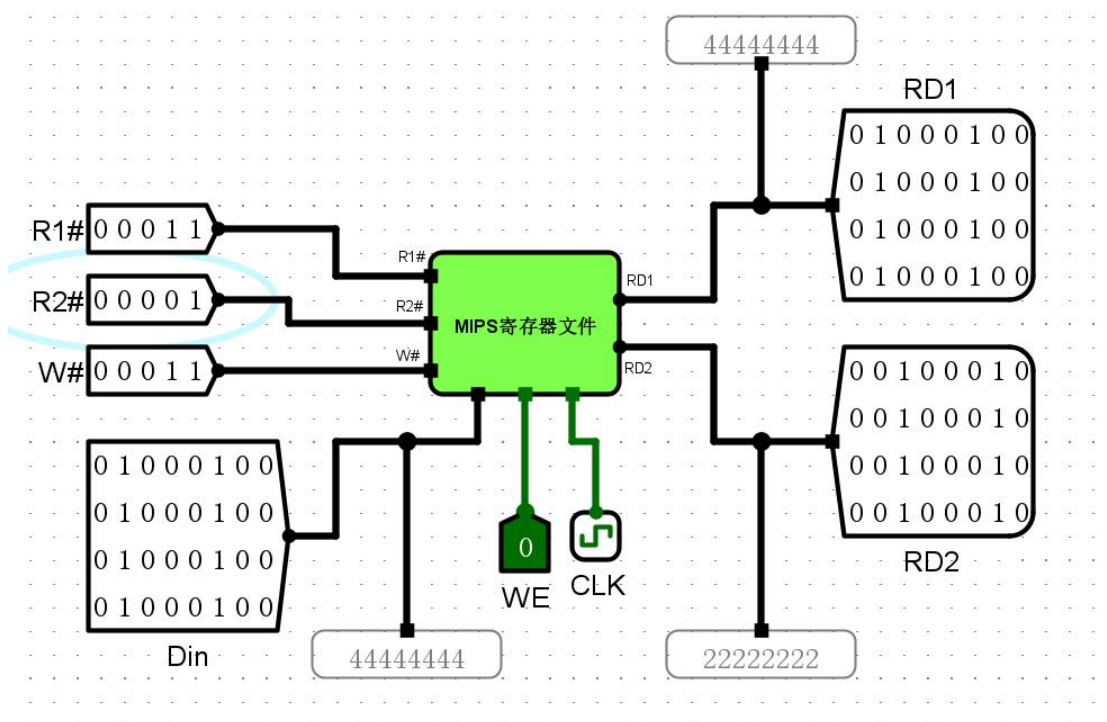


第 3 步：从 RD2 口读寄存器 0~寄存器 3。

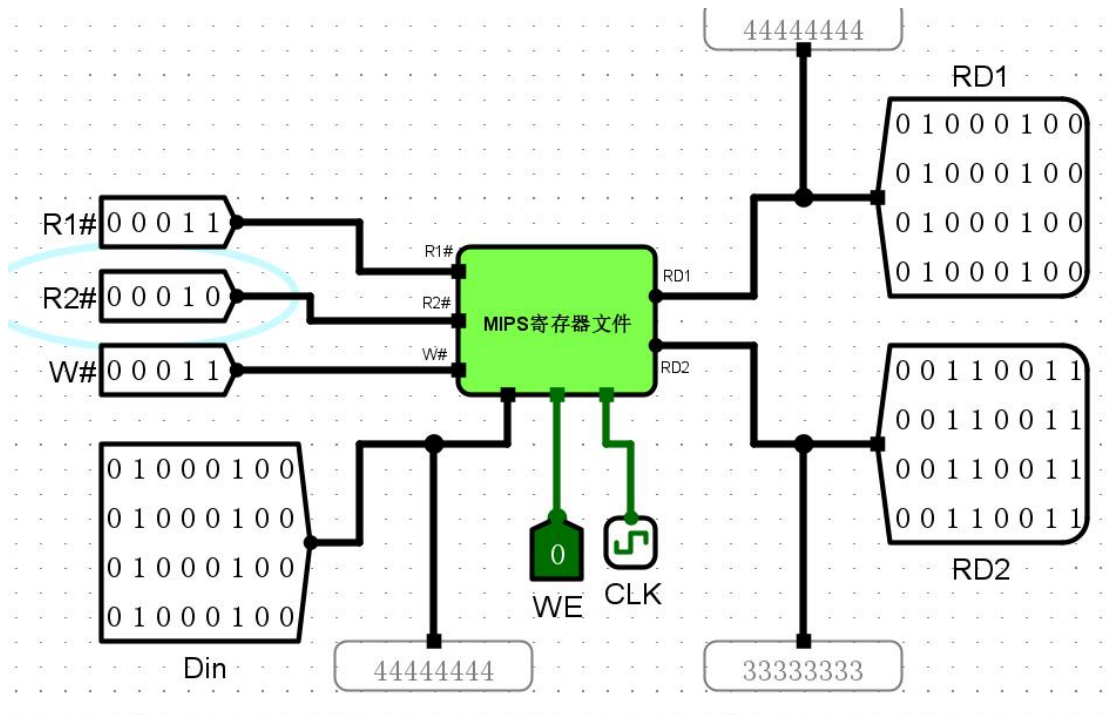
- 置 R2#=00000, WE=0, 观察 RD2 是否=11111111H



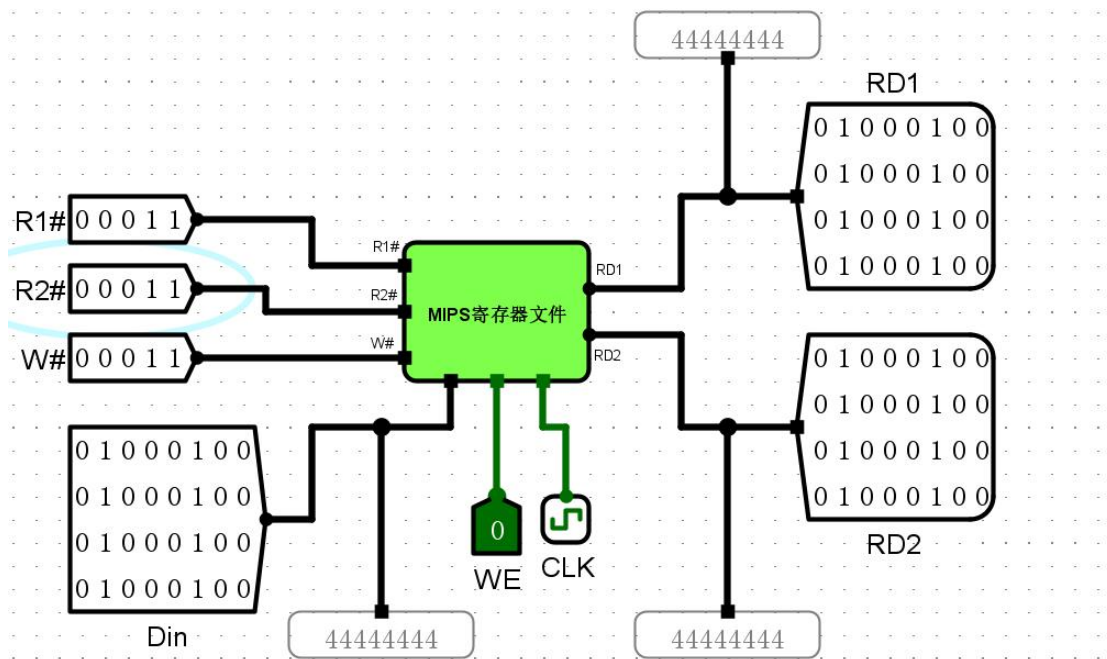
- 置 R2#=00001, WE=0, 观察 RD2 是否=22222222H



- 置 R2#=00010, WE=0, 观察 RD2 是否=33333333H

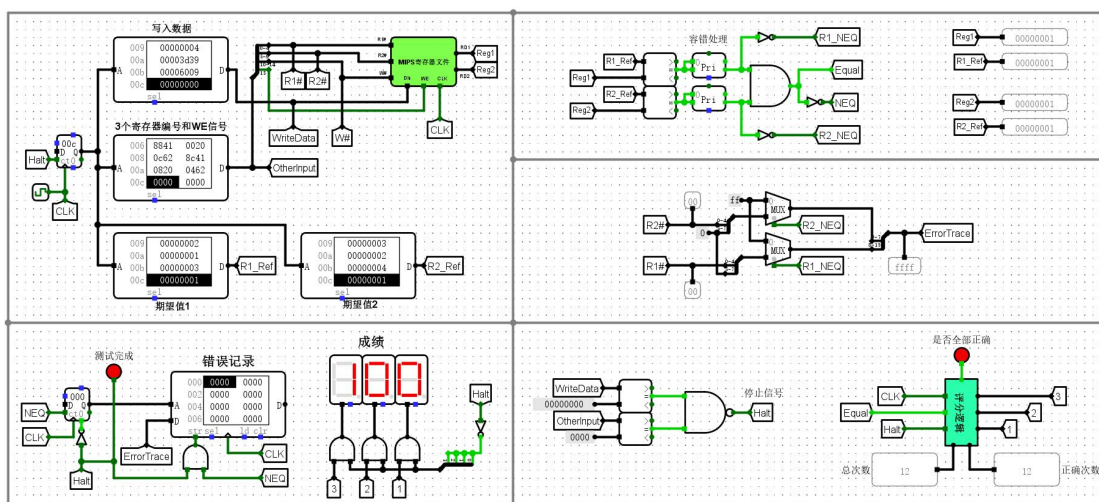


- 置 R2#=00011, WE=0, 观察 RD2 是否=44444444H



复杂的 MIPS 寄存器文件测试电路：

先按 Ctrl+R, 使电路复位；然后按 Ctrl+K, 启动时钟；如果显示“100”，则表示测试通过。

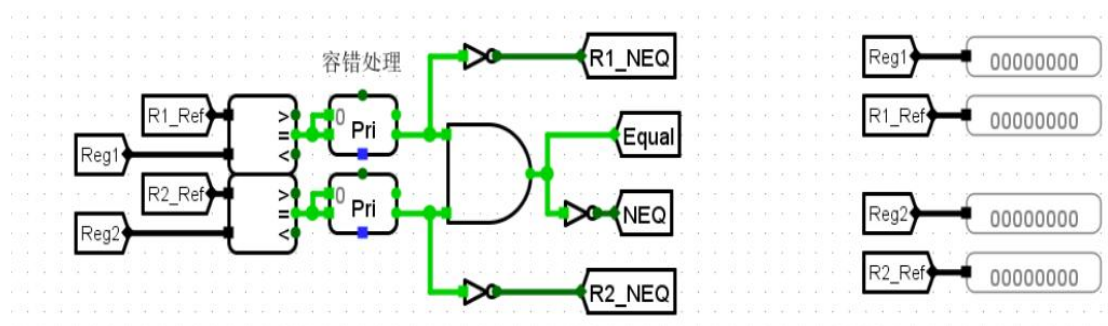


回答问题:

1) MIPS 寄存器文件电路中 3-8 译码器和 2-4 译码器的与 5:32 译码器是什么关系？结合电路图阐述原因

答：4 个 3-8 译码器和 1 个 2-4 译码器共同通过级联的方式，实现一个完整的 5:32 译码器功能。高 2 位接给 2-4 译码器，其 4 条输入线分别连接上 4 个 3-8 译码器的使能端口，根据高 2 位来控制激活哪个 3-8 译码器，如此设计能够降低电路的复杂度。

2) 复杂的 MIPS 寄存器文件测试电路中下述电路的作用什么？（提示：需借助 Pri 优先编码器进行分析）采用文字，截屏等方式阐述



答：Reg1 和 Reg2 是来自 MIPS 寄存器文件的实际读出数据，R1_Ref 和 R2_Ref 是预先存储在 ROM 中的标准参考值，通过比较器的结果进入 Pri 优先编码器，其用于屏蔽不关心的信号状态并进行容错处理，输出信号即为对数据是否正确的判断。

3) 复杂 MIPS 寄存器文件测试电路中，如果不是 100，说明什么？请针对性给出例子，并用电路运行结果进行佐证和分析。

答：说明测试结果与期望值不同，可能是 MIPS 寄存器文件电路有问题，修改写入

数据存储器的内容，修改期望值 1 存储器或期望值 2 存储器。

电路仿真 窗口 帮助

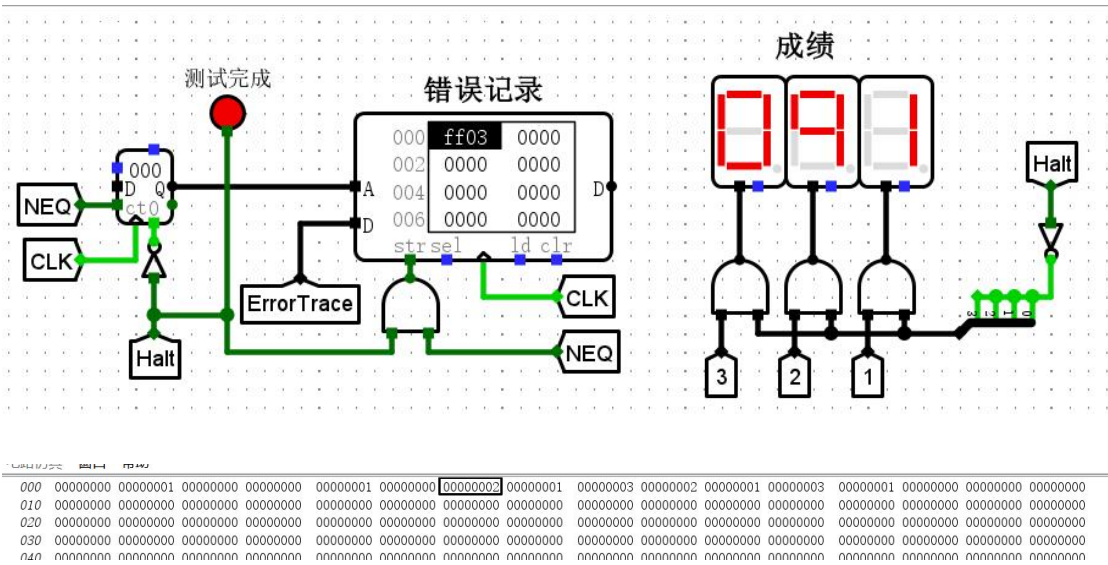
000	00000001	000041b8	00001bfe	00000002	000044a8	00001ca7	00000003	00005f7a	00002f2a	00000004	00003d39	00006009	00000000	00000000	00000000	00000000
010	00000000	00000000	00000000	00000000	00000000	00000000	00000000	00000000	00000000	00000000	00000000	00000000	00000000	00000000	00000000	00000000
020	00000000	00000000	00000000	00000000	00000000	00000000	00000000	00000000	00000000	00000000	00000000	00000000	00000000	00000000	00000000	00000000
030	00000000	00000000	00000000	00000000	00000000	00000000	00000000	00000000	00000000	00000000	00000000	00000000	00000000	00000000	00000000	00000000

修改写入数据存储器内容 4->5

工程 电路仿真 窗口 帮助

000	00000001	000041b8	00001bfe	00000002	000044a8	00001ca7	00000003	00005f7a	00002f2a	00000005	00003d39	00006009	00000000	00000000	00000000	00000000
010	00000000	00000000	00000000	00000000	00000000	00000000	00000000	00000000	00000000	00000000	00000000	00000000	00000000	00000000	00000000	00000000
020	00000000	00000000	00000000	00000000	00000000	00000000	00000000	00000000	00000000	00000000	00000000	00000000	00000000	00000000	00000000	00000000
030	00000000	00000000	00000000	00000000	00000000	00000000	00000000	00000000	00000000	00000000	00000000	00000000	00000000	00000000	00000000	00000000
040	00000000	00000000	00000000	00000000	00000000	00000000	00000000	00000000	00000000	00000000	00000000	00000000	00000000	00000000	00000000	00000000

运行

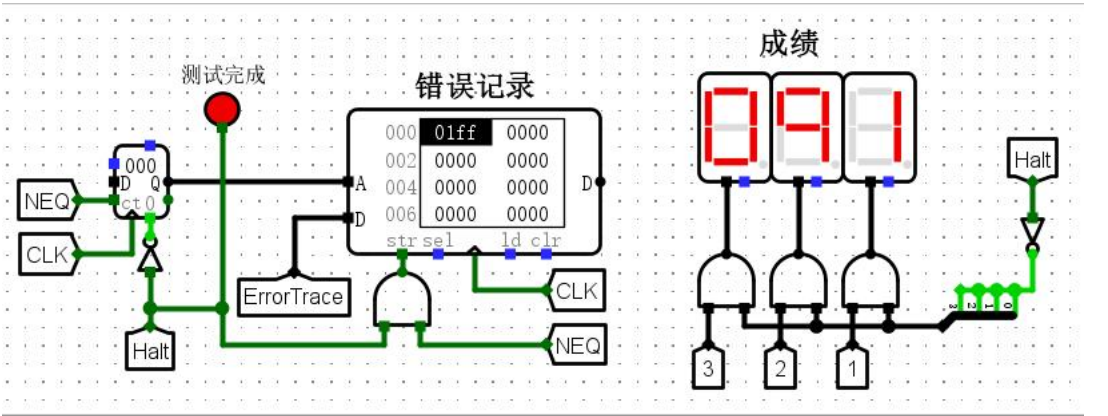


恢复写入数据存储器内容，修改期望值 1 存储器内容 2->1

工程 电路仿真 窗口 帮助

000	00000000	00000001	00000000	00000000	00000001	00000000	00000000	00000001	00000003	00000002	00000001	00000003	00000001	00000000	00000000	00000000
010	00000000	00000000	00000000	00000000	00000000	00000000	00000000	00000000	00000000	00000000	00000000	00000000	00000000	00000000	00000000	00000000
020	00000000	00000000	00000000	00000000	00000000	00000000	00000000	00000000	00000000	00000000	00000000	00000000	00000000	00000000	00000000	00000000
030	00000000	00000000	00000000	00000000	00000000	00000000	00000000	00000000	00000000	00000000	00000000	00000000	00000000	00000000	00000000	00000000

运行



电路仿真 窗口 帮助

```

000 00000000 00000000 00000000 00000000 00000002 00000000 00000000 00000002 00000000 00000003 00000002 00000004 00000001 00000000 00000000 00000000
010 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000
020 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000
030 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000
040 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000
050 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000

```

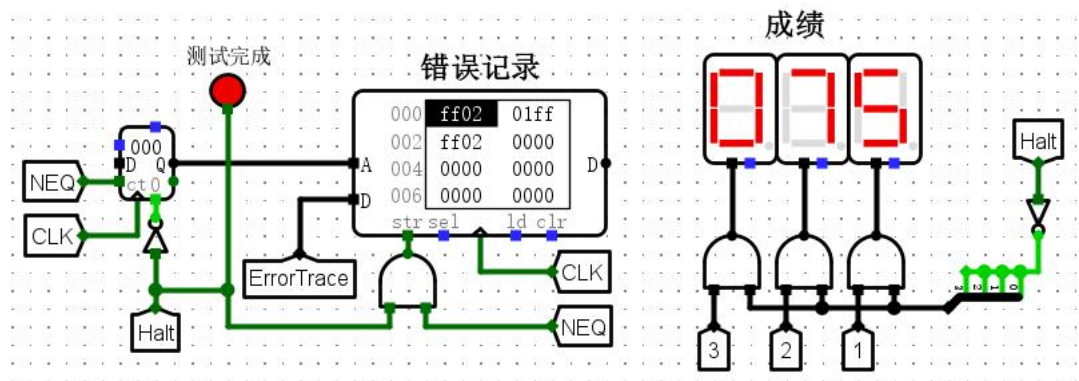
修改期望值 2 存储器内容 0->1, 3->2

```

000 00000001 00000000 00000000 00000000 00000002 00000000 00000000 00000002 00000000 00000003 00000002 00000004 00000001 00000000 00000000 00000000
010 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000
020 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000
030 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000
040 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000

```

运行



4) 分析评分逻辑电路的工作原理（输入输出之间的逻辑关系，即：high, mid, low 以及 right 这些逻辑变量与输入以及逻辑电路之间的关系，可利用组合电路逻辑表达式）

答：输入：EQ：当寄存器输出与预期一致时为 1；EN：控制计数器是否工作；CLK：时钟信号。

Total：记录测试运行的总周期数。

正确条数：记录 EQ 有效时的累加值。

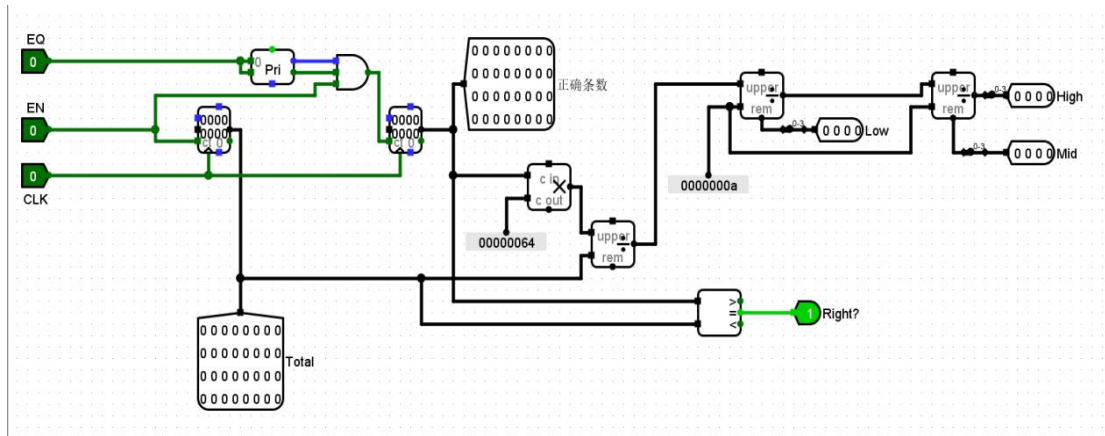
Right? = (Total==正确条数)

正确率=正确条数*100/Total

Low=正确率%10

Mid=(正确率/10)%10

High=(正确率/100)%10

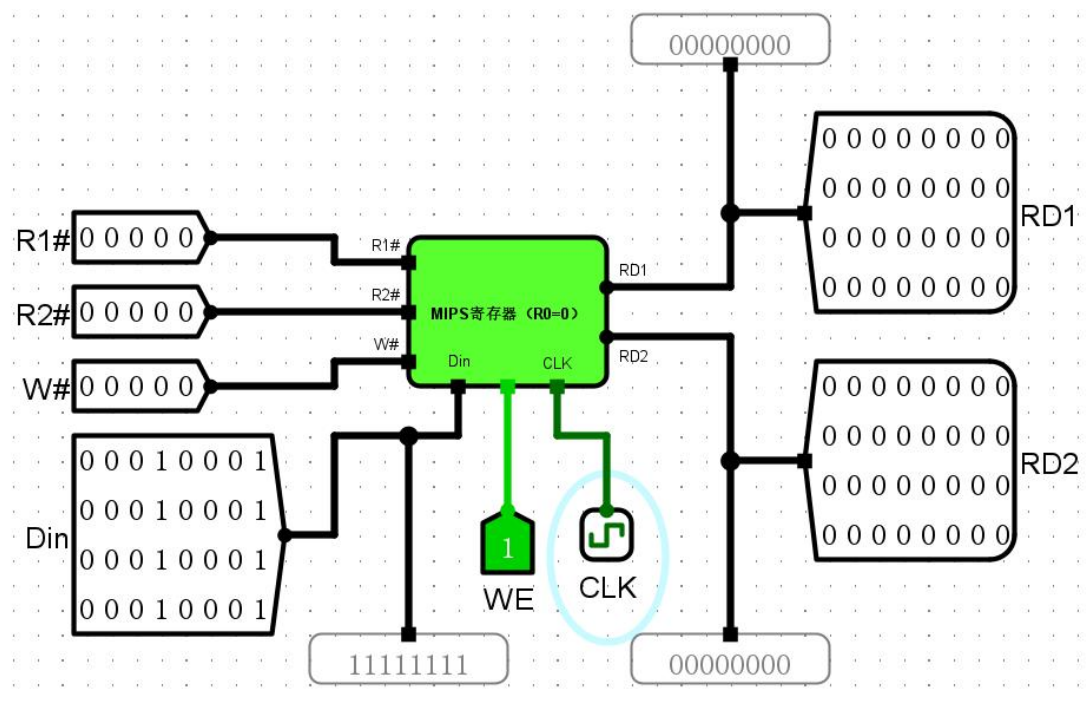


(5) MIPS 寄存器文件电路 (R0=0)、简单的 MIPS 寄存器文件测试电路 (R0=0)、复杂的 MIPS 寄存器文件测试电路 (R0=0)。

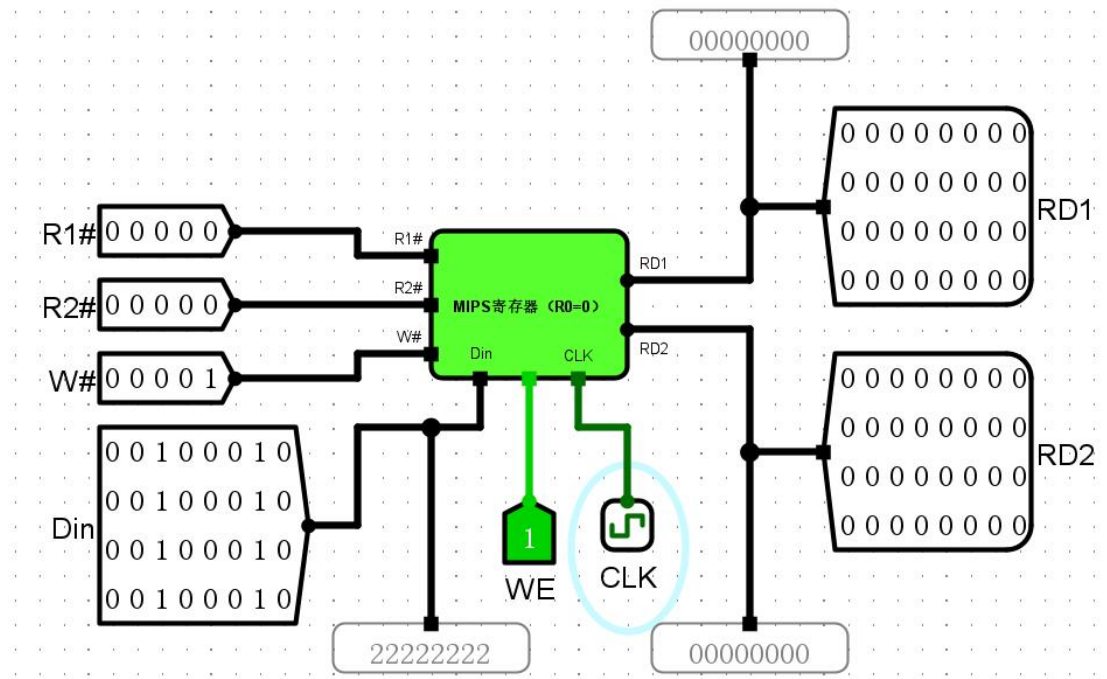
简单的 MIPS 寄存器文件测试电路 (R0=0)：

第 1 步：向寄存器 0~寄存器 3，分别写入 11111111、 22222222、 33333333、 44444444（均为十六进制，下同）。

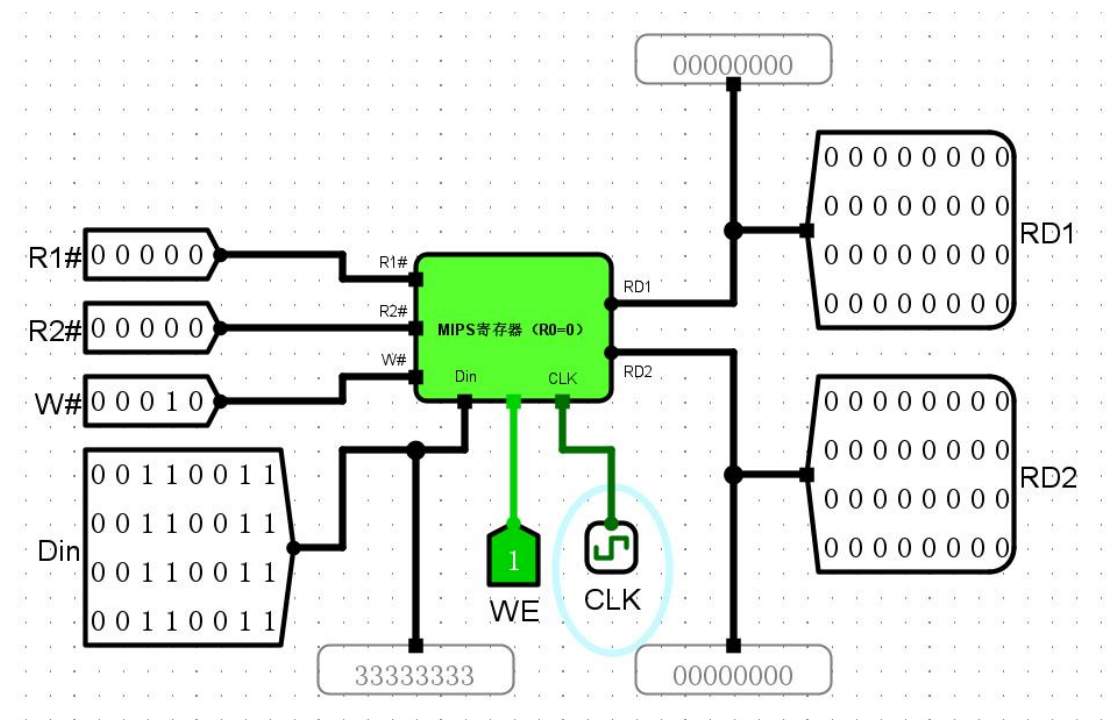
- 置 W#=00000， Din=11111111H， WE=1，按时钟 CLK 按钮 2 次



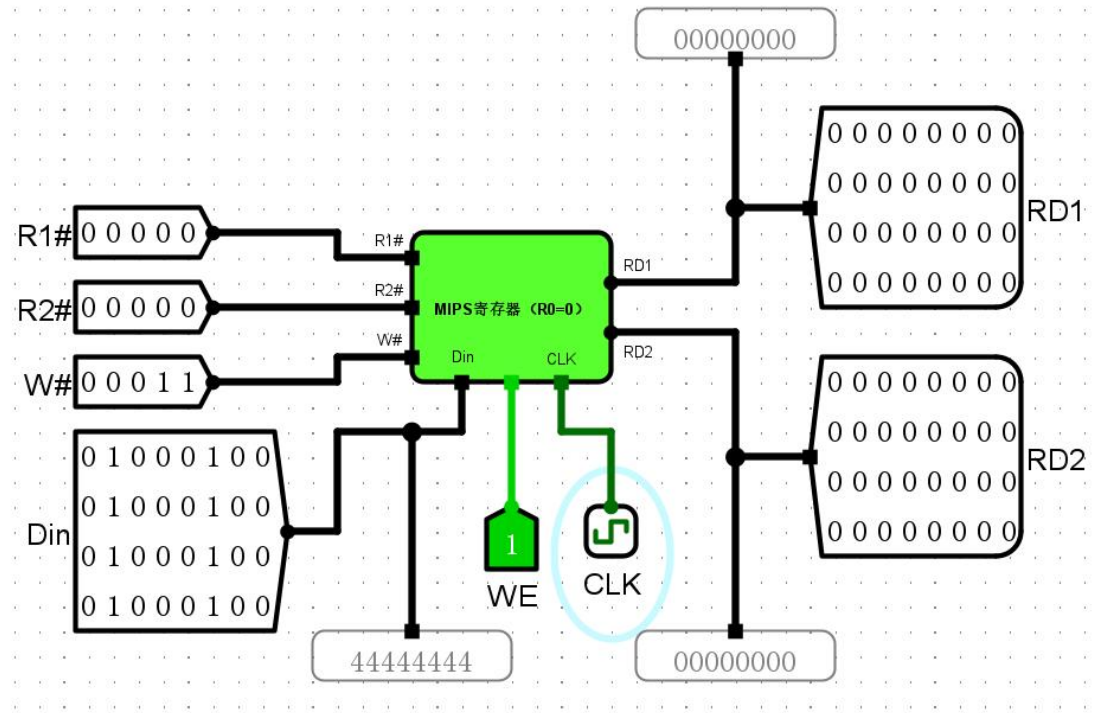
- 置 W#=00001， Din=22222222H， WE=1，按时钟 CLK 按钮 2 次



- 置 W#=00010, Din=33333333H, WE=1, 按时钟 CLK 按钮 2 次

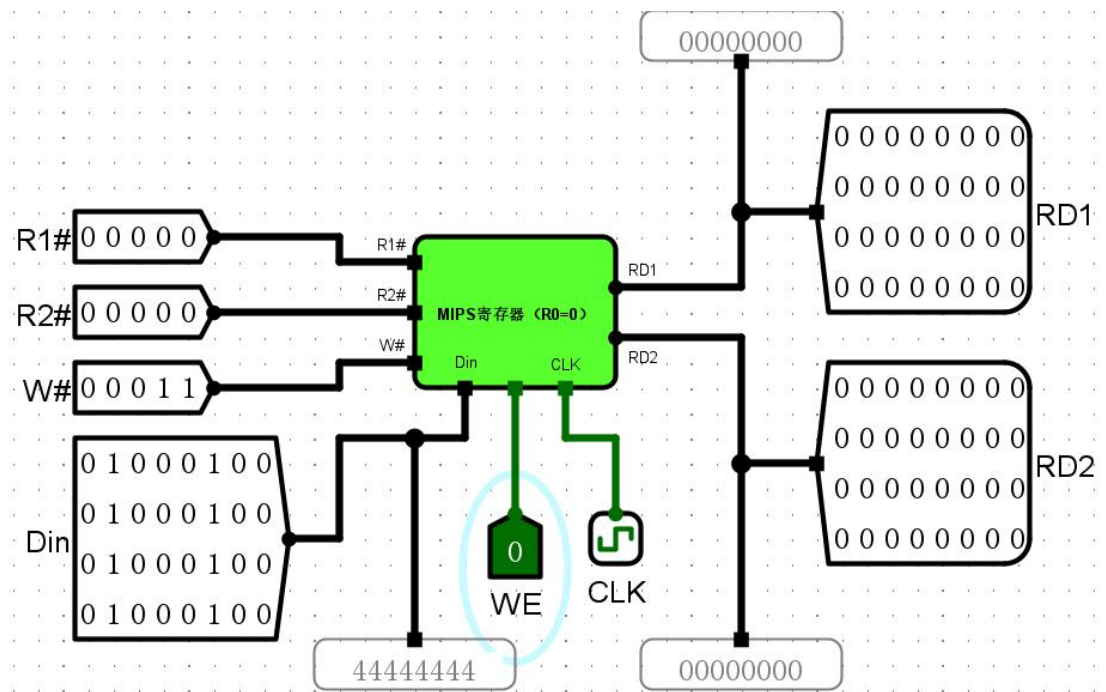


- 置 W#=00011, Din=44444444H, WE=1, 按时钟 CLK 按钮 2 次

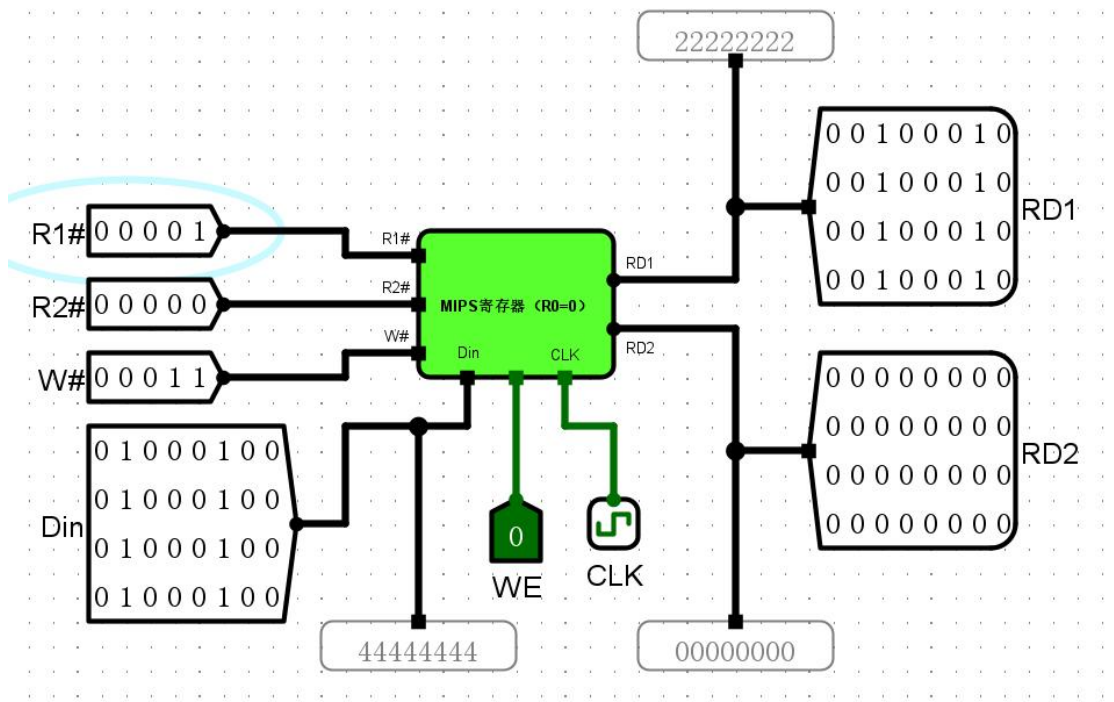


第2步：从RD1口读寄存器0~寄存器3。

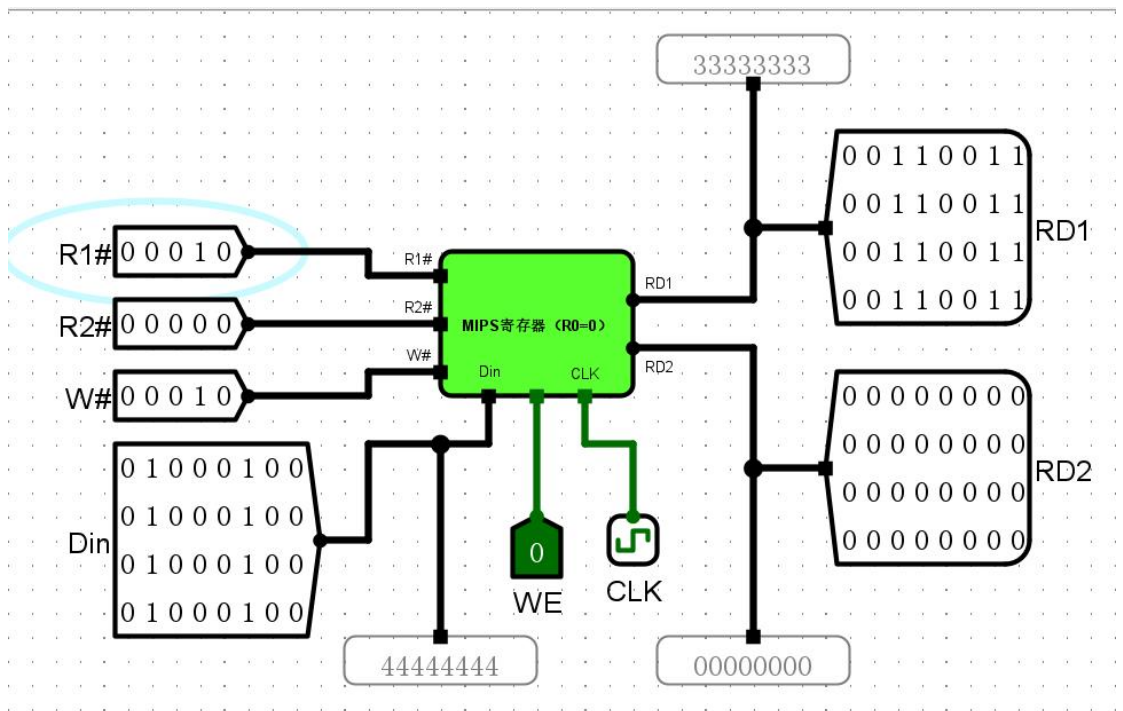
- 置 R1#=00000， WE=0， 观察 RD1 是否=00000000H（虽然前面向 R0 中写入了 11111111H，但是这里读出的 R0 仍然为 0）



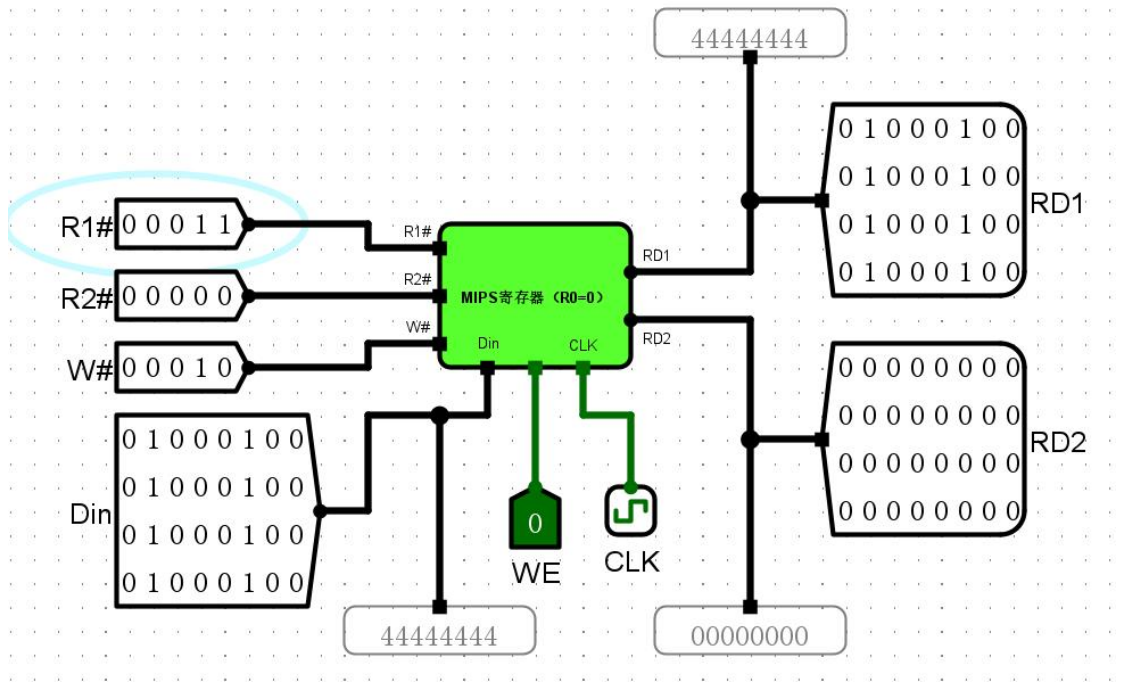
- 置 R1#=00001， WE=0， 观察 RD1 是否=22222222H



- 置 R1#=00010, WE=0, 观察 RD1 是否=33333333H

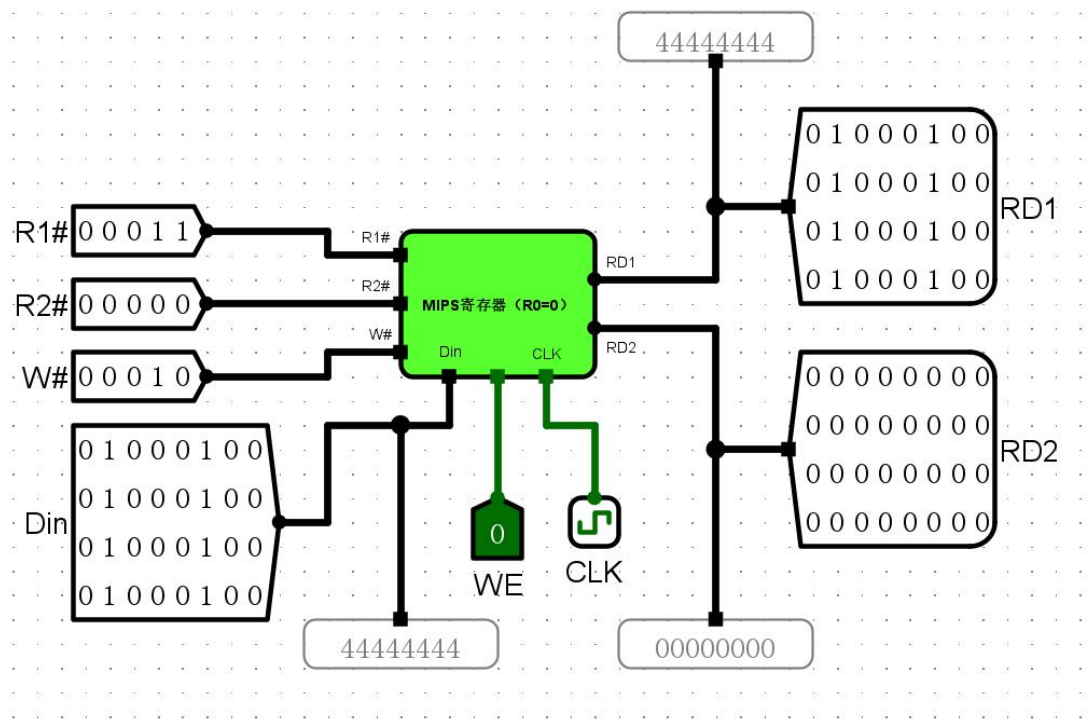


- 置 R1#=00011, WE=0, 观察 RD1 是否=44444444H

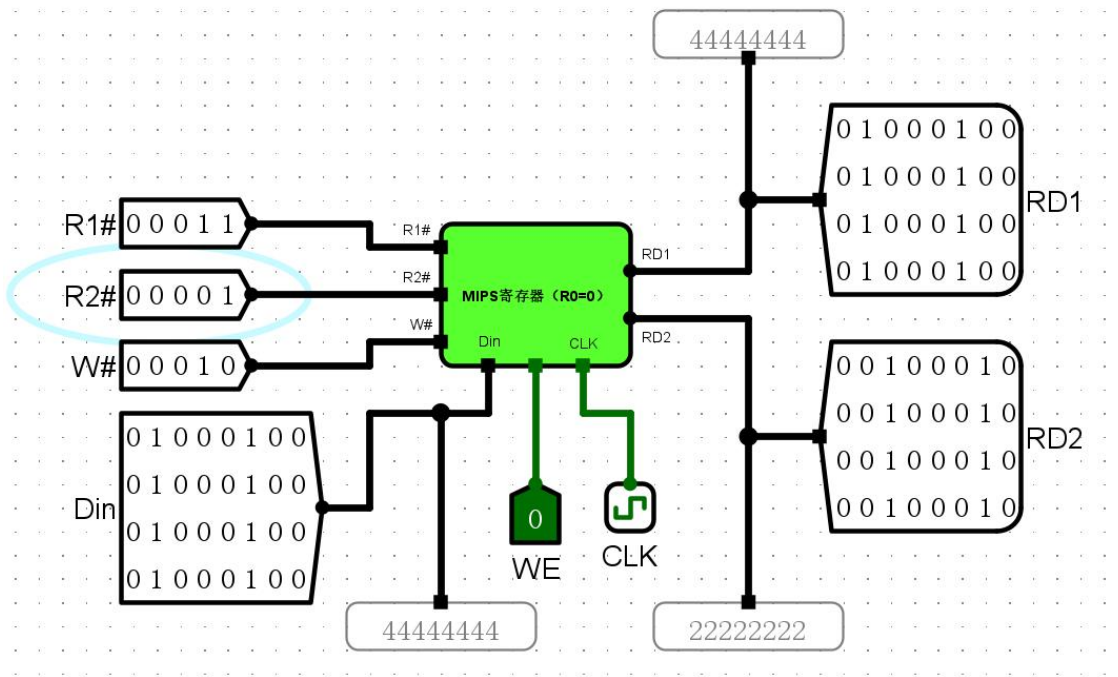


第3步：从RD2口读寄存器0~寄存器3。

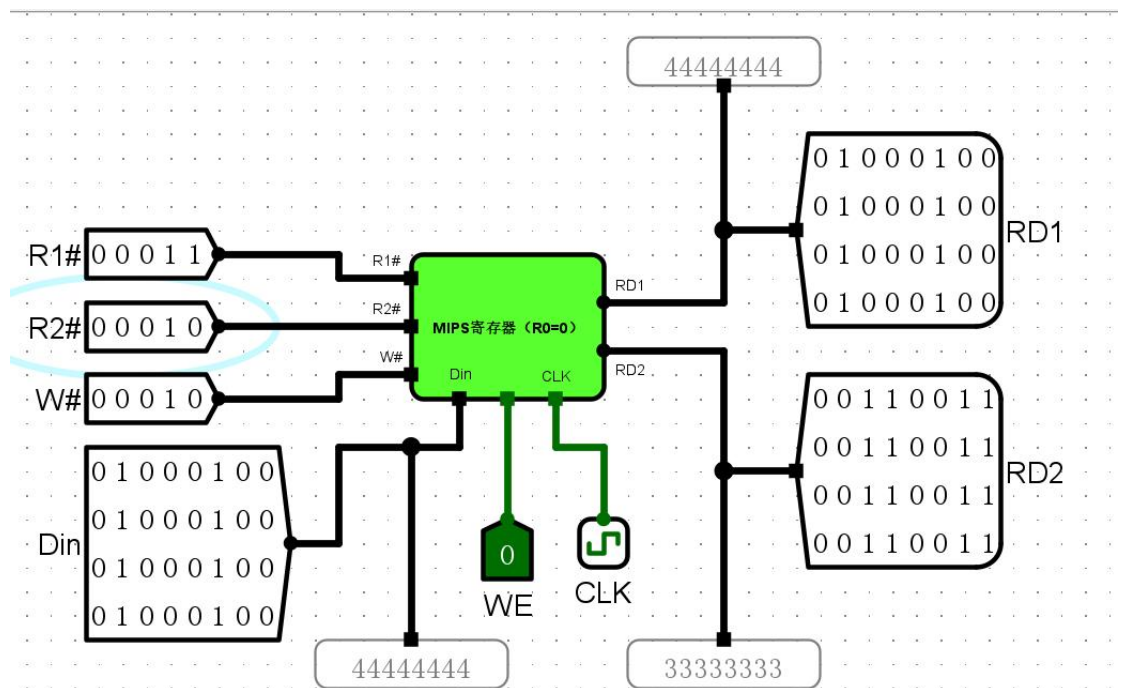
- 置 R2#=00000， WE=0， 观察 RD2 是否=00000000H（虽然前面向 R0 中写入了 11111111H， 但是这里读出的 R0 仍然为 0）



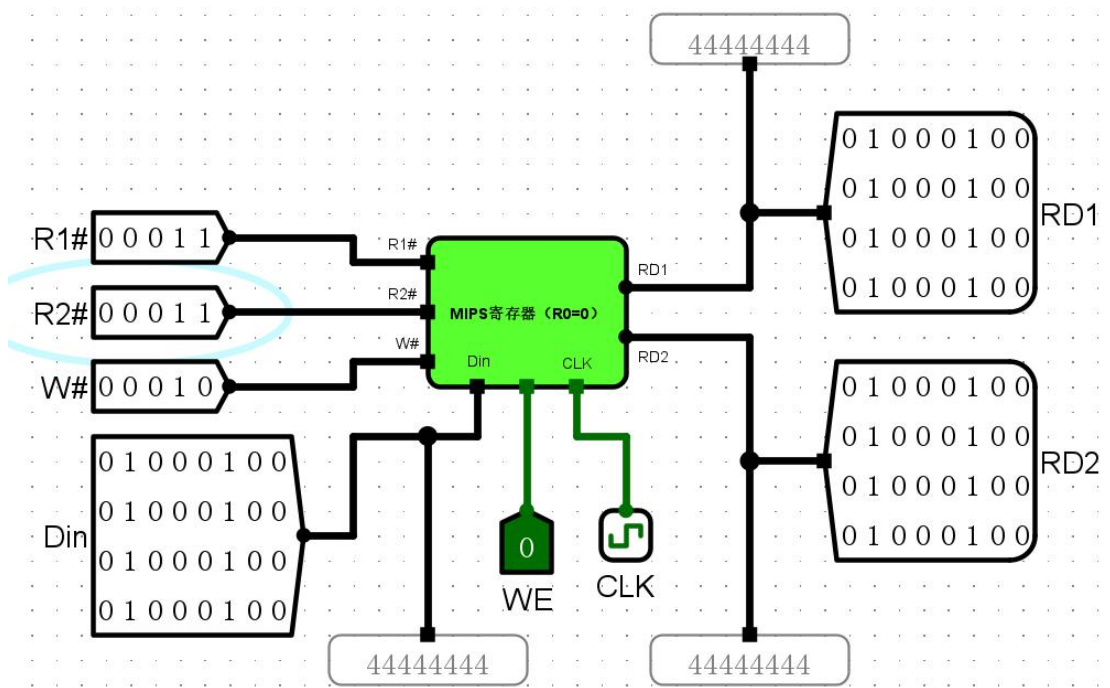
- 置 R2#=00001， WE=0， 观察 RD2 是否=22222222H



- 置 R2#=00010, WE=0, 观察 RD2 是否=33333333H

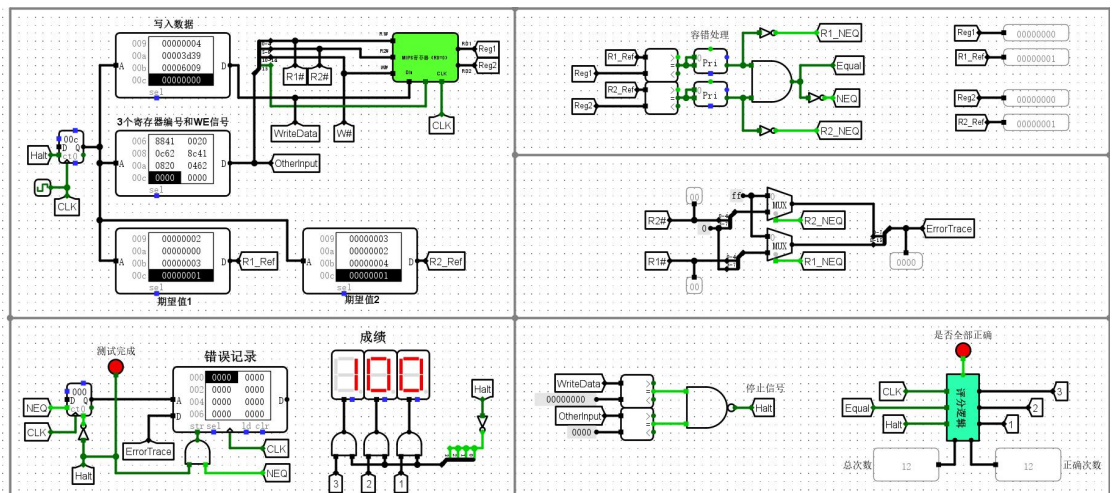


- 置 R2#=00011, WE=0, 观察 RD2 是否=44444444H



复杂的 MIPS 寄存器文件测试电路（R0=0）：

先按 Ctrl+R，使电路复位；然后按 Ctrl+K，启动时钟；如果显示“100”，表示测试通过。



回答问题：

1) 为了控制 R0 寄存器恒等于 0，在电路设计上采取了什么方式或者手段？

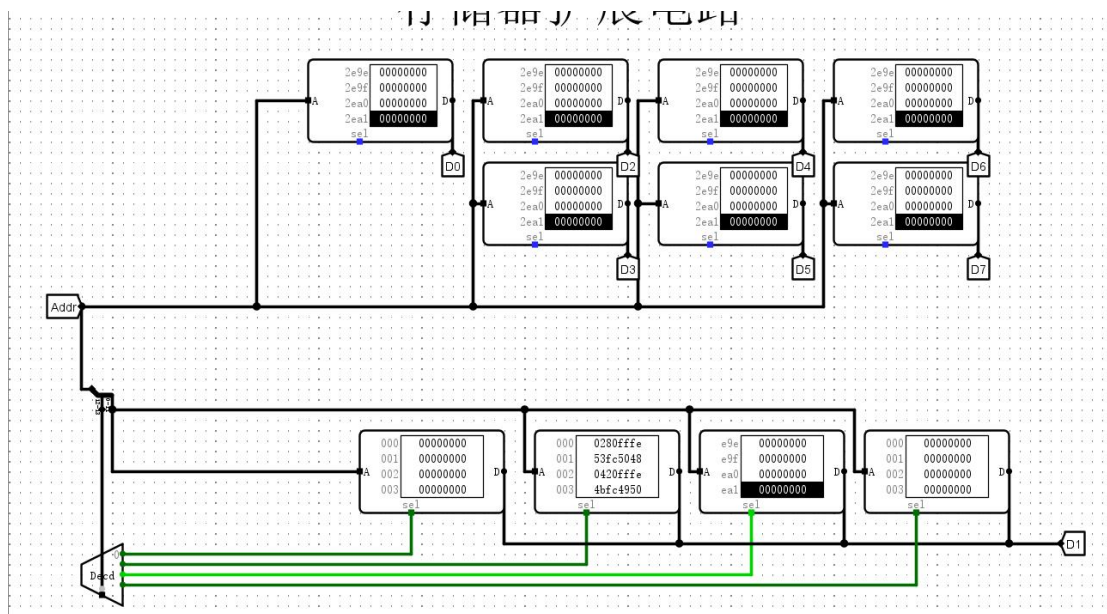
答：使用 5-32 译码器，首个寄存器不接入 Din，而是接入常数 0。

2) 分析 MIPS 寄存器文件电路（R0=0），阐述该电路是如何实现“从 R1/R2 口读出某个寄存器的内容，保证 R0=0”。

答：R1 和 R2 多路选择器对应相同的寄存器，当它们为 00000 时，读取首个寄存器的值，该寄存器存储常数 0，因此保证了 R0=0。

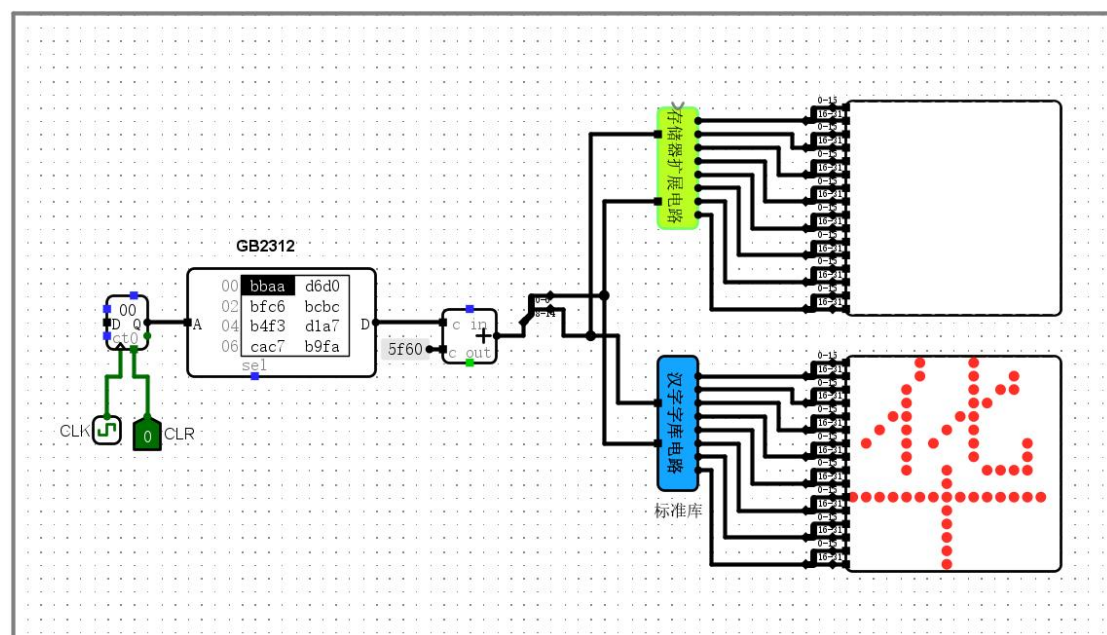
3.2 设计实验

(1) 存储器扩展电路、存储器扩展测试电路——设计文件名：**存储器扩展设计实验.circ**。



测试步骤：

① 将设计好的存储器扩展电路进行封装，然后放置到存储器扩展测试电路中。

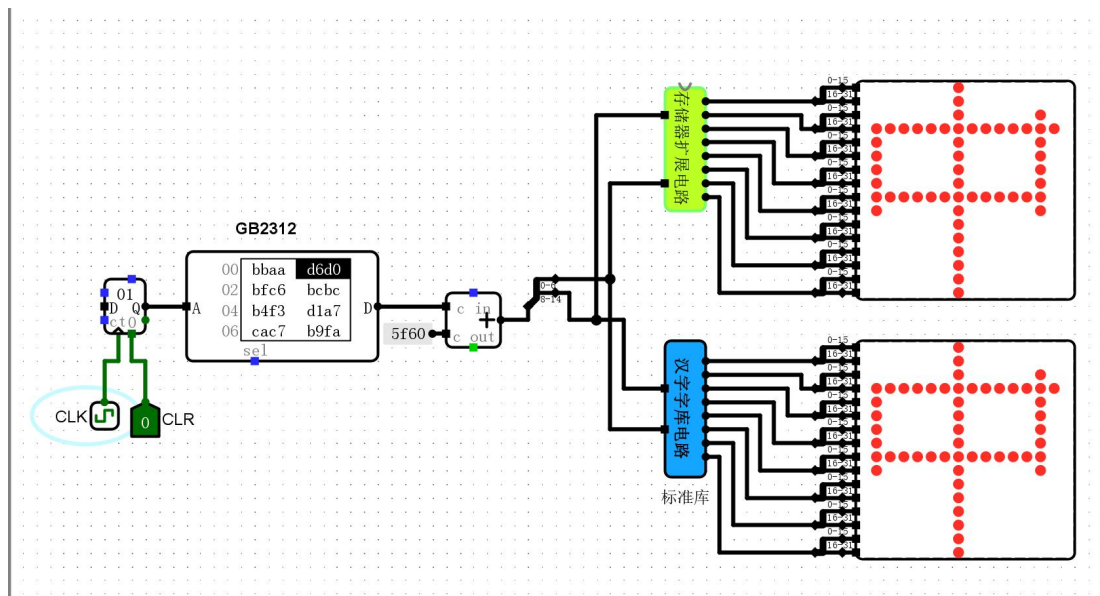


② 将“D0 存储器的内容.txt”、“D2 存储器的内容.txt”、“D3 存储器的内容.txt”、“D4 存储器的内容.txt”、“D5 存储器的内容.txt”、“D6 存储器的内容.txt”、“D7 存储器的

内容.txt”中的内容分别装入到设计好的存储器扩展电路的 7 个 16Kx32 位 ROM 存储器中。

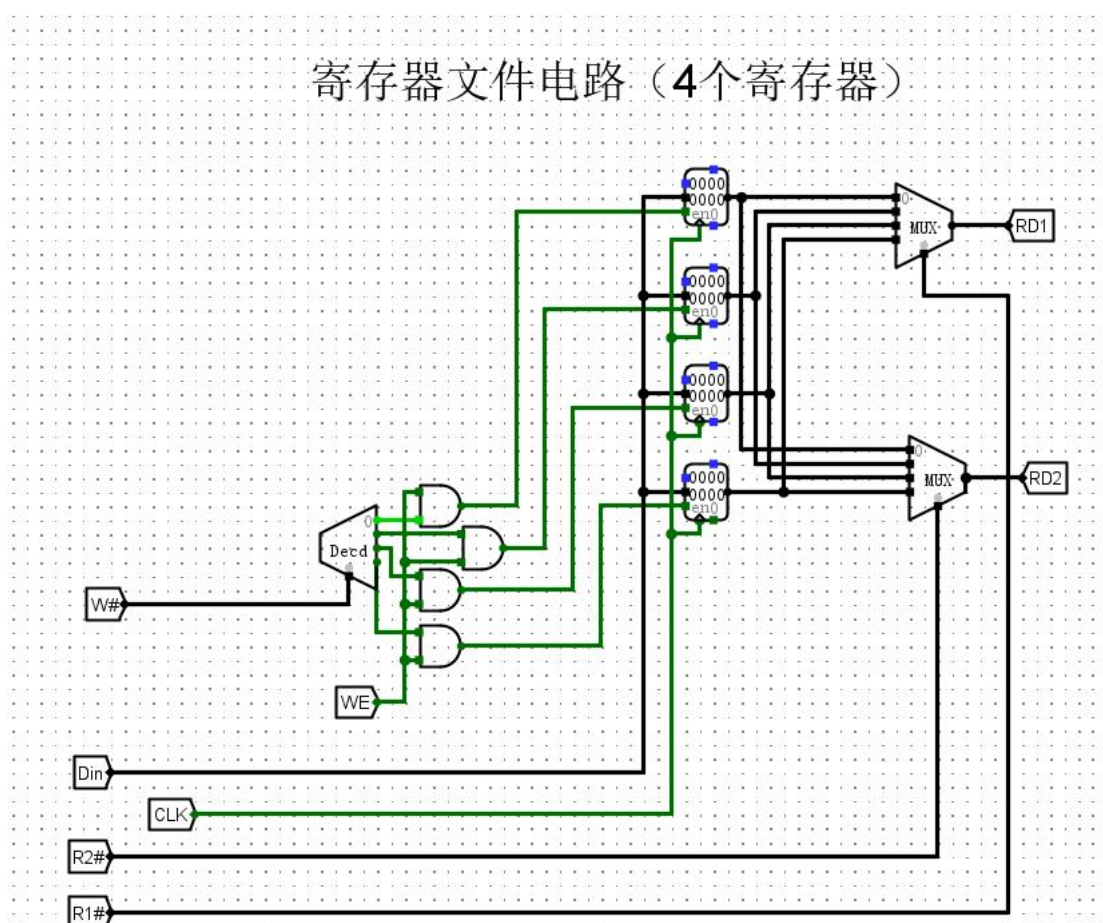
文件	编辑	工程	电路仿真	窗口	帮助
0000	00000000	00000000	00000000	00000000	00000000
0010	00000000	00000000	00000000	00000000	00000000
0020	00000000	00000000	00000000	00000000	00000000
0030	0c1803e0	00000000	00000000	00000000	00000000
0040	00800000	01c00000	01c00000	00200000	00900000
0050	03c00000	0c1803e0	01400080	01c00080	00000000
0060	00000000	00000000	00000000	00000000	00000000
0070	00000000	00800000	00000000	00000000	00000000
0080	00000000	00000000	00800000	00800000	00800000
0090	10080000	10080000	10080000	10080000	10080000
00a0	00000000	00000000	10080000	10080000	10080000
00b0	00000000	00000000	00000000	00000000	00000000
00c0	00000000	00000000	00000000	00040000	20000000
00d0	00000000	00000000	00000000	00000000	00000000
00e0	00000000	00000000	00000000	00000000	00000000
00f0	00000000	00000000	00000000	00000000	00000000
0100	00000000	00000000	00000000	00000000	00000000
0110	00000000	00000000	00000000	00000000	00000000
0120	00000000	00000000	00000000	00000000	00000000
0130	00000000	00100000	00800000	00860008	00000000
0140	00030004	00000000	00000000	00000000	00000000
0150	00000000	00000000	00100000	00000000	00f80000
0160	00000000	00800000	00000000	00000000	00800000
0170	00000000	00000000	00000000	00000000	00000000
0180	00000000	00100000	01100000	01160008	02200000
0190	00000000	00060008	00000000	00060008	00000000
01a0	02060008	00000000	00000000	00000000	02200000
01b0	00060008	00040000	00000000	00060008	00020000
01c0	00000000	00400000	00000000	00000000	00000000
01d0	00000000	00000000	00000000	00000000	00000000
01e0	00800000	00000000	00000000	00000000	00000000
01f0	00000000	00000000	00000000	00000000	00000000
0200	00000000	00000000	00000000	00400000	00000000
0210	00000000	00000000	00000000	00000000	00000000
0220	00000000	00000000	00000000	00000000	00000000
0230	00000000	00000000	00000000	00000000	00000000
0240	00000000	00000000	00000000	00000000	00000000
0250	00000000	00000000	00000000	00000000	00000000
0260	00000000	00000000	00000000	00000000	00000000
0270	00000000	00000000	00000000	00000000	00000000
0280	00000000	00000000	00000000	00000000	00000000
0290	00000000	00000000	00000000	00800000	04400000
02a0	04400000	02200000	00000000	00800000	02800440
02b0	02000000	00000000	00000000	00000000	00000000
02c0	00000000	00200000	00800000	00000000	01000000
02d0	00000000	02000000	00000000	00000000	01000000
02e0	00000000	00000000	00000000	00000000	00000000
02f0	00000000	00000000	00000000	00000000	00000000
0300	00000000	00000000	00000000	00000000	00000000
0310	01000100	01800180	01000100	01800180	01800180
0320	00000000	00000000	00000000	00000000	01000100
0330	01000100	01000100	01000100	01800180	01000100
0340	00000000	00000000	00000000	00000000	00000000
0350	00000000	00000000	00000000	00000000	00000000
0360	00000000	00200000	00000000	00000000	00000000
0370	00000000	00000000	00000000	00000000	00000000
0380	00000000	00000000	00000000	00000000	00000000
0390	00000000	00000000	00000000	00000000	00000000
03a0	00000000	00000000	00000000	00000000	00000000
03b0	00000000	00000000	00000000	00080000	10000000
03c0	00000000	00000000	00000000	00000000	00000000
03d0	00000000	00000000	00000000	00000000	00000000
03e0	00000000	00000000	00000000	00000000	00000000
03f0	00000000	00000000	00000000	00000000	00000000
0400	00000000	00000000	00000000	00000000	00000000

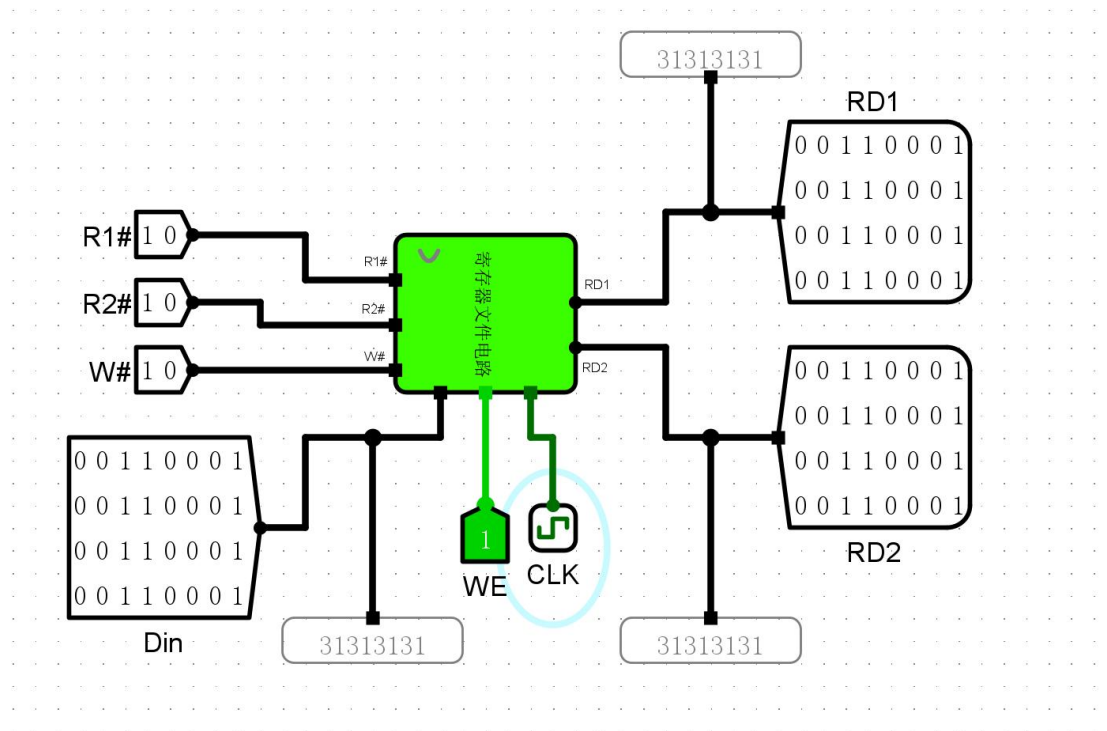
③ 将“D1-0 存储器的内容.txt”、“D1-1 存储器的内容.txt”、“D1-2 存储器的内容.txt”、“D1-3 存储器的内容.txt” 中的内容分别装入到设计好的存储器扩展电路的 4 个 4Kx32 位 ROM 存储器中。



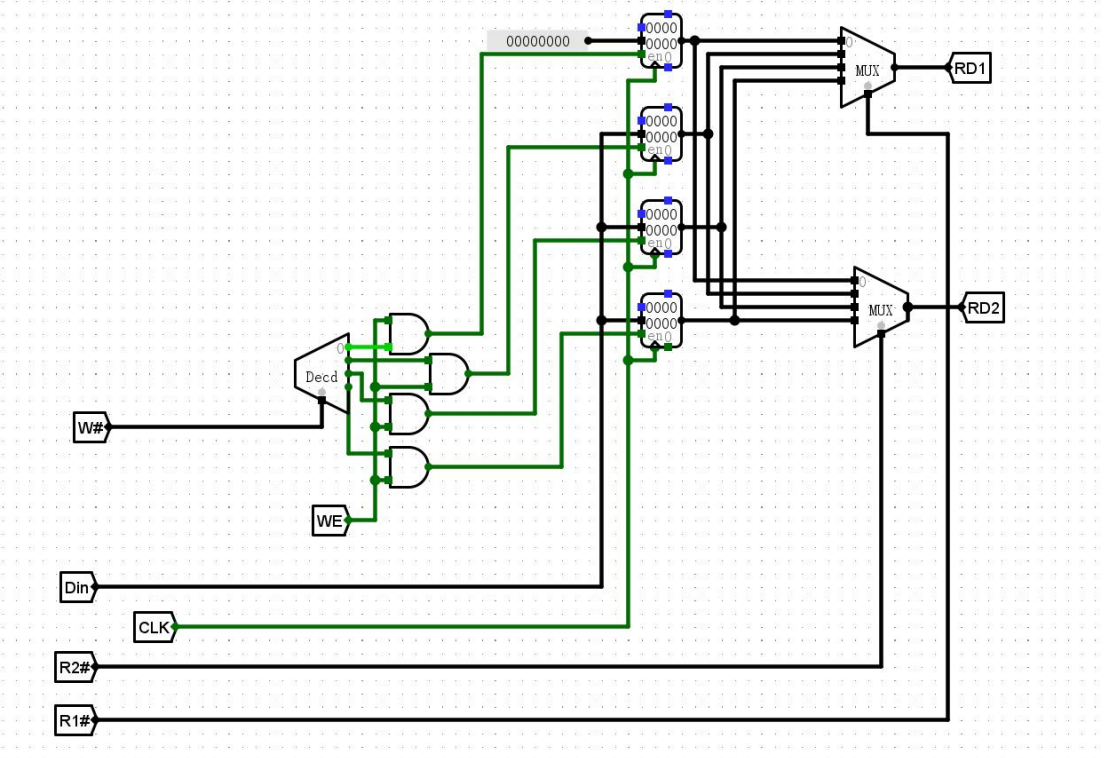
⑥ 如果上下两个 LED 点阵显示的汉字是相同的，则表示存储器扩展电路是正确的；否则，要么是存储器扩展电路设计不正确，要么是第②、③步的汉字点阵码没有装入正确。

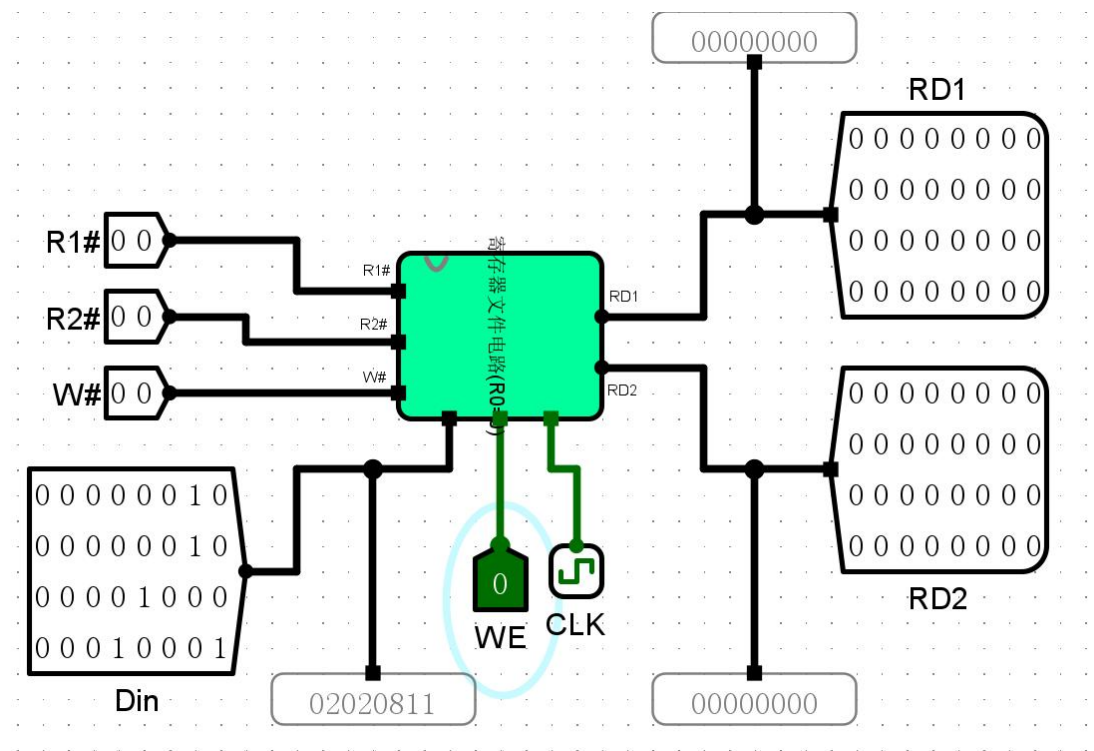
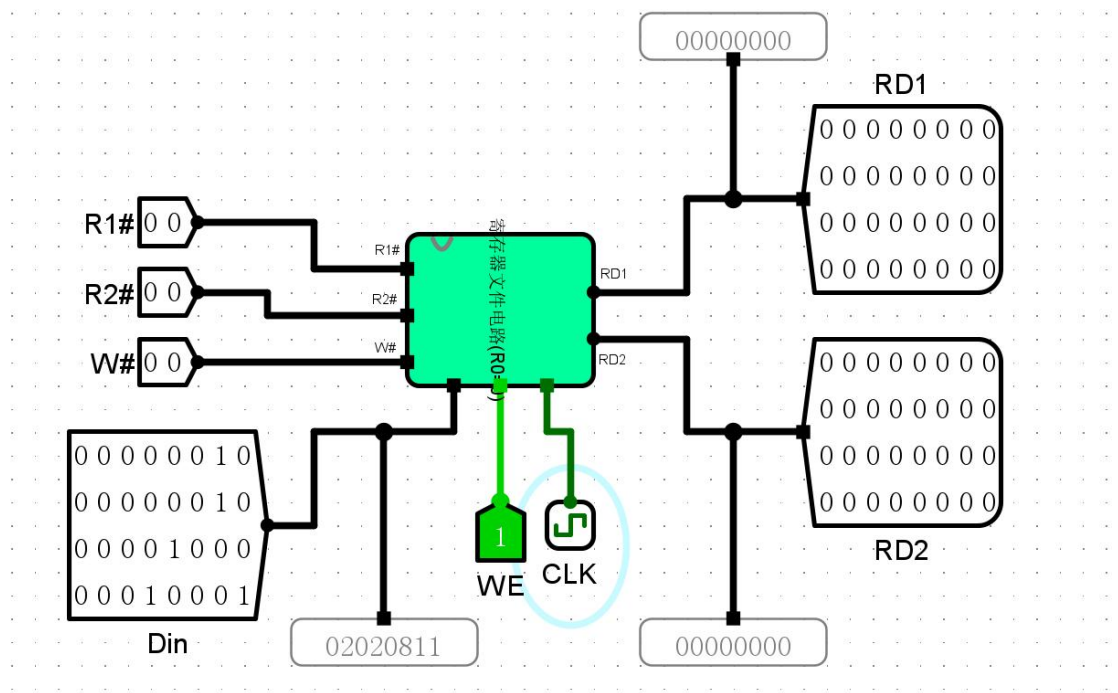
(2) 寄存器文件电路（4 个寄存器）、寄存器文件电路（4 个寄存器，R0=0）——设计文件名：寄存器文件设计实验.circ。

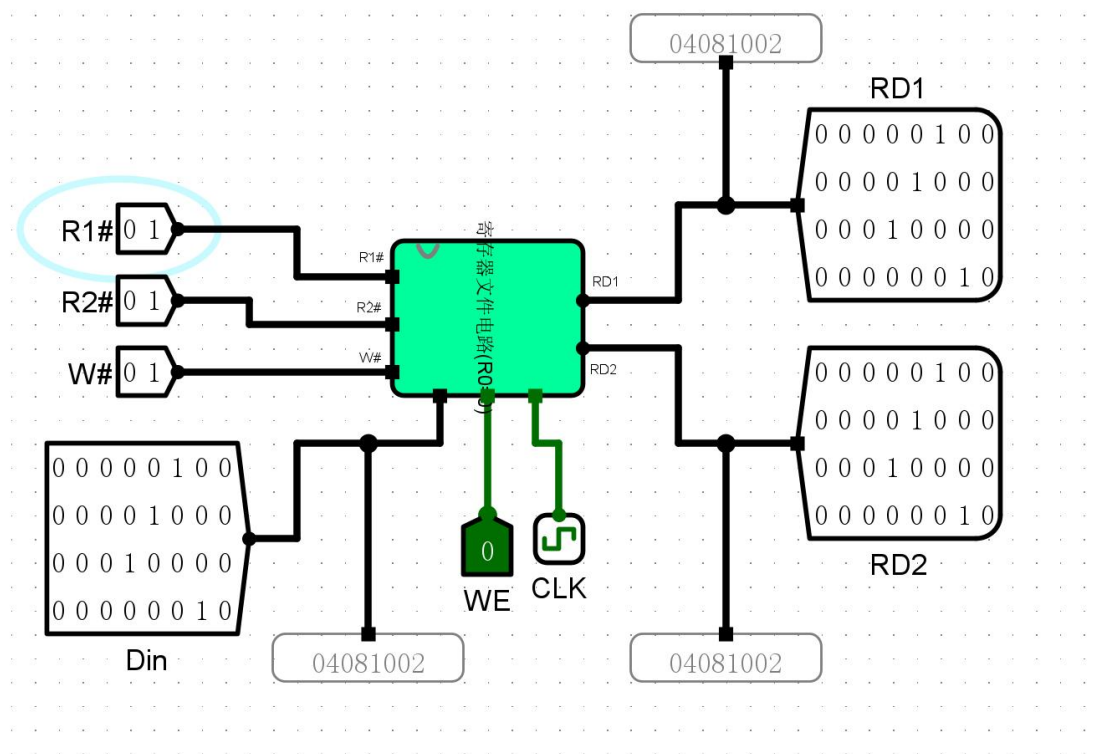




寄存器文件电路（4个寄存器，R0=0）







3.3 挑战性实验

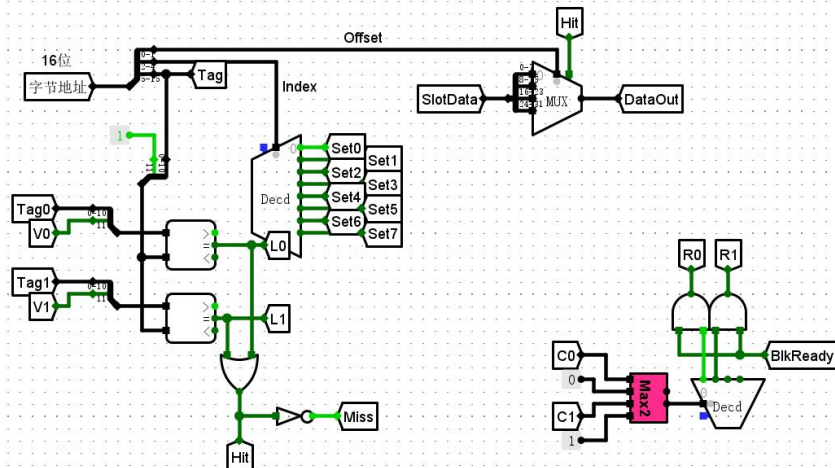
第 3 组: 2 路组相联映射方式 cache 电路 (16 个 cache 块)

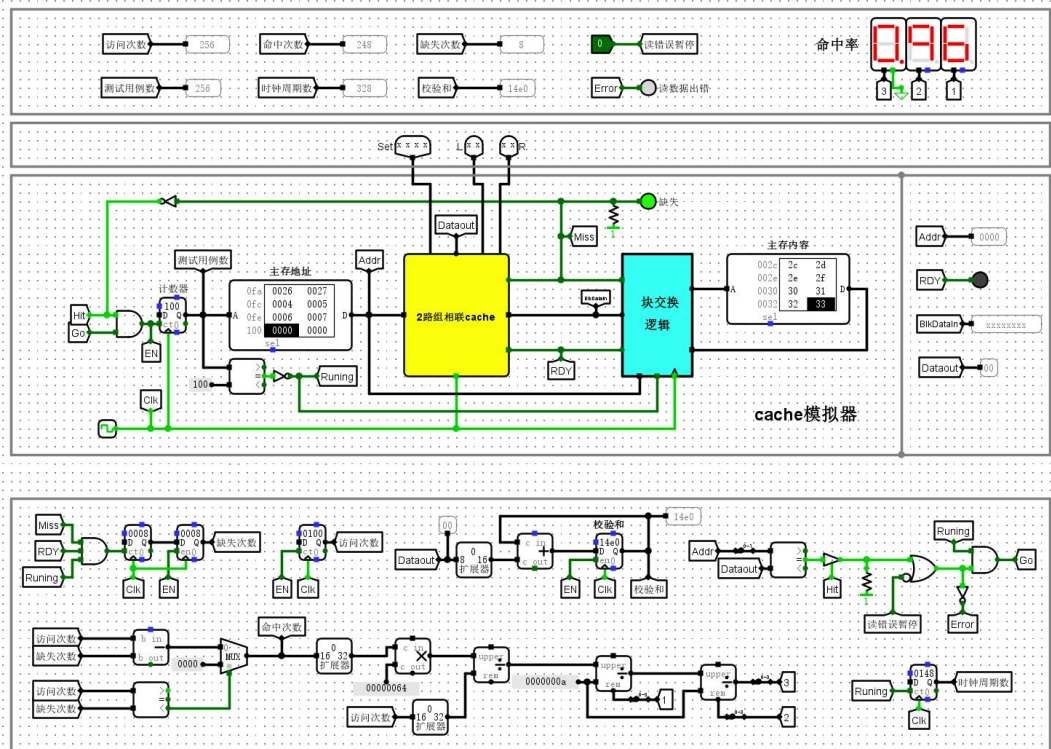
直接相联映射方式 cache 电路 (16 个 cache 块)

2 组相联映射方式 cache 电路 (16 个 cache 块):

cache（2路组相联映射）电路（16个cache块）

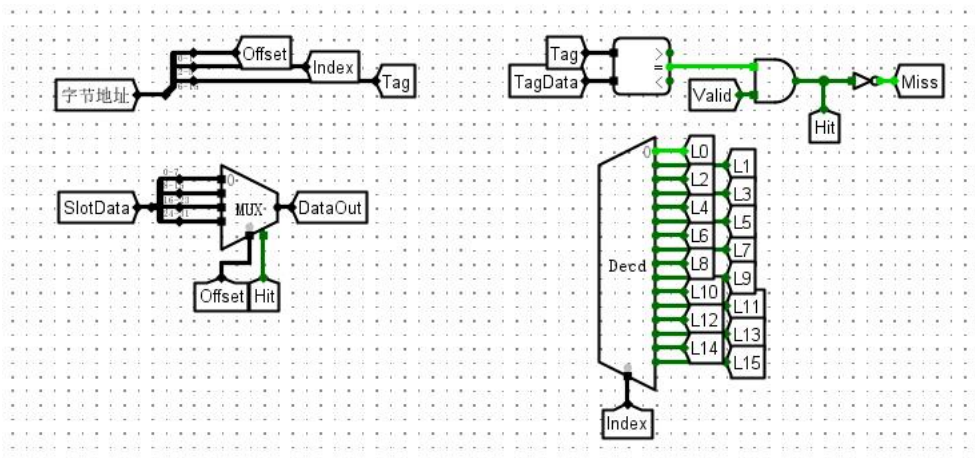
请同学们在此设计电路！

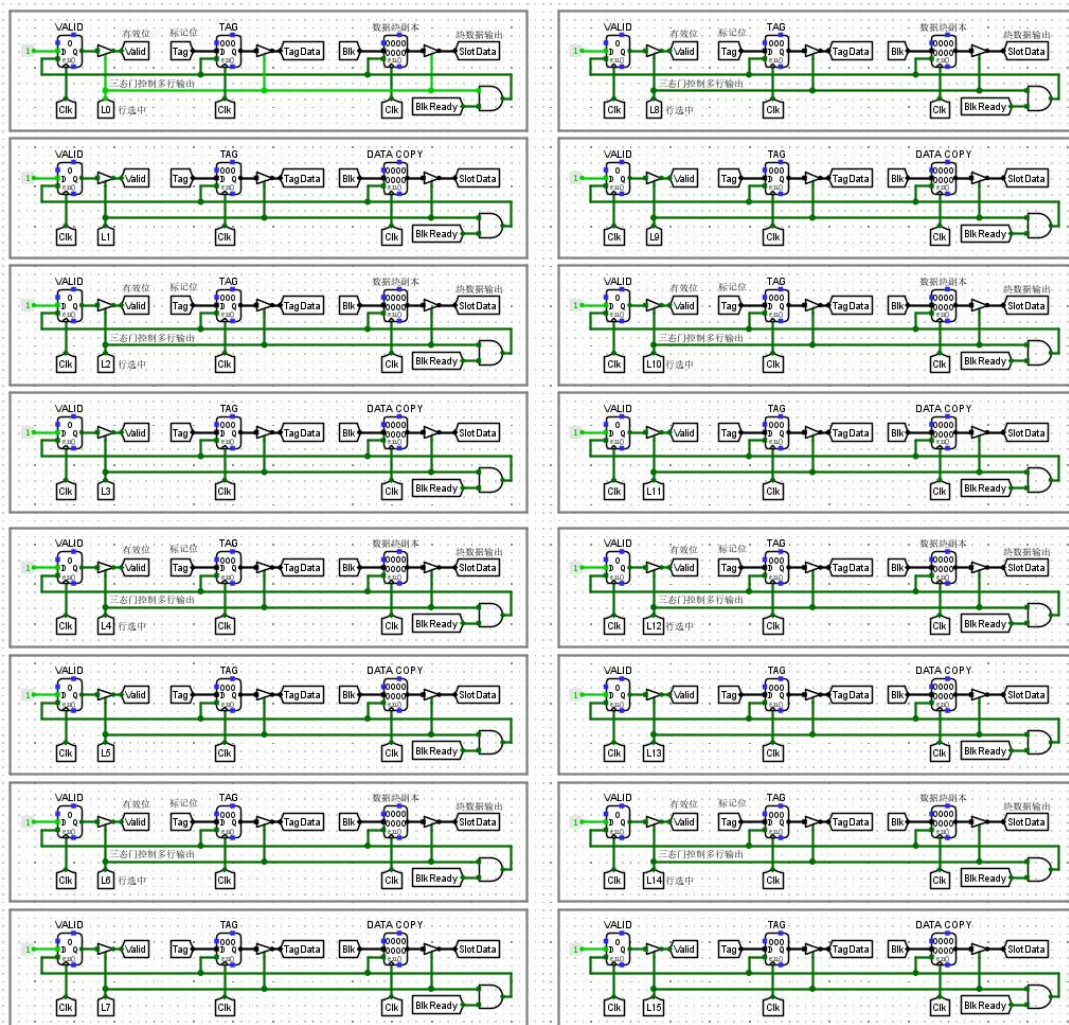


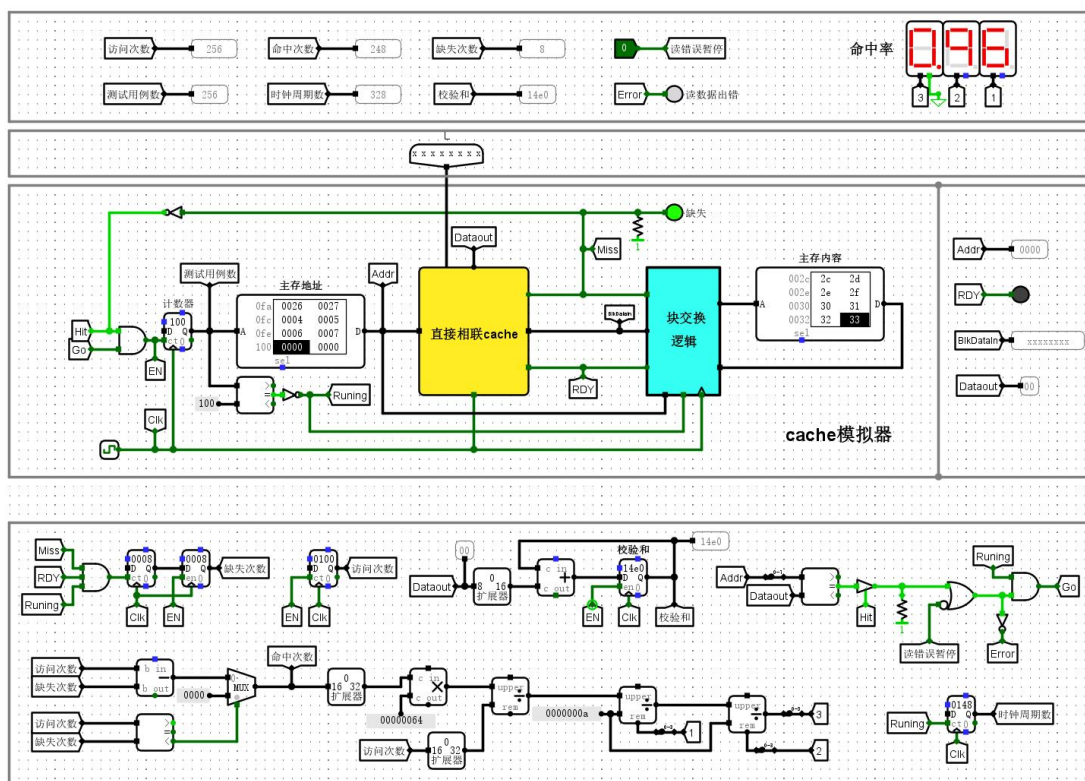


2路组相联映射cache测试电路

直接相联方式 cache 电路（16 个 cache 块）：







直接相联映射cache测试电路